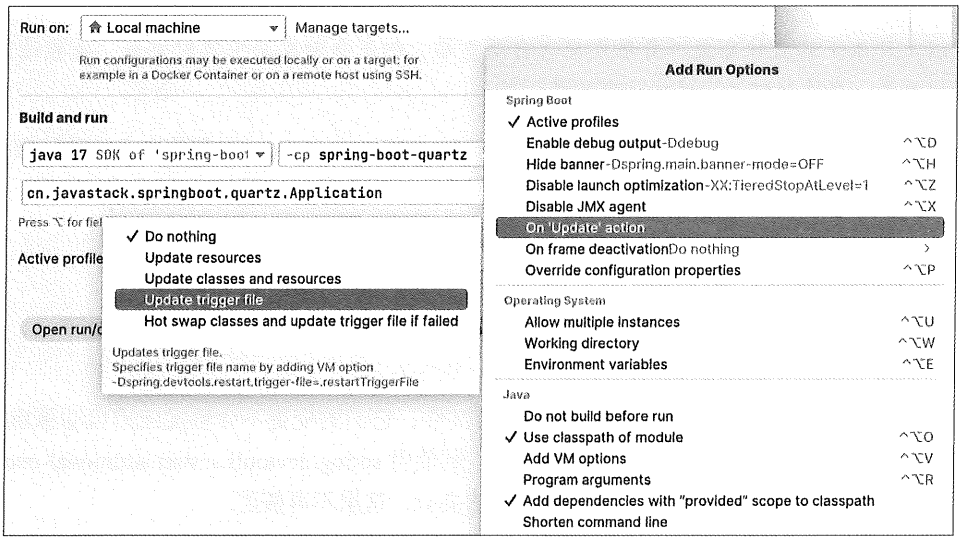
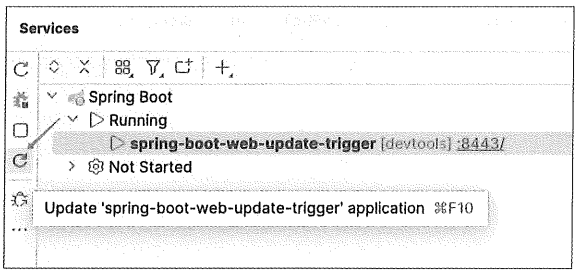
```
restart:
  trigger-file: .reloadtrigger
```

首先在配置文件中指定自动重启的触发文件，然后在资源目录下创建配置中指定的.reloadtrigger触发文件，这时，只有该触发文件被修改（更新）后应用才会自动重启。

另外，并不一定要手动更新这个触发文件，在各种主流的IDE开发工具中都可以设置一个触发动作，比如在IntelliJ IDEA中可以设置运行更新时的动作，设置只更新触发文件，如下图配置所示。



然后在Services面板上会有一个更新的按钮，如下图所示。



不过这个设置只限于使用main方法启动的Spring Boot应用，不支持使用Spring Boot Maven插件方式启动的应用，笔者没有找到相关设置和按钮，需要手动更新触发文件。

3. 排除资源

某些资源是没有必要触发应用自动重启的，默认情况下，以下目录下的资源不会触发自动重启：

- /META-INF/maven。
- /META-INF/resources。
- /resources。
- /static。
- /public。
- /templates。

> 虽然以上目录下的文件变更不会触发应用自动重启，但是会触发应用实时重载，也能达到快速刷新的效果，具体介绍见后续的实时重载一节。

如果不想使用以上默认值，则可以通过 spring.devtools.restart.exclude 参数自定义排除项，如以下配置示例所示。

```
spring:
  devtools:
    restart:
      exclude: static/**,public/**
```

这里只排除 static 和 public 两个目录下的所有文件，它们的变更不会触发应用自动重启。如果要保留所有的默认目录，再添加其他的额外目录，则使用 spring.devtools.restart.additional-exclude 参数即可，这个和配置管理一章中的 Profile 配置用法类似，这里不再赘述。

4. 禁用日志报告

应用自动重启时，每次都会打印变更日志报告，如下图所示。

```
==========================
CONDITION EVALUATION DELTA
==========================


Positive matches:
-----------------

    None


Negative matches:
-----------------

   JacksonHttpMessageConvertersConfiguration.MappingJackson2HttpMessageConverterConfiguration#mappingJackson2H
      Did not match:
         - @ConditionalOnMissingBean (types: org.springframework.http.converter.json.MappingJackson2HttpMessag
```

如果不想应用自动重启时显示报告日志，可以通过配置禁用，如以下配置所示。

```
spring:
  devtools:
    restart:
      log-condition-evaluation-delta: false
```

5. 禁用自动重启

如果需要保留开发者工具的所有功能，但又不想启用应用自动重启功能，则可以通过配置单独禁用自动重启功能，如以下配置所示。

```
spring:
  devtools:
    restart:
      enabled: false
```

这个配置虽然会禁用自动重启及文件监测功能，但还是会初始化自动重启类加载器，自动重启类加载器的存在可能导致一些反序列化方面的问题，比如可能发生以下两个问题：

- 和Java原生ObjectInputStream反序列化类不兼容，可以考虑使用Spring中的ConfigurableObjectInputStream类并结合Thread.currentThread().getContextClassLoader()类加载器。
- 有一些第三方的序列化类库没有考虑上下文类加载器，也可能存在反序列化问题。

所以，如果自动重启类加载器和其他类库存在兼容性问题，则可以在应用启动之前通过系统参数彻底禁用自动重启功能，如以下代码所示。

```
@SpringBootApplication
public class MyApplication {

    public static void main(String[] args) {
        System.setProperty("spring.devtools.restart.enabled", "false");
        SpringApplication.run(MyApplication.class, args);
    }

}
```

10.2.4 实时重载

Spring Boot开发者工具支持应用文件变更后网页实时重载内容，即不需要手动刷新网页，当类

路径下的资源发生变更时可以触发浏览器自动刷新，自动刷新的前提是需要浏览器安装 LiveReload 扩展插件，并开启应用开发者工具中的自动重启功能。

Spring Boot 开发者工具内嵌了一个 LiveReload 服务器，这个可以从应用启动日志中发现，如下图所示。

```
Will secure any request with [org.springframework.security.web.session.DisableEncod
LiveReload server is running on port 35729
Tomcat started on port(s): 8443 (https) 8080 (http) with context path ''
Started Application in 2.785 seconds (process running for 3.162)
Initializing Spring DispatcherServlet 'dispatcherServlet'
Initializing Servlet 'dispatcherServlet'
Completed initialization in 1 ms
```

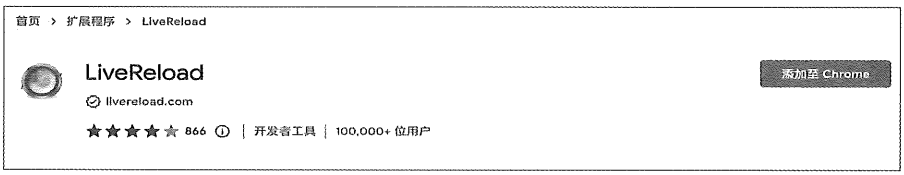
LiveReload 服务器运行在 35729 端口，用来与浏览器插件进行实时交互。

需要注意的是：

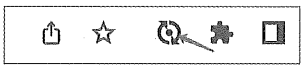
同时只能运行一个 LiveReload 服务器，如果在 IDE 中启动了多个应用，那么只有第一个启动的应用才支持实时重载。

LiveReload 的浏览器扩展插件的下载地址为链接 13。

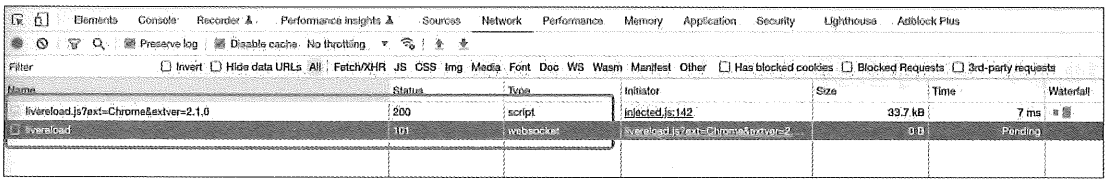
也可以直接去应用商店快速安装，比如在 Chrome 应用商店中搜索“livereload”，然后选择“添加至 Chrome”即可，如下图所示。

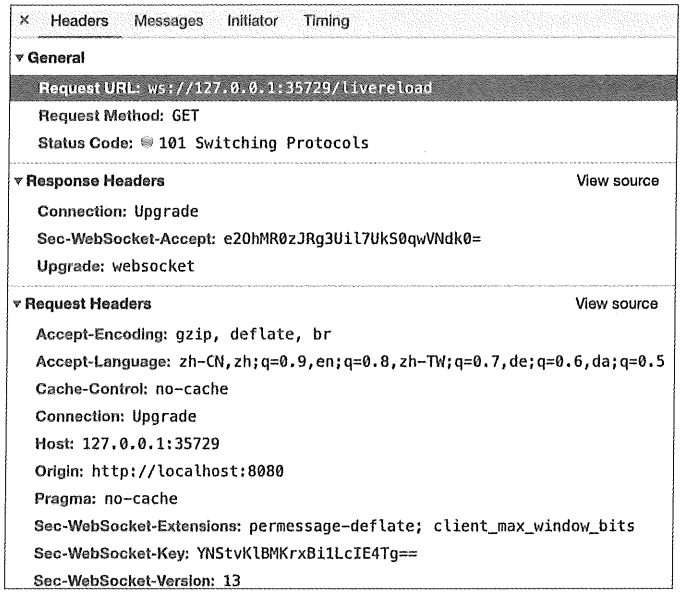


安装完插件之后，在需要自动刷新的网页上面单击 LiveReload 插件图标，让图标从“圆心”变成“实心”状态，如下图所示。



圆心图标表示该网页未开启自动刷新功能，实心图标则表示该网页开启了自动刷新功能，开启自动刷新功能后会发现网络面板出现了 WebSocket 长连接请求，如下面两个图所示。





URL请求地址中的35729便是LiveReload服务器的端口，和LiveReload服务器建立长连接后将实时动态通知浏览器插件，这便是浏览器能自动刷新的原因。随便修改一处代码，自动重启之后网页就会自动刷新。

再回到自动重启一节中提到的以下资源目录：

- /META-INF/maven。
- /META-INF/resources。
- /resources
- /static。
- /public。
- /templates。

前面提到，修改这些目录下的资源虽然默认不会触发应用自动重启，但是会触发应用实时重载，因为开发者工具默认会禁用各种缓存，所以不需要自动重启服务器，这些目录下的资源被修改后，网页就可以实时重载最新的内容。

如果不想开启实时重载，则可以通过配置禁用，如以下配置所示。

```
spring:
  devtools:
```

```
  livereload:
    enabled: false
```

10.2.5 全局配置

在实际开发工作中，同时开发的项目可能会非常多，现在都是微服务多模块开发，一个服务就是一个模块，即一个项目，如果逐个地配置开发者工具会比较麻烦，也不好维护管理，所以开发者工具也支持全局配置，这样所有 Spring Boot 项目都能应用生效。

全局配置文件位于 $HOME/.config/spring-boot 目录下，配置文件名为 spring-boot-devtools，支持 .properties 和 .yaml 两种配置文件类型，可以是以下几种文件后缀：

- spring-boot-devtools.properties。
- spring-boot-devtools.yaml。
- spring-boot-devtools.yml。

$HOME 默认指的是用户的主目录，如果要自定义这个目录，则可以通过以下两种方式设置：

- 设置 SPRING_DEVTOOLS_HOME 环境变量。
- 设置 spring.devtools.home 系统参数。

笔者这里添加了一个 spring-boot-devtools.yml 全局配置文件，禁用自动重启日志报告，以及指定了一个触发文件，如以下配置示例所示。

```
spring:
 devtools:
   restart:
     log-condition-evaluation-delta: false
     trigger-file: ".reloadtrigger"
```

建议使用全局配置文件，这样就不用在每个项目中单独配置开发者工具了。

需要注意的是：

（1）spring-boot-devtools 配置文件不支持任何基于 Profile 的配置。

（2）如果项目同时涉及 Spring Boot 新老版本开发，则可以使用 $HOME/.spring-boot-devtools.properties 配置文件，新老版本可以兼容这种配置文件。

.spring-boot-devtools.properties 配置文件示例如下：

```
spring.devtools.restart.log-condition-evaluation-delta=false
spring.devtools.restart.trigger-file=.reloadtrigger
```

注意：.properties 配置不能带双引号。

10.3 单元测试

10.3.1 概述

Spring Boot 提供了丰富的测试功能，主要由以下两个模块组成：

- spring-boot-test：提供测试核心功能。
- spring-boot-test-autoconfigure：提供对测试的自动配置。

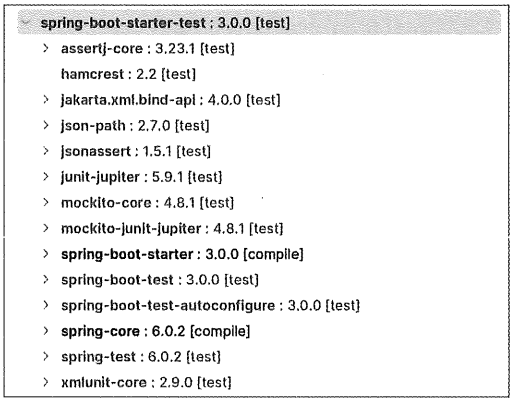
Spring Boot 提供了一个 spring-boot-starter-test 一站式启动器，如以下依赖配置所示。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

测试启动器依赖不仅包含以上两个 Spring Boot 模块，还包含 Spring Test 测试模块，以及其他第三方测试类库，如下所示。

- JUnit 5：Java 最主流的单元测试框架。
- AssertJ：一款快速断言库。
- Hamcrest：一款单元测试匹配库。
- Mockito：一款 Mock 测试框架。
- JSONassert：一款 JSON 断言库。
- JsonPath：一款 JSON XPath 库。

更多测试相关的依赖可见具体的依赖关系树，如下图所示。



以上这些都是 Spring Boot 提供的常用的测试类库，如果上面的测试类库还不能满足你的需要，也可以任意添加以上没有的类库。

现在基本上使用的是 JUnit 5，如果应用还在使用 JUnit 4 写的单元测试用例，那么也可以使用 JUnit 5 的 Vintage 引擎来运行，如下面的依赖配置所示。

```
<dependency>
    <groupId>org.junit.vintage</groupId>
    <artifactId>junit-vintage-engine</artifactId>
    <scope>test</scope>
    <exclusions>
        <exclusion>
            <groupId>org.hamcrest</groupId>
            <artifactId>hamcrest-core</artifactId>
        </exclusion>
    </exclusions>
</dependency>
```

需要排除 hamcrest-core 依赖，因为该依赖已经改坐标了，并且默认内置在 Spring Boot 依赖管理中，如上面的依赖关系树所示，最新的 Hamcrest 依赖已经是 org.hamcrest:hamcrest 坐标了。

Spring Boot提供了一个 @SpringBootTest 注解，用在单元测试类上以启用支持 Spring Boot 特性的单元测试，如果使用的是 JUnit 4，那么测试类上还需要额外的 @RunWith(SpringRunner. class) 注解，然后在测试类方法上添加 @Test 注解即可，每一个 @Test 注解修饰的方法就是一个单元测试方法。

@SpringBootTest 注解有一个最重要的 webEnvironment 环境参数，支持以下几种环境设置：

- MOCK（默认）：加载一个 Web ApplicationContext 并提供一个 Mock Web Environment，但不会启动内嵌的 Web 服务器，并可以结合 @AutoConfigureMockMvcor 和 @AutoConfigure-WebTestClient 注解一起使用进行 Mock 测试。

- RANDOM_PORT：加载一个 WebServerApplicationContext，以及提供一个真实的 Web Environment，并以随机端口启动内嵌服务器。
- DEFINED_PORT：和 RANDOM_PORT 一样，不同的是 DEFINED_PORT 是以应用指定的端口运行的，默认端口为 8080。
- NONE：加载一个 ApplicationContext，但不会提供任何 Web Environment。

如果使用的 @SpringBootTest 注解不带任何参数，则默认为 Mock 环境。

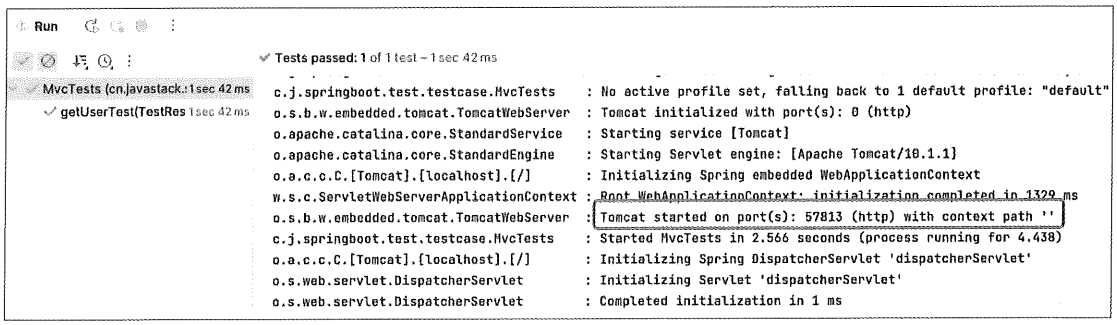
10.3.2 真实环境测试

在 @SpringBootTest 注解中指定基于随机端口的真实 Web 环境，然后在类成员变量或者方法参数上注入 TestRestTemplate 实例，就可以完成对 Spring MVC 接口的真实环境测试。

下面是一个基于随机端口的真实环境的测试用例：

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
public class MvcTest {

    @Test
    public void getUserTest(@Autowired TestRestTemplate testRestTemplate) {
        Map<String, String> multiValueMap = new HashMap<>();
        multiValueMap.put("username", "Java 技术栈 ");
        Result result = testRestTemplate.getForObject("/user/get?username={username}",
                Result.class, multiValueMap);
        assertThat(result.getCode()).isEqualTo(0);
        assertThat(result.getMsg()).isEqualTo("ok");
    }

}
```

测试当前应用下的 /user/get 接口，传入对应的用户名参数，最后检查接口返回结果是否和预期一致，测试结果如下图所示。



单元测试通过，从执行日志可以看到，它启动了一个嵌入式的 Tomcat 容器来测试真实的 Web 应用环境。

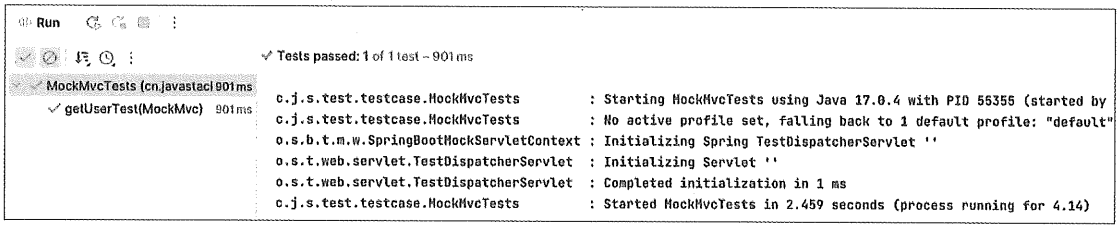
10.3.3 Mock 环境测试

通过在类上面使用 @AutoConfigureMockMvc 注解，然后在类成员变量或者方法参数上注入 MockMvc 实例，就可以完成对 Spring MVC 接口的 Mock 测试。

下面是一个基于默认 Mock 环境的测试用例：

```
@SpringBootTest
@AutoConfigureMockMvc
class MockMvcTests {

    @Test
    public void getUserTest(@Autowired MockMvc mvc) throws Exception {
        mvc.perform(MockMvcRequestBuilders.get("/user/get?username={username}", "test"))
                .andExpect(status().isOk())
                .andExpect(content().string("{\"code\":0,\"msg\":\"ok\",\"
data\":\"test\"}"));
    }

}
```

测试当前应用下的 /user/get 接口，传入对应的用户名参数，最后检查请求状态是否 OK（200），响应的内容是否和预期一致，测试结果如下图所示。



单元测试通过，从执行日志可以看到，它并未启动真实的 Web 环境来测试，而是使用 Mock 环境测试的。

10.3.4 Mock 组件测试

某些时候可能还需要模拟一些组件，比如某些服务只有上线之后才能调用，在开发阶段不可用，这时就需要 Mock 模拟测试了，提供各种模拟组件以完成测试。

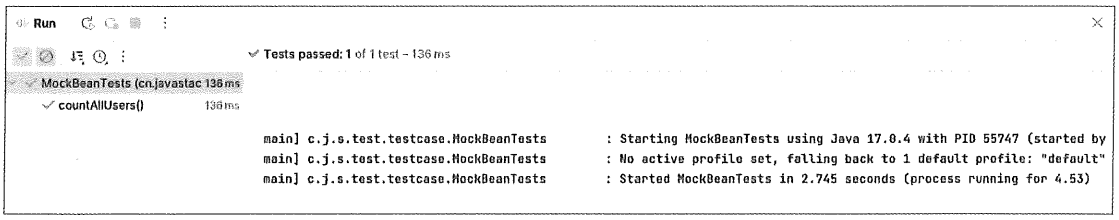
Spring Boot 提供了一个 @MockBean 注解，可为 Spring 中的 Bean 组件定义基于 Mockito 的 Mock 测试，它可以创建一个新 Bean 以覆盖 Spring 环境中已有的 Bean，它可以用在测试类、成员变量上，或者 @Configuration 配置类、成员变量上，被模拟的 Bean 在每次测试结束后自动重置。

假现现在有一个远程的服务 userService，本地不能调用，现在进行 Mock 测试，如以下使用示例所示。

```
@SpringBootTest
class MockBeanTests {

//    @Autowired
//    private UserService userService;

    @MockBean
    private UserService userService;

    @Test
    public void countAllUsers() {
        BDDMockito.given(this.userService.countAllUsers()).willReturn(88);
        assertThat(this.userService.countAllUsers()).isEqualTo(88);
    }

}
```

这里的 @MockBean 注解使用在 UserService 变量上，表明这个 userService 实例在当前测试用例中是被 Mock 覆盖的，如果要模拟的 Bean 有多个，则可以使用 @Qualifier 注解指定，然后通过 Mockito 提供的代理工具类方法创建模拟返回数据，运行该服务的测试方法，当模拟数据和预期结果一致时才会测试通过。

这里通过 BDDMockito 工具类模拟 userService#countAllUsers 方法并让它返回统计的用户总数（88），最后检查该方法的返回值是否和预期一致，测试结果如下图所示。



单元测试通过，也可以使用 @SpyBean 注解代替 @MockBean 注解，两者的区别是：

- @SpyBean—如果没有提供 Mockito 代理方法，则会调用真实的 Bean 来获取数据。

- @MockBean—不管有没有提供 Mockito 代理方法，都会调用 Mock 的 Bean 来获取数据。

@MockBean、@SpyBean 注解既可作用于 Mock 环境，也可作用于真实环境，它只是用来模拟、替换环境中指定的 Bean 而已，但不能用于模拟在应用上下文刷新期间 Bean 的行为，因为在执行测试用例时应用上下文已经刷新完成了，所以不可能再去模拟了，这种情况下建议使用 @Bean 方法来创建模拟配置。

10.3.5 技术框架测试

除了支持 Spring Boot 环境的单元测试，Spring Boot 还提供了各种技术组件的单元测试配置，比如 JSON、Spring MVC、WebFlux、Spring Data JAP、Spring Data JDBC、Spring Data Redis 等。

比如想测试 JSON 序列化 / 反序列化功能，其自动配置类为 JsonTestersAutoConfiguration，它提供了对各个 JSON 框架的测试支持，只需要在单元测试类上添加一个 @JsonTest 注解，注入对应的 JacksonTester、GsonTester、JsonbTester 实例就可以直接测试。

首先在资源目录下创建 jack.json 文件，JSON 文件的内容如下所示。

```
{"id":10001, "name":"Jack", "birthday": "2000-10-08 21:00:00"}
```

以下是使用 Jackson 序列化 / 反序列化的测试示例：

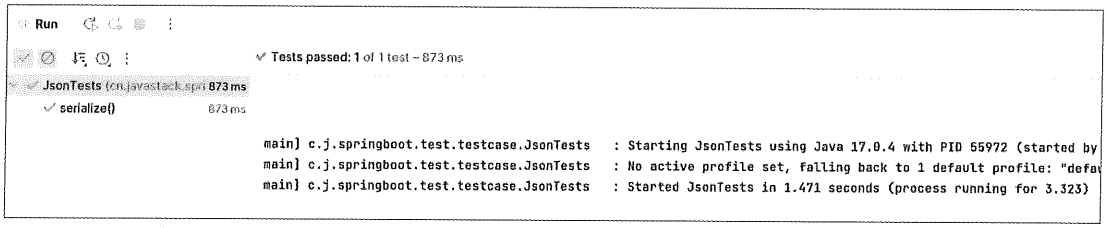
```
@JsonTest
class JsonTests {

    @Autowired
    private JacksonTester<User> json;

    @Test
    void serialize() throws Exception {
        User user = new User(10001L, "Jack",
                LocalDateTime.of(2000, 10, 8, 21, 0, 0));
        System.out.println(this.json.write(user));
        assertThat(this.json.write(user)).isEqualToJson("/jack.json");
        assertThat(this.json.write(user)).hasJsonPathStringValue("@.name");
        assertThat(this.json.write(user)).
                extractingJsonPathStringValue("@.name").isEqualTo("Jack");
    }

    @Test
    void deserialize() throws Exception {
```

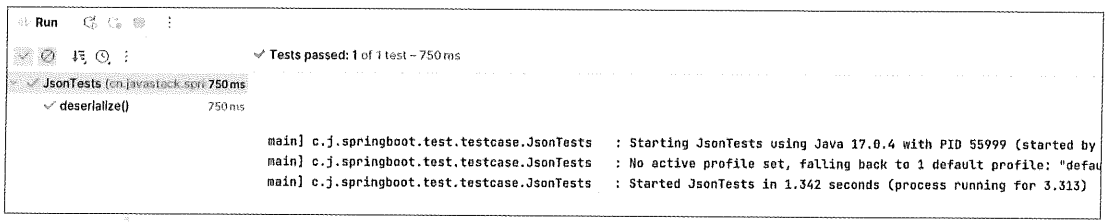
```
        String content = "{\"id\":10002, \"name\":\"Petty\", \"birthday\":
\"2021-01-21T02:32:00\"}";
        assertThat(this.json.parse(content))
                .isEqualTo(new User(10002L, "Petty",
                        LocalDateTime.of(2021, 1, 21, 2, 32, 0)));
        assertThat(this.json.parseObject(content).getName()).isEqualTo("Petty");
    }

}
```

这里提供了两个测试方法：

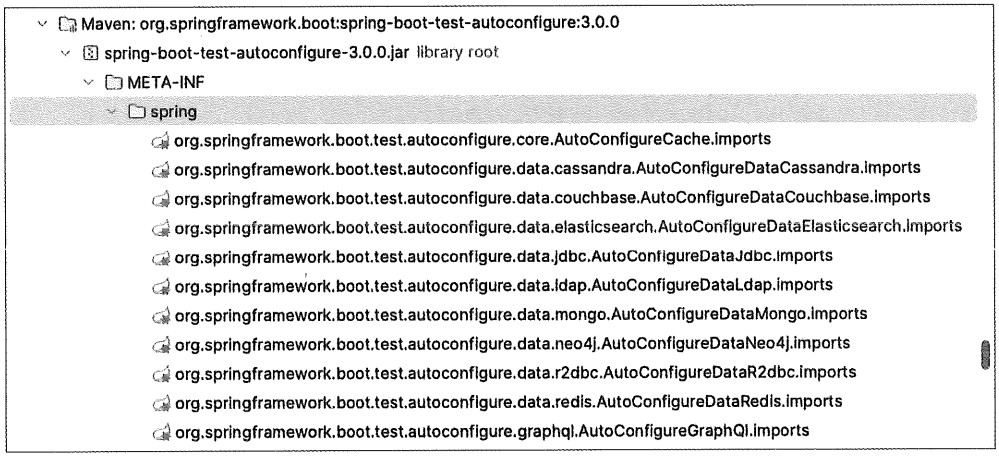
- serialize：测试 JSON 序列化功能，判断序列化后的内容和资源目录下的 jack.json 文件的内容是否一致，是否有 name 这个字段，并且该字段值是否为 Jack。
- deserialize：测试 JSON 反序列化功能，判断反序列化后的用户对象和新创建的用户对象是否一致，并且反序列化后的用户对象的 name 值是否为 Petty。

测试结果如下面的两个图所示。

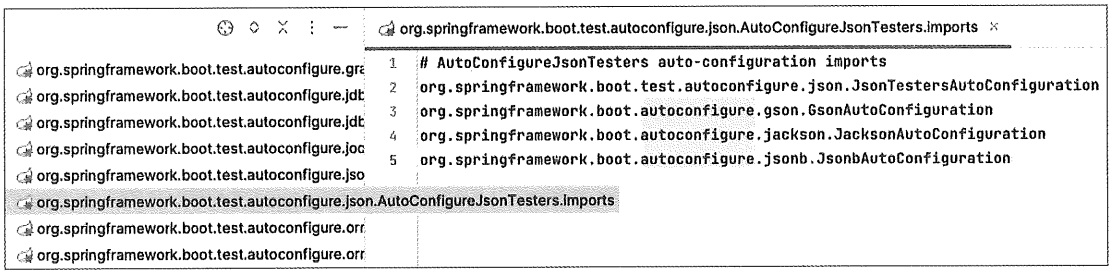




更多技术框架的自动配置可以参考 spring-boot-test-autoconfigure 测试自动配置包中的自动配置文件，如下图所示。



比如上面测试的 JSON 序列化和反序列化功能，它注册了 Spring Boot 所支持的几种 JSON 工具类的自动配置类，如下图所示。



这里可以看到 JSON 测试支持的自动配置类 JsonTestersAutoConfiguration，以及 JSON 框架 Jackson、Gson、Jsonb 的自动配置类都在此处进行了注册。

第 11 章
Spring Boot 打包与部署

Spring Boot 应用部署到生产环境中的方式多种多样，既支持嵌入式容器直接运行应用，也支持将应用部署到外置的 Servlet 容器中。当然，重点是嵌入式容器直接运行应用的方式，这也是 Spring Boot 的核心特性，这种方式可以直接使用 java 命令运行应用，也可以免去 java 命令直接像运行系统脚本一样运行应用，或者直接以操作系统服务运行应用，既可以整包运行应用，也可以拆包运行应用，还可以将应用生成 Docker 镜像部署到 Docker 容器中，或者将应用部署到云平台中，Spring Boot 3.0 还支持以 GraalVM 原生镜像运行应用，可以说几乎满足所有的部署场景的需要。本章从应用打包开始，讲解上述涉及的所有内容。

11.1 应用打包（jar）

11.1.1 概述

传统的 Java 应用都需要打一个 war 包，然后把它部署到 tomcat/webapps 目录下运行，虽然 Java 支持把应用打成 jar 包，但它并没有提供一个标准的方式把工程代码打包成可执行的 Java Web 应用，因为它没有提供内置的 Servlet 容器。

有了 Spring Boot 框架之后，这一切都变得简单，开发者可以很方便地将 Spring Boot 应用打成一个可执行 jar 包部署，同时支持打成 war 包部署，但 Spring Boot 首推以 jar 的形式打包和发布应用，因为使用 jar 包部署比较灵活，方便应用快速启动和扩容。

11.1.2 快速打包

基于 Maven 的应用，如果不指定 packaging 属性，那么默认就是 jar 打包方式，如以下配置所示。

```
<packaging>jar</packaging>
```

packaging 默认就是 jar 类型，可以省略不用指定。另外还需要引入 spring-boot-maven-plugin 打包插件，如以下配置所示。

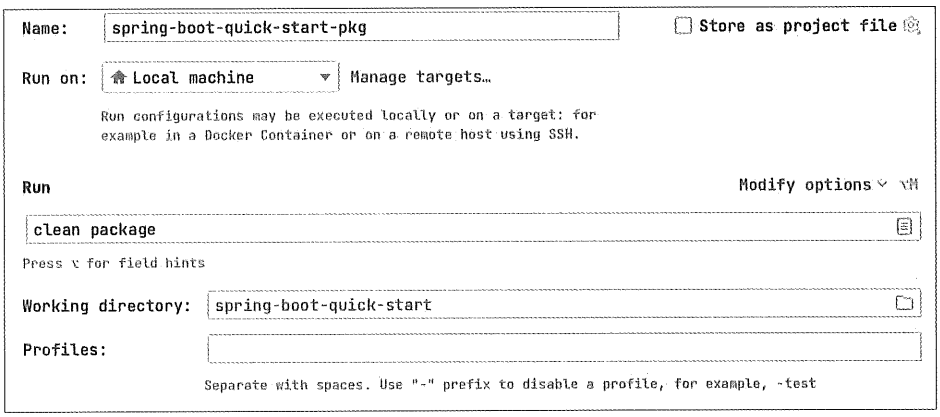
```
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

然后使用 mvn package 或者 mvn install 命令打包，如以下参考命令所示。

```
$ mvn package
$ mvn install
```

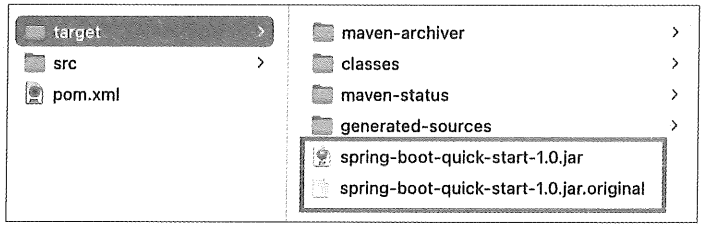
两个命令二选一即可，前者 package 是只打包到应用的 target 目录下，后者 install 是同时打包到应用的 target 目录下并安装到本地 Maven 仓库中，如果是打包为可执行的应用，则建议使用 package 命令。

在开发工具中可以省略 mvn 命令的前缀，如下图所示。



打包命令一般可以结合 clean 使用，即先清理之前的包再打包新的包，执行打包命令后会在当

前应用 target 目录下生成两个 jar 包，如下图所示。



应用先打包为 spring-boot-quick-start-1.0.jar 原始 jar 包，包含所有依赖项，它是不能直接运行的，再通过 spring-boot-maven-plugin 插件中的 repackage 目标重新打包为可执行的 jar 包，然后重命名原始 jar 包为 *.jar.original。

第一个 jar 包（spring-boot-quick-start-1.0.jar）是可以直接运行的 jar 包，可以通过 java 命令运行，如以下运行命令所示。

```
$ java -jar spring-boot-quick-start-1.0.jar
```

这个 jar 包包括了所有编译完的 class 类文件，还内置了 Servlet 容器，以及所有需要依赖的其他 jar 包等，这个 jar 包的目录结构如下：

```
.
├── BOOT-INF
│   ├── classes
│   │   └── cn
│   │       └── javastack
│   │           └── springboot
│   │               └── quckstart
│   └── lib
├── META-INF
│   └── maven
│       └── cn.javastack
│           └── spring-boot-quick-start
└── org
    └── springframework
        └── boot
            └── loader
                ├── archive
                ├── data
                ├── jar
                ├── jarmode
                └── util
```

在 META-INF 目录下可以找到 MANIFEST.MF 描述文件，内容如下所示。

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.3.0
Build-Jdk-Spec: 17
Implementation-Title: spring-boot-quick-start
Implementation-Version: 1.0
Main-Class: org.springframework.boot.loader.JarLauncher
Start-Class: cn.javastack.springboot.quckstart.Application
Spring-Boot-Version: 3.0.0
Spring-Boot-Classes: BOOT-INF/classes/
Spring-Boot-Lib: BOOT-INF/lib/
Spring-Boot-Classpath-Index: BOOT-INF/classpath.idx
Spring-Boot-Layers-Index: BOOT-INF/layers.idx
```

它包含了一个 jar 包的相关信息，从中就可以找到应用的引导类和启动类，这个在第 4 章 Spring Boot 启动过程分析中已有详细介绍，这里不再赘述。

第二个 jar 包是包含了所有代码和资源文件的普通 jar 包，这个 jar 包的目录结构如下所示。

```
.
├── cn
│   └── javastack
│       └── springboot
│           └── quckstart
└── META-INF
    └── maven
        └── cn.javastack
            └── spring-boot-quick-start
```

再来看其 META-INF 目录下的 MANIFEST.MF 描述文件内容：

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.3.0
Build-Jdk-Spec: 17
Implementation-Title: spring-boot-quick-start
Implementation-Version: 1.0
```

它没有包含应用启动入口类，只是作为一个普通的 jar 包存在，不能使用 java 命令运行，一般作为依赖被传递到其他应用中。不是所有 Spring Boot 应用都需要部署运行，比如一些通用模块的应用，它们只是作为普通 jar 包集成到可执行的 jar 包中使用而已。

11.1.3 自定义打包

如果应用使用的是继承 spring-boot-starter-parent 父依赖的方式，那么它提供了 spring-boot-maven-plugin 打包插件的基本配置，如以下配置所示。

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <executions>
    <execution>
      <id>repackage</id>
      <goals>
        <goal>repackage</goal>
      </goals>
    </execution>
  </executions>
  <configuration>
    <mainClass>${start-class}</mainClass>
  </configuration>
</plugin>
```

它默认提供了 repackage 打包目标、配置应用启动类，所以应用可以直接继承父项目提供的 spring-boot-maven-plugin 插件，不用做任何配置，只需要坐标即可，如下面的配置示例所示。

```
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

如果有需要，还可以定制配置，常用的定制配置参数如下：

（1）layout：指定打包的项目类型，根据应用原有类型默认打包为 jar、war 包，具体可以设置以下几种。

- JAR：可执行的 jar 包。
- WAR：可执行的 war 包。
- ZIP：和 jar 包相似。

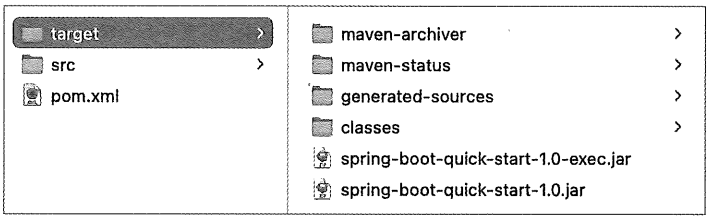
- NONE：打包所有依赖项和项目资源，但不绑定任何启动加载器。

（2）mainClass：指定 Spring Boot 的启动类，如果启动类在根目录下则不用指定。

（3）classifier：指定可执行 jar 包的扩展标识符，比如下面的示例，对 jar 包名称扩展标识符进行定制。

```
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <classifier>exec</classifier>
            </configuration>
        </plugin>
    </plugins>
</build>
```

重新打包后，再看一下 jar 包的名称，如下图所示。



如上图所示，可执行 jar 包带上了插件配置的 exec 标识符，有了这个标识符，就可以用来区分是可执行的 jar 包，还是普通的 jar 包。

这就是继承 spring-boot-starter-parent 父依赖的好处，如果使用导入 spring-boot-dependencies 依赖的集成方式，则需要在应用中提供插件的一整套配置，如以下配置示例所示。

```
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <executions>
              <execution>
                <id>repackage</id>
                <goals>
```

```
                <goal>repackage</goal>
            </goals>
        </execution>
    </executions>
    <configuration>
        <mainClass>${start-class}</mainClass>
        <classifier>exec</classifier>
    </configuration>
</plugin>
</plugins>
</build>
```

所以，应用选用哪种 Spring Boot 集成方式很重要，如果打包失败，则需要检查应用使用的是哪种集成方式。

11.2 应用打包（war）

11.2.1 概述

Spring Boot 也支持将应用打包成 war 包，这样应用就不会内置 Servlet 容器，也不能直接运行，需要部署到外部的 Servlet 容器中。另外，基于 WebFlux 的响应式 Web 应用不会严格依赖 Servlet API，并且应用默认嵌入在 Reactor Netty 服务器中部署，所以 WebFlux 应用不支持 war 包部署。

应用打包成 war 包的好处是，可以让运维人员用熟悉的方式来统一管理和维护外置的 Servlet 容器，从而降低了容器管理的复杂性，如果是微服务，那么建议打包为 jar 包，方便快速动态扩容。

11.2.2 配置 war 包

1. 继承 SpringBootServletInitializer 抽象类

配置 war 包的第一步是继承 SpringBootServletInitializer 抽象类，并重写其 configure 配置方法，如以下示例所示。

```
@SpringBootApplication
public class Application extends SpringBootServletInitializer {

    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder application) {
```

```
        return application.sources(Application.class);
    }

}
```

2. 修改打包类型

在 Maven pom.xml 配置文件中，修改打包类型为 war，如以下配置所示。

```
<packaging>war</packaging>
```

3. 添加插件

如果是继承 pring-boot-starter-parent 父依赖的集成方式，那么 Spring Boot 也提供了一个 maven-war-plugin 插件，用于打包 war 应用，如以下配置所示。

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-war-plugin</artifactId>
  <configuration>
    <archive>
      <manifest>
        <mainClass>${start-class}</mainClass>
        <addDefaultImplementationEntries>true</addDefaultImplementationEntries>
      </manifest>
    </archive>
  </configuration>
</plugin>
```

同理，继承的插件都提供了默认配置，在应用中只需要引入坐标即可，如以下配置所示。

```
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-war-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

所以，如果使用导入 spring-boot-dependencies 依赖的集成方式，则需要在应用中提供插件的一整套配置。

4. 排除嵌入式容器

Spring Boot 默认使用的是 Tomcat 嵌入式容器，打包成 war 包需要排除嵌入式容器，以免应用部署到外部 Tomcat 时发生冲突，排除配置如下所示。

```
<dependencies>
    <!-- 打 war 包后不携带嵌入式 Tomcat 容器 -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-tomcat</artifactId>
        <scope>provided</scope>
    </dependency>
</dependencies>
```

因为在编译启动类时，继承的 SpringBootServletInitializer 类还是会用到 Servlet 相关的依赖，所以把嵌入式 Tomcat 依赖的 scope 范围设置为 provided，表示该依赖只在编译和测试时才有效，但不会出现在外部 Servlet 的 war 包中，因为外部 Tomcat 自身会提供相关类库。

5. 兼容嵌入式容器（可选）

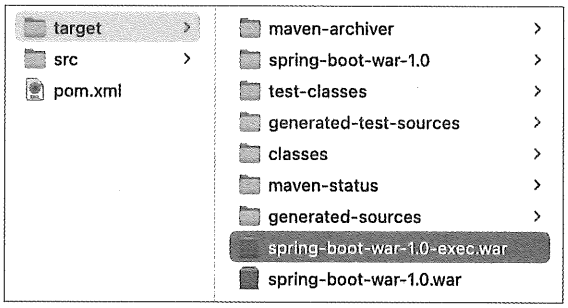
如果既要支持将 war 包部署到外部 Servlet 容器中，又要支持直接运行 war 包，则可以保留应用启动类中的 main 方法，并把公共启动方法抽出来，然后两边同时调用公共方法，如下面的示例所示。

```
@SpringBootApplication
@RestController
public class Application extends SpringBootServletInitializer {

    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder builder) {
        return startBuilder(builder);
    }

    public static void main(String[] args) {
        startBuilder(new SpringApplicationBuilder()).run(args);
    }

    private static SpringApplicationBuilder startBuilder(SpringApplicationBuilder builder) {
        return builder.sources(Application.class);
    }

}
```

同时添加 Spring Boot 的 spring-boot-maven-plugin 打包插件，这个插件是专门用来打 jar 包的，虽然上面把嵌入式容器排除了，但只是在外部 Servlet 的 war 包中排除了，此插件会再生成一个可以直接运行的 war 包，排除的嵌入式容器会以 lib-provided 目录出现在可以直接运行的 war 包中，这样就可以同时兼容嵌入式容器和外部容器。

11.2.3 开始打包

在开发工具中使用 clean packge 命令将应用打包后，前往应用的 target 目录，如下图所示。



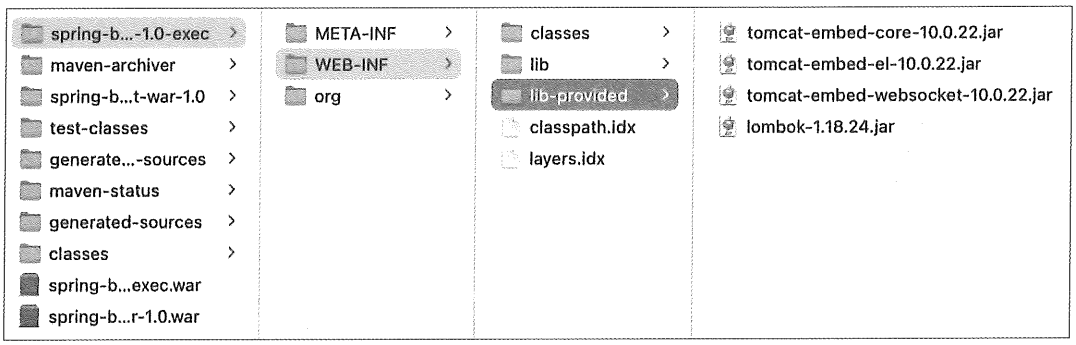
这里出现了两个 war 包，第一个 war 包（spring-boot-war-1.0-exec.war）是可以同时部署 jar 和 war 二合一的包，如果不配置兼容嵌入式容器，则第一个 war 包就不会生成。

这个 war 包的目录结构如下：

```
.
├── META-INF
│   └── maven
│       └── cn.javastack
│           └── spring-boot-war
├── org
│   └── springframework
│       └── boot
│           └── loader
│               ├── archive
│               ├── data
│               ├── jar
│               ├── jarmode
│               └── util
└── WEB-INF
    ├── classes
```

```
|  └── cn
|      └── javastack
├── lib
└── lib-provided
```

provided 范围的依赖会放到 lib-provided 目录下，如下图所示。



然后通过后面两个 idx 索引文件在 MANIFEST.MF 描述文件中指定相关文件：

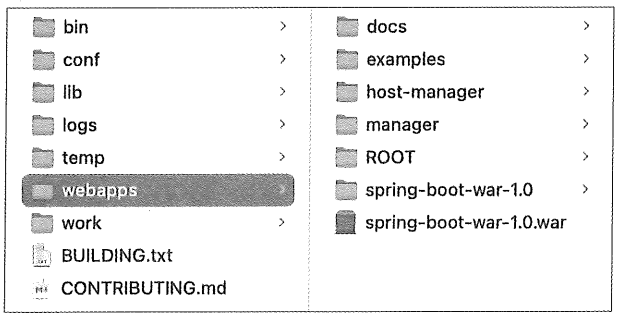
- classpath.idx：引入了一系列 jar 包的 classpath 索引文件。
- layers.idx：分层索引文件。

这两个索引文件只有在打可直接运行的包时才会出现，这个 war 包既包含了直接运行 Spring Boot 应用的所有文件，包括嵌入式容器相关包（lib-provided），又包含了部署到外部 Servlet 容器中的所有文件，所以既可以像之前运行 jar 包一样直接运行可执行的 war 包，又可以作为 war 包部署到其他 Servlet 容器中。

比如直接运行这个 war 包，参考命令如下：

```
$ java -jar spring-boot-war-1.0-exec.war
```

这个 war包也可以直接放到外部 Servlet 容器中运行，比如放在 Tomcat 中的 webapps 目录下，启动 Tomcat 就可以自动解压缩并部署应用了，如下图所示。



第二个 war包（spring-boot-war-1.0.war）便是仅部署到外部 Servlet 容器中的 war 包，如果不配置兼容嵌入式容器，那么只会生成这个 war 包。

这个 war 包的目录结构如下：

```
.
├── META-INF
│   └── maven
│       └── cn.javastack
│           └── spring-boot-war
└── WEB-INF
    ├── classes
    │   └── cn
    │       └── javastack
    └── lib
```

它仅包含了部署到外部 Servlet 容器中的类库，和第一个 war 包一样直接放到外部 Servlet 容器中运行，但是不能使用 java -jar 命令直接运行。

如果把 war包部署到外部 Servlet 容器中，那么 Spring Boot 应用配置文件中的 server.* 参数就不会生效了，相关参数需要在外部容器中配置。

11.3 应用运行（嵌入式容器）

11.3.1 使用 java 命令运行

前面的章节中介绍过，Spring Boot 嵌入式容器的 jar/war 包一般是通过 java -jar 命令运行的，如以下参考命令所示。

```
$ java -jar spring-boot-quick-start-1.0-exec.jar
```

关于java运行命令的更多用法，比如怎么设置 JVM 参数、传递应用参数等，可以参考相关文档，这里不再赘述。

11.3.2 直接运行

Spring Boot 也支持像运行操作系统脚本一样直接运行嵌入式容器的 jar/war 包，如下面的运行命令所示。

```
$ ./spring-boot-quick-start-1.0-exec.jar
```

这种直接运行方式需要在 spring-boot-maven-plugin 打包插件中指定 executable 配置参数为 true，如下面的配置所示。

```
<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <configuration>
        <executable>true</executable>
    </configuration>
</plugin>
```

这样打出来的包既可以使用 java -jar 命令运行，也可以直接运行，它实际上就是一种 zip 格式的 jar 包，打包的时候在 zip 文件的最前面打入一个额外的脚本，这样 Linux 操作系统就会把它当作一个脚本来运行，推荐使用这种打包方式，像运行系统脚本一样运行应用包，更简单、更方便。

打包时默认嵌入的脚本支持大部分 Linux 操作系统，比如 CentOS、Ubuntu，笔者在 macOS Monterey 操作系统上测试了也是可行的，其他像 macOS、FreeBSD 等不支持的操作系统需要在 spring-boot-maven-plugin 插件中配置自定义的 embeddedLaunchScript 脚本，配置格式如下所示。

```
<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <configuration>
        <classifier>exec</classifier>
        <executable>true</executable>
        <embeddedLaunchScript>...</embeddedLaunchScript>
    </configuration>
</plugin>
```

11.3.3 系统服务运行

Spring Boot 可执行的应用包可以使用 init.d 或 systemd 轻松启动并作为 UNIX/Linux 中的服务，即以 Linux 中的服务启动应用，如以下参考命令所示。

```
$ service mybatis start
$ systemctl start mybatis
```

1.init.d

init.d服务在 /etc/init.d 目录下，只需要在该目录下创建一个 Spring Boot 应用的软链接即可启动

init.d 服务，参考命令如下所示。

```
$ sudo ln -s ~/spring-boot-mybatis-1.0-exec.jar /etc/init.d/mybatis
```

软链接创建成功后就可以运行 init.d 服务了，参考命令如下所示。

```
$ sudo service mybatis start
```

2. systemd

因为 init.d 服务是串行启动的，启动时间较长，所以 systemd 是用来取代 init.d 服务的，systemd 的功能也比 init.d 更强大，systemd 现在已经成为各大 Linux 系统的标配。

systemd 服务在 /etc/systemd/system 目录下，只需要在该目录下创建一个 Spring Boot 应用的脚本文件即可启动 systemd 服务，文件格式为 xx.service，比如下面创建一个 mybatis.service 服务的配置文件，参考内容如下所示。

```
[Unit]
Description=mybatis
After=syslog.target

[Service]
User=test
ExecStart=/home/test/spring-boot-mybatis-1.0-exec.jar
SuccessExitStatus=143

[Install]
WantedBy=multi-user.target
```

然后启动服务，参考命令如下：

```
$ sudo systemctl start mybatis
```

将 systemd 服务设置为开机自动启动，参考命令如下：

```
$ sudo systemctl enable mybatis
```

更多的服务操作指令可以参考下表。

服务操作	init.d	systemd
启动服务	service xxx start	systemctl start xxx
停止服务	service xxx stop	systemctl stop xxx
重新加载配置	service xxx force-reload	systemctl daemon-reload
重启服务	service xxx restart	systemctl restart xxx

续表

服务开机启动	chkconfig --level 5 xxx on	systemctl enable xxx
服务开机启动（取消）	chkconfig --level 5 xxx off	systemctl disable xxx
查询所有开机启动服务	chkconfig --list	systemctl list-unit-files | grep enabled
查询服务是否开机启动	chkconfig --list | grep xxx	systemctl is-enabled xxx
查询所有启动失败的服务		systemctl --failed
检查服务状态	service xxx status	systemctl status xxx
删除服务	chkconfig --del xxx	删除对应的配置文件

通过 Linux 中的 init.d 和 systemd 服务可以实现优雅地部署和管理 Spring Boot 应用服务，并且不用担心系统重启而导致应用长时间终止运行。

11.3.4 拆包运行

Spring Boot 除了支持整包运行，还支持拆包运行，即先解压压缩包文件，再执行解压出来的启动类。

首先解压可执行应用包，参考命令如下：

```
$ jar -xf myapp.jar
```

然后调用 java 命令运行 Spring Boot 提供的 JarLauncher 类，参考命令如下：

```
$ java org.springframework.boot.loader.JarLauncher
```

也可以直接运行应用的原生启动类，参考命令如下：

```
$ java -cp BOOT-INF/classes:BOOT-INF/lib/* cn.javastack.mybatis.Application
```

前者可以自动检测应用的启动类，后者则是直接运行启动类，前者更实用，后者的启动时间可能要更短一点。但无论如何，拆包运行方式都要比整包运行方式快，快多少取决于整包的大小，其他方面并没什么差异，不会影响应用的正常运行。

11.4 部署 Docker 容器

11.4.1 概述

Docker 是现在最主流的开源容器，遵循 Apache 2.0 协议，使用 Go 语言开发，始于 2013 年，现已加入 Linux 基金会。Docker 的核心思想是：Build once, Run anywhere，即一次构建，处处运行—

可以是一个操作系统、一个 Web 应用，或者一个数据库服务等。

Docker 环境主要包含镜像和容器，镜像只是一个只读的静态文件，不能运行，容器才是镜像的运行实例，所以需要新增一个容器。我们可以将 Docker 理解为一个轻量级的沙盒，Docker 容器完全使用沙盒机制，每个容器内都有其独立运行的环境和组件，容器之间是相互隔离互不影响的，也可以进行容器间的通信。

Docker 支持所有主流的操作系统，如 Linux/macOS/Windows，所有主流的云服务也支持 Docker，所以，我们可以将应用及依赖、环境信息等打包到一个 Docker 容器中，可以很轻松地实现应用的发布、迁移等。

Spring Boot 应用既可以基于一个 Dockerfile 文件构建 Docker 镜像，也可以使用 Maven/Gradle 插件基于 Cloud Native Buildpacks 构建 Docker 镜像。

Spring Boot 既支持基于整个应用 jar 包构建 Docker 镜像，也支持构建分层的 Docker 镜像。不建议在 Docker 镜像中构建整包，由上面的章节知道，整包运行要比拆包运行慢，这在容器环境中就更加明显。另外，将应用的所有代码和依赖库都放到 Docker 镜像的一层中不是最优的，因为平时开发过程中经常变动的可能只有 class 文件代码，其他依赖项很少变动，所以推荐构建分层的 Docker 镜像，这时 Docker 只需要替换变动的层，其他未变动的层可以从缓存中提取，还能以拆包的方式运行应用。

11.4.2 Docker 环境搭建

1. 安装 Docker

Docker 有社区版和企业版，社区版是开源免费的，企业版是收费的，当然企业版也会有更高级的特性和服务。Docker 的安装十分简单，笔者这里以 Mac 和 Docker 社区版为例进行演示。

打开 Docker 开始页，见链接 14。

根据不同的操作系统直接下载并安装 Docker Desktop 桌面管理工具，它包含了一个 Docker 可视化开发工具和一个本地的 Kubernates 环境。

> Linux 系统只支持 Ubuntu、Debian、Fedora 三个平台，其他没有界面的、不支持安装 Docker Desktop 的 Linux 系统可以独立安装 Docker 引擎，安装链接为链接 15。

安装完后使用 docker -v 命令可以查看 Docker 版本：

```
$ docker -v
Docker version 20.10.20, build 9fdeb9c
```

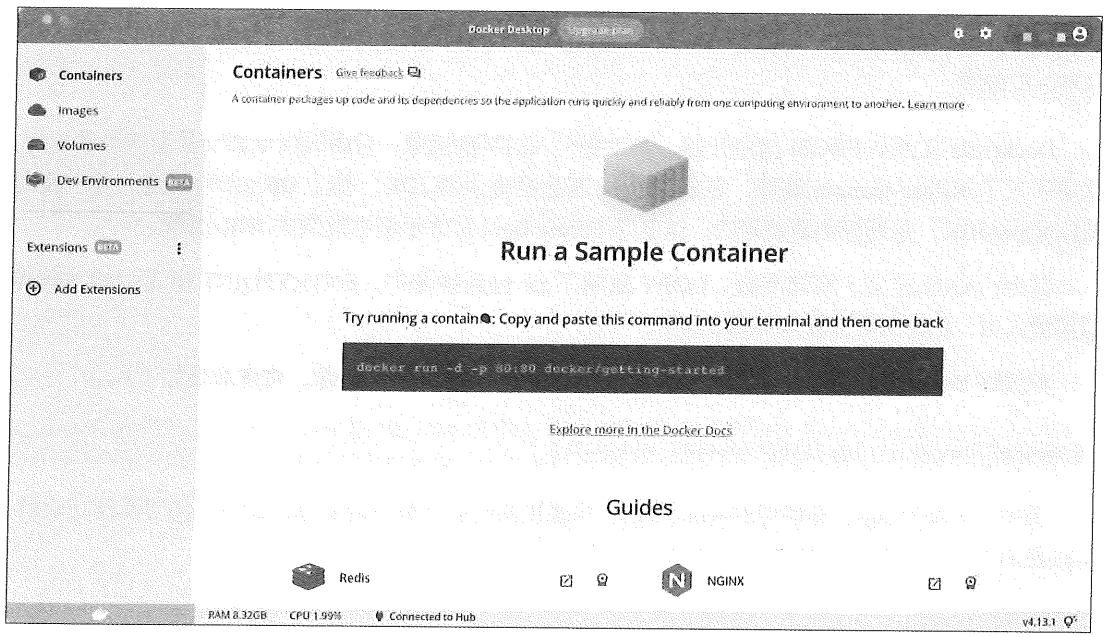
使用 docker version 命令还可以看到完整的版本信息：

```
$ docker version
Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker daemon
running?
Client:
 Cloud integration: v1.0.29
 Version:           20.10.20
 API version:       1.41
 Go version:        go1.18.7
 Git commit:        9fdeb9c
 Built:             Tue Oct 18 18:20:35 2022
 OS/Arch:           darwin/amd64
 Context:           default
 Experimental:      true
```

看到 Docker 版本信息表示 Docker 安装成功了。

2. 启动 Docker

Docker Desktop 的安装和启动都非常简单，系统安装了 Docker Desktop 工具后，只要启动该工具就能自动启动 Docker 了，还可以在该工具中设置开机自动启动，如下图所示。



通过 Docker Desktop 工具可以进行 Docker 容器、镜像、数据卷的管理和环境设置等，可以简化 Docker 命令行的使用难度，对不是很熟悉运维的开发人员来说还是挺友好的，可以显著提升生产力。

如果是 CentOS 等不支持 Docker Desktop 的操作系统，则要先安装好 Docker 引擎，然后以系统服务运行 Docker，比如以下参考命令：

```
$ sudo systemctl start docker
```

11.4.3 基于 Dockerfile 构建镜像

1. 整包构建镜像

Dockerfile 是创建 Docker 镜像的基本配置文件，其本身是一个文本文件，由各种 Dockerfile 指令组成。

在应用的根目录下添加一个 Dockerfile 文件，参考内容如下：

```
# 整包构建
FROM eclipse-temurin:17-jre
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} application.jar
ENTRYPOINT ["java","-Djava.security.egd=file:/dev/./urandom","-jar","
/application.jar"]
EXPOSE 8080
```

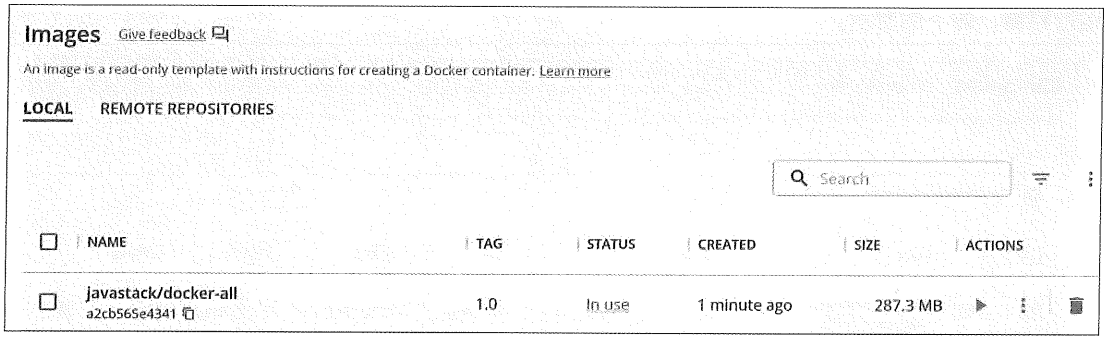
Dockerfile 文件从 FROM 指令开始，表示要引入的基础镜像，后面的指令都会基于该镜像执行，这里引入了 eclipse-temurin 镜像，它是 Eclipse 基金会提供的 JDK，基于 OpenJDK 构建，其前身是 AdoptOpenJDK，是开源免费使用的，这也是 Spring Boot 官方示例中提供的 JDK 镜像。

其中，ARG 定义了构建参数，COPY 复制了 jar 包到镜像中，ENTRYPOINT 指定了启动镜像应用的入口，EXPOSE 指定了镜像的端口。

配置好 Dockerfile 文件之后，再回到应用根目录下构建 Docker 镜像，构建命令如下：

```
$ docker build -t javastack/docker-all:1.0 .
```

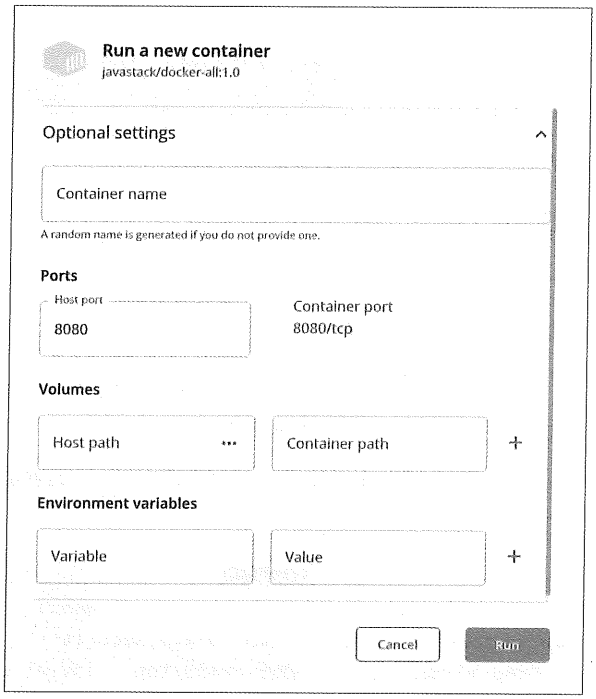
其中，-t 表示 tags，即镜像的 name:tags，构建镜像时会下载对应的基础镜像，构建操作完成后，可以在 Docker Desktop 工具中看到该镜像，如下图所示。



也可以通过以下 docker images 命令查询所有镜像：

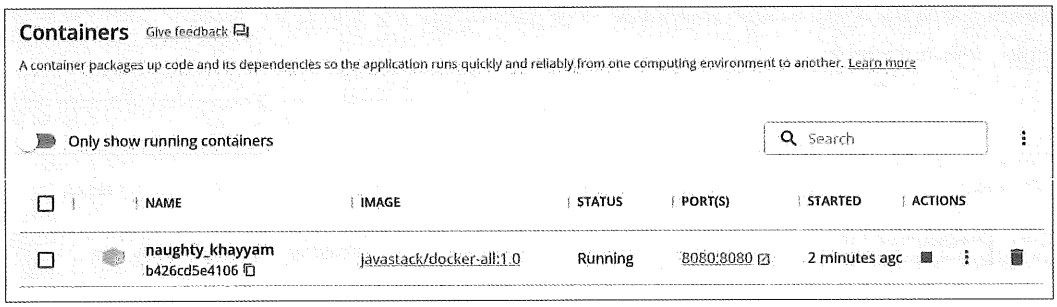
```
$ docker images
REPOSITORY                TAG       IMAGE ID       CREATED          SIZE
javastack/docker-all      1.0       a2cb565e4341   2 minutes ago    287MB
```

镜像只是一个只读的静态文件，不能运行，容器才是镜像的运行实例，所以需要新增一个容器，可以在 Docker Desktop 镜像列表中单击对应的运行按钮新建一个容器，如下图所示。



以上都为选填参数，包括容器名称，笔者这里指定了宿主机端口为 8080，然后单击 Run 按钮

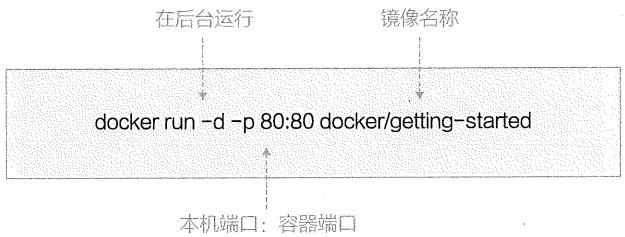
运行容器，如下图所示。



也可以使用以下命令来运行容器：

```
$ docker run -dp 8080:8080 javastack/docker-all:1.0
d8f4f27963006aced0679d1238308466c46ae11e9e0e14a004b7f62e9e1dc574
```

这样就等同于基于镜像新建容器、再运行该容器，各命令参数的意义如下图所示。



其中几个运行参数的含义如下所示。

- -d：表示在后台运行。
- -p：表示要绑定的本机端口：容器端口。

启动容器时需要指定完整的镜像名称和具体标签，容器启动成功后会显示当前容器的完整 ID，容器 ID 是容器的唯一标识，可用于停止、删除容器等操作。

容器启动完成之后，可以通过以下 docker ps 命令查询所有运行中的容器：

```
$ docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED
STATUS         PORTS                                    NAMES
d8f4f2796300   javastack/docker-all:1.0   "java -Djava.securit…"   4 seconds ago
Up 2 seconds   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   jovial_williams
```

使用 docker ps -a 命令可以查询所有状态的容器。

再访问应用中的 /docker/hello 接口，如下图所示。

接口返回成功，说明容器正常运行了。

2. 分层构建镜像

Spring Boot 默认支持应用代码分层，在之前的章节中知道定义分层的是 layers.idx 文件，该分层也可以在插件中自定义，以下是 layers.idx 文件默认的分层：

```
- "dependencies":
  - "BOOT-INF/lib/"
- "spring-boot-loader":
  - "org/"
- "snapshot-dependencies":
- "application":
  - "BOOT-INF/classes/"
  - "BOOT-INF/classpath.idx"
  - "BOOT-INF/layers.idx"
  - "META-INF/"
```

一般变动的只有 class 类文件，引入的类库等文件是很少变动的，这样分层后，如果类库层没有变化，那么在构建 Docker 镜像时就会重用缓存中的类库层，而不会重新构建类库层，这样有利于提升构建速度。

在应用的根目录下添加一个分层构建的 Dockerfile 文件，参考内容如下：

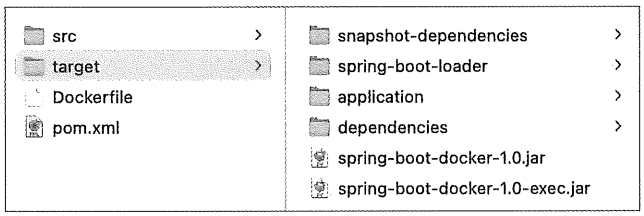
```
FROM eclipse-temurin:17-jre as builder
WORKDIR application
ARG JAR_FILE=target/*-exec.jar
COPY ${JAR_FILE} application.jar
RUN java -Djarmode=layertools -jar application.jar extract

FROM eclipse-temurin:17-jre
WORKDIR application
COPY --from=builder application/dependencies/ ./
COPY --from=builder application/spring-boot-loader/ ./
COPY --from=builder application/snapshot-dependencies/ ./
COPY --from=builder application/application/ ./
ENTRYPOINT ["java", "org.springframework.boot.loader.JarLauncher"]
```

```
EXPOSE 8081
```

需要关注的是 RUN 指令中的 -Djarmode=layertools 参数，在应用分层包中会有一个 spring-boot-jarmode-layertools.jar 包，它可以让应用以特殊的模式启动，比如在 tartget 目录下以该模式启动，并配合 extract 命令可以把层以目录形式提取出来，参考命令如下：

```
$ java -Djarmode=layertools -jar spring-boot-docker-1.0-exec.jar extract
```

提取后的目录如下图所示。



layertools 还支持使用一个 list 参数输出当前包中可以提取出来的层，参考命令如下：

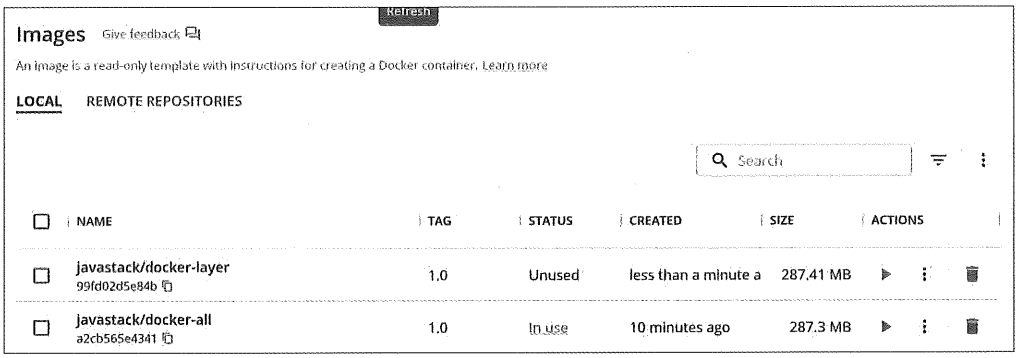
```
$ java -Djarmode=layertools -jar spring-boot-docker-1.0-exec.jar list
dependencies
spring-boot-loader
snapshot-dependencies
application
```

层提取出来之后，Dockerfile 文件中的 COPY 指令就会将它们复制到镜像中，layertools 提供了一种提取包分层的方法，可以让不同层的文件存放在对应的层中，这样就可以做到镜像的分层构建。

配置好 Dockerfile 文件之后，再回到应用根目录下构建 Docker 镜像，构建命令如下：

```
$ docker build -t javastack/docker-layer:1.0 .
```

构建操作完成之后，在 Docker Desktop 中可以看到该镜像了，如下图所示。



也可以通过以下 docker images 命令查询所有镜像：

```
$ docker images
REPOSITORY                  TAG         IMAGE ID        CREATED              SIZE
javastack/docker-layer      1.0         99fd02d5e84b    About a minute ago   287MB
javastack/docker-all        1.0         a2cb565e4341    11 minutes ago       287MB
```

这里可以看到整包构建和分层构建的两个镜像，大小都是一样的。

注意：分层构建不支持可直接执行的包，不然会出现以下兼容性错误。

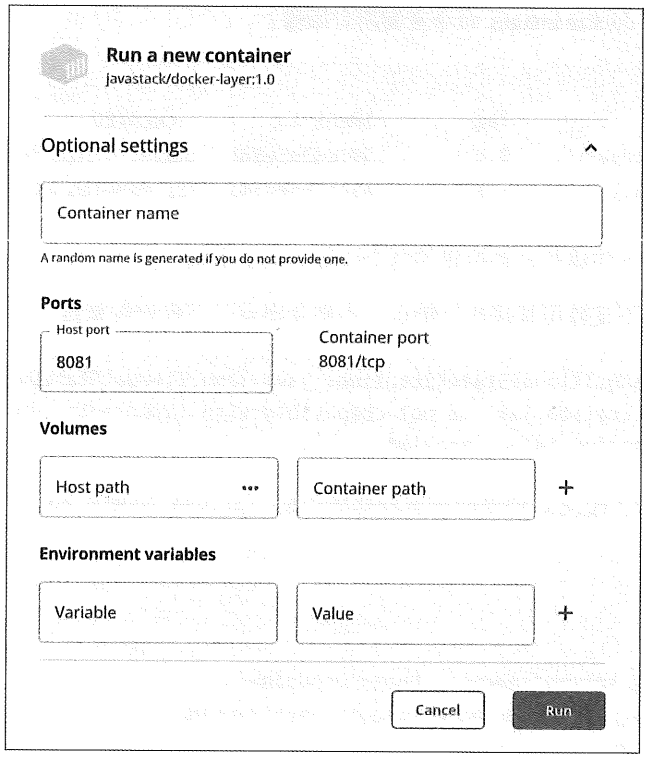
```
Caused by: java.lang.IllegalStateException: java.lang.IllegalStateException: File
'/application/application.jar' is not compatible with layertools; ensure jar file is
valid and launch script is not enabled
```

所以，在构建分层 Docker 镜像时需要把插件中的 executable 设置为 false，如下面的配置所示。

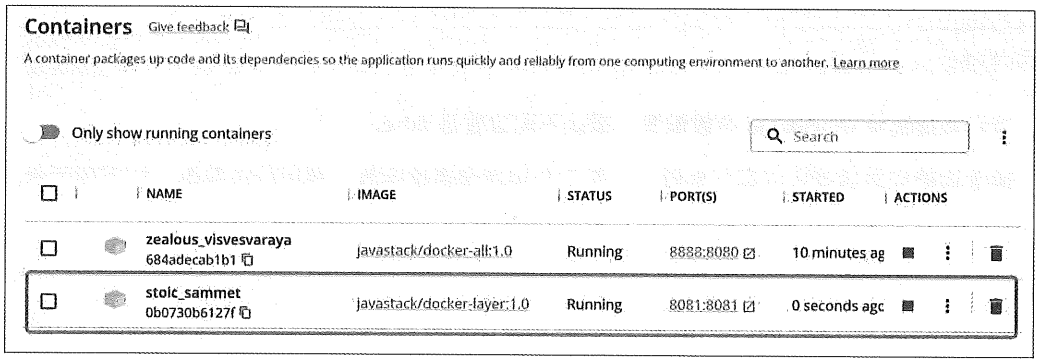
```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <executable>false</executable>
      </configuration>
    </plugin>
  </plugins>
</build>
```

或者直接删除 executable 参数配置，默认不配置就是 false。

镜像构建完之后就可以运行容器了，基于分层镜像新建容器、再运行该容器，如下图所示。



分层构建的镜像新建容器时使用 8081 端口进行区分，不能是之前整包用过的正在运行的 8080 端口，否则会报端口绑定错误。单击 Run 按钮运行容器，如下图所示。



对应的新建和运行容器的命令为：

```
$ docker run -dp 8081:8081 javastack/docker-layer:1.0
7588cc29798dce2ab14ff998608dd6be543662c0b10ad12480e16bbeb14bce5d
```

容器启动完成之后，再通过以下 docker ps 命令查询所有运行中的容器：

```
$ docker ps
CONTAINER ID    IMAGE                          COMMAND                  CREATED
STATUS          PORTS                                        NAMES
7588cc29798d    javastack/docker-layer:1.0   "java org.springfram…"   4 seconds ago
Up 2 seconds    0.0.0.0:8081->8081/tcp, :::8081->8081/tcp   wonderful_bouman
d8f4f2796300    javastack/docker-all:1.0     "java -Djava.securit…"   12 minutes ago
Up 12 minutes   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   jovial_williams
```

这里可以看到整包镜像的容器和分层镜像的容器，然后分别访问 8080 和 8081 端口的 /docker/hello 接口，两个容器都可以正常响应。

前面介绍了，构建的镜像是分层的，如果对应的层没有变动就会重用缓存中的层，现在重复构建镜像验证一下，参考命令如下：

```
$ docker build -t javastack/docker-layer:1.0 .
[+] Building 1.2s (13/13) FINISHED
 => [internal] load build definition from Dockerfile                          0.0s
 => => transferring dockerfile: 37B                                           0.0s
 => [internal] load .dockerignore                                             0.0s
 => => transferring context: 2B                                               0.0s
 => [internal] load metadata for docker.io/library/eclipse-temurin:17-jre     1.1s
 => [internal] load build context                                             0.0s
 => => transferring context: 88B                                              0.0s
 => [stage-1 1/6] FROM docker.io/library/eclipse-temurin:17-jre@sha256:
87f454bb6797b918c6fcfb04d18c8bc55f87f7b209dede52a24e2385f84f9ba3              0.0s
 => CACHED [stage-1 2/6] WORKDIR application                                  0.0s
 => CACHED [builder 3/4] COPY target/*-exec.jar application.jar               0.0s
 => CACHED [builder 4/4] RUN java -Djarmode=layertools -jar application.jar extract
                                                                              0.0s
 => CACHED [stage-1 3/6] COPY --from=builder application/dependencies/ ./     0.0s
 => CACHED [stage-1 4/6] COPY --from=builder application/spring-boot-loader/ ./
                                                                              0.0s
 => CACHED [stage-1 5/6] COPY --from=builder application/snapshot-dependencies/ ./
                                                                              0.0s
 => CACHED [stage-1 6/6] COPY --from=builder application/application/ ./      0.0s
 => exporting to image                                                        0.0s
 => => exporting layers                                                       0.0s
 => => writing image sha256:c900f6909e6691d818198a27521dda3d8df396471c97c47e207eefddd0ebb3b4
                                                                              0.0s
 => => naming to docker.io/javastack/docker-layer:1.0                         0.0s
```

可以看到许多 CACHED 字样的日志，因为没有修改任何代码，所以所有的层都是从缓存中提取的，构建镜像仅用了 1.2 秒。现在任意修改一个类的代码再重新构建镜像，参考命令如下：

```
$ docker build -t javastack/docker-layer:1.0 .
[+] Building 2.8s (13/13) FINISHED
 => [internal] load build definition from Dockerfile                    0.0s
 => => transferring dockerfile: 37B                                     0.0s
 => [internal] load .dockerignore                                       0.0s
 => => transferring context: 2B                                         0.0s
 => [internal] load metadata for docker.io/library/eclipse-temurin:17-jre  1.2s
 => [internal] load build context                                       0.4s
 => => transferring context: 20.28MB                                    0.4s
 => [stage-1 1/6] FROM docker.io/library/eclipse-temurin:17-jre@sha256:
87f454bb6797b918c6fcfb04d18c8bc55f87f7b209dede52a24e2385f84f9ba3        0.0s
 => CACHED [stage-1 2/6] WORKDIR application                            0.0s
 => [builder 3/4] COPY target/*-exec.jar application.jar                0.1s
 => [builder 4/4] RUN java -Djarmode=layertools -jar application.jar extract 0.9s
 => CACHED [stage-1 3/6] COPY --from=builder application/dependencies/ ./  0.0s
 => CACHED [stage-1 4/6] COPY --from=builder application/spring-boot-loader/ ./
                                                                        0.0s
 => CACHED [stage-1 5/6] COPY --from=builder application/snapshot-dependencies/ ./
                                                                        0.0s
 => [stage-1 6/6] COPY --from=builder application/application/ ./       0.0s
 => exporting to image                                                  0.0s
 => => exporting layers                                                 0.0s
 => => writing image sha256:c900f6909e6691d818198a27521dda3d8df396471c97c47e207eefddd0ebb3b4
                                                                        0.0s
 => => naming to docker.io/javastack/docker-layer:1.0                   0.0s
```

此时只有 application 应用层不是从缓存中提取的，构建时间也变成了 2.8 秒，所以对于构建层、内容的多少，构建时间也是有显著区别的。

11.4.4 基于 Cloud Native Buildpacks 构建镜像

除了可以使用 Dockerfile 配置文件构建 Docker 镜像，还可以通过 Maven/Gradle 插件使用 Cloud Native Buildpacks 构建镜像。Cloud Native Buildpacks 于 2018 年由 Pivotal 和 Heroku 发起，并且在同年加入了云原生基金会，使用了现代的容器标准，比如 OCI 镜像格式。Cloud Native Buildpacks 既可以将应用构建成在任何云平台上运行的镜像，比如 Cloud Foundry，又可以构建兼容 Docker 的镜像。

添加 Spring Boot Maven 打包插件配置：

```
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <executions>
                <execution>
                    <goals>
                        <goal>build-image</goal>
                    </goals>
                </execution>
            </executions>
            <configuration>
                <!-- 分层镜像不兼容可直接执行的包 -->
                <executable>false</executable>
                <image>
                    <name>javastack/docker-cnb:${project.version}</name>
                </image>
            </configuration>
        </plugin>
    </plugins>
</build>
```

其中指定了 build-image 构建镜像目标，并且通过 image 自定义了镜像名称，默认为 docker.io/library/${project.artifactId}:${project.version} 命名规则。

回到应用根目录，直接使用 mvn 打包命令构建 Docker 镜像：

```
$ mvn clean package
```

也可以直接调用配置好的 build-image 目标构建 Docker 镜像：

```
$ mvn spring-boot:build-image
```

这两个命令默认都会先打包，然后构建镜像，首次构建时会拉取 Cloud Native Buildpacks 基础镜像，首次构建时的拉取过程会比较长。

另外，从构建日志中可以看出，构建镜像也支持分层构建：

```
[INFO]     [creator]         Creating slices from layers index
[INFO]     [creator]         dependencies (19.2 MB)
[INFO]     [creator]         spring-boot-loader (271.3 KB)
[INFO]     [creator]         snapshot-dependencies (0.0 B)
[INFO]     [creator]         application (40.5 KB)
```

但从整体上来说，使用 Cloud Native Buildpacks 方式的构建过程比较长，即使经过多次构建，后续每次构建都花了 20 多秒：

```
[INFO] Successfully built image 'docker.io/javastack/docker-cnb:1.0'
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  23.074 s
[INFO] Finished at: ...
[INFO] ------------------------------------------------------------------------
```

可以优化的一点是设置不用每次拉取基础镜像，如以下配置所示。

```
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <executions>
                <execution>
                    <goals>
                        <goal>build-image</goal>
                    </goals>
                </execution>
            </executions>
            <configuration>
                <!-- 分层镜像不兼容可直接执行的包 -->
                <executable>false</executable>
                <image>
                    <name>javastack/docker-cnb:${project.version}</name>
                    <pullPolicy>IF_NOT_PRESENT</pullPolicy>
                </image>
            </configuration>
        </plugin>
    </plugins>
</build>
```

配置项 pullPolicy 支持的值如下：

- ALWAYS：默认值，每次都拉取基础镜像。
- IF_NOT_PRESENT：不存在基础镜像时才拉取。

- NEVER：从不拉取基础镜像。

更多 Maven 插件的配置可以参考链接 16。

修改此配置后再次构建镜像，如以下日志所示。

```
[INFO] Successfully built image 'docker.io/javastack/docker-cnb:1.0'
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  13.329 s
[INFO] Finished at: ...
[INFO] ------------------------------------------------------------------------
```

优化拉取基础镜像策略后，每次构建只需要 10 多秒了。即使如此，可能还有优化的空间，但也会明显慢于基于 Dockerfile 配置文件的秒级构建，基于 Cloud Native Buildpacks 的构建方式慢了不止一点，所以，如果应用不是要部署到相关的云平台，那么使用 Docker 原生构建命令要快得多。

等构建操作完成之后，可以通过以下 docker images 命令查询所有镜像：

```
$ docker images
REPOSITORY                  ...  SIZE
javastack/docker-layer      ...  286MB
javastack/docker-all        ...  286MB
javastack/docker-cnb        ...  277MB
```

这里可以看到之前使用 Dockerfile 构建的两个镜像和使用 Cloud Native Buildpacks 构建的镜像，Cloud Native Buildpacks 构建的镜像较 Dockerfile 构建的镜像略小一些。

镜像构建完之后就可以新建容器来运行应用了，笔者未找到如何在 Maven 插件中定义镜像的端口的方法，这里先使用命令的方式基于 Cloud Native Buildpacks 镜像新建容器、再运行该容器。为了区分基于 Dockerfile 配置文件构建的镜像，这里使用 8082 端口进行映射：

```
$ docker run -dp 8082:8082 javastack/docker-cnb:1.0
e99a3425f1e44b3da049ca788dabc24d5ba25abd834a5223774d47c1124b0e84
```

容器启动完成之后，再通过以下 docker ps 命令查询所有运行中的容器：

```
$ docker ps
CONTAINER ID  IMAGE                       ... PORTS                                      ...
e99a3425f1e4  javastack/docker-cnb:1.0    ... 0.0.0.0:8082->8082/tcp, :::8082->8082/tcp ...
c0f40ca3fd6b  javastack/docker-layer:1.0 ... 0.0.0.0:8081->8081/tcp, :::8081->8081/tcp ...
```

```
d8f4f2796300  javastack/docker-all:1.0   ... 0.0.0.0:8080->8080/tcp, :::8080->8080/tcp ...
```

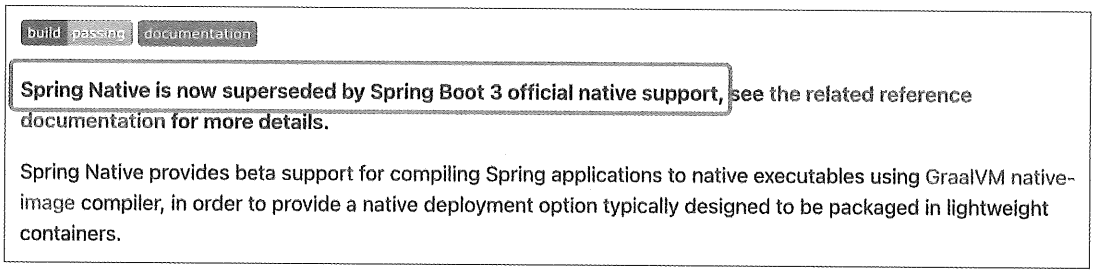
这里可以看到基于 Dockerfile 配置文件构建的镜像容器，还有通过 Maven 插件使用 Cloud Native Buildpacks 构建的镜像容器，然后分别访问 8080、8081、8082 端口的 /docker/hello 接口，三个容器都可以正常响应。

11.5 GraalVM 原生镜像（Spring Boot 3.0+）

11.5.1 概述

Spring Boot 3.0 应用现在可以支持生成 GraalVM 原生镜像了，这可以提供显著的内存和启动性能改进，支持 GraalVM 原生镜像也是整个 Spring 产品组合中一项重大能力的提升。

GraalVM 原生镜像直接取代了实验性的 Spring Native 项目，如下图所示。



Spring Boot 3.0 最低支持 GraalVM 22.3+ 和 Native Build Tools Plugin 0.9.17+。

GraalVM 是 Oracle 于 2018 年发布的一个高性能的、跨语言的通用虚拟机，号称是一个全新的全栈虚拟机，并具有高性能、跨语言交互等特性。GraalVM 主要用于提升 Java、Scala、Groovy、Kotlin 等基于 JVM 的语言的应用的执行性能，同时还支持其他像 JavaScript、Ruby、Python 和 R 等语言并为其提供运行环境。

GraalVM 在 HotSpot Java 虚拟机中添加了一个用 Java 编写的高级的 JIT 优化编译器，除了运行 Java 和基于 JVM 的语言，借助 GraalVM 的语言实现框架 Truffle，GraalVM 还能做到在 JVM 之上运行 JavaScript、Ruby、Python 等其他流行的编程语言，还支持 Java 和其他语言之间直接互相操作并在同一内存空间来回传递数据。

GraalVM 虚拟机有以下几个特性：

- 更加高效快速地运行代码。

- 能与大多数编程语言直接交互。
- 使用 Graal SDK嵌入多语言。
- 创建预编译的原生镜像。
- 提供一系列工具来监视、调试和配置所有代码。

下面通过两个示例详细介绍 GraalVM 虚拟机的优势，以下这段示例代码来自 GraalVM 官网：

```
const express = require('express');
const app = express();
app.listen(3000);
app.get('/', function(req, res) {
  var text = 'Hello World!';
  const BigInteger = Java.type(
    'java.math.BigInteger');
  text += BigInteger.valueOf(2)
    .pow(100).toString(16);
  text += Polyglot.eval(
    'R', 'runif(100)')[0];
  res.send(text);
})
```

上面这段代码同时使用了 Node.js、Java、R 三种语言，GraalVM 消除了各种编程语言之间的隔离性，多编程语言可以结合使用，并且不会有任何性能影响，零开销的互操作，这样我们就可以为应用选择最佳的编程语言组合。

下面这段命令同样来自 GraalVM 官网：

```
$ javac HelloWorld.java
$ time java HelloWorld
user 0.070s
$ native-image HelloWorld
$ time ./helloworld
user 0.005s
```

GraalVM 可以将 JVM 系语言的代码预编译成原生镜像，即通过提前处理已编译的 Java 应用来生成原生镜像文件，它是一个独立的基于特定平台的可执行文件，这样可以极大缩短 JVM 系语言的启动时间，也能减少 JVM 应用的内存占用。

11.5.2 GraalVM 应用与传统应用的区别

对于 Java 应用，GraalVM 提供了两种运行方法：

- 在 HotSpot JVM 上使用 Graal 的 JIT 编译和运行应用，应用在运行时编译，边运行边编译。
- 作为 AOT 编译的 GraalVM 原生镜像可执行文件运行，应用在运行前编译，即静态编译或者预先编译；在创建 GraalVM 原生镜像时，应用需要经过预先编译处理才能创建可执行文件，这种预先编译涉及从应用的主要入口点开始进行代码静态分析。

传统的 Java 应用必须依赖于 Java 虚拟机（JVM）运行，Spring Boot 3.0 发布后可以不需要依赖 JVM 运行应用，它提供了另一种运行和部署 Spring 应用的方式，即通过 GraalVM 将 Spring 应用编译成原生镜像的可执行文件来运行。

GraalVM 应用与传统应用的区别如下：

- 构建镜像时会进行应用的静态分析，从主入口点开始执行。
- 创建原生映像时无法访问的代码将被删除，并且不会成为可执行文件的一部分。
- GraalVM 不能直接感知代码的动态元素，比如反射、资源、序列化和动态代理等动态代码必须提前声明并配置。
- 应用的 classpath 路径在构建时是固定不变的，不能修改。
- 不支持 Bean 的延迟加载，可执行文件包含的所有内容都在启动时加载到内存中。
- 目前某些方面还存在一些限制和局限性，并不能完全兼容传统的 Java 应用。

传统的 Spring Boot 应用非常动态化，可以在运行时动态生成任意代码，Spring Boot 自动配置机制在很大程度上也取决于对运行时状态的反应，以便能正确配置。虽然可以将应用的动态代码配置并告诉 GraalVM，但这样做会抵消静态分析的大部分好处。因此，当 Spring Boot 在创建原生镜像时，应用在动态方面会受到限制，比如：

- 应用的 classpath 路径在构建时是固定不变的，不能修改。
- Bean 在运行时不能被修改，这也就意味着在 Spring 应用中，不支持 @Profile 注解及基于 Profile 配置，另外像 @ConditionalOnProperty 等条件注解也都会受到限制。

Spring AOT 在构建原生镜像时会执行预先编译并生成以下文件：

- Java 源代码。
- 字节码（用于动态代理等场景）。

- GraalVM 相关的 JSON 描述文件，包含 Resource hints（resource-config.json）、Reflection hints（reflect-config.json）、Serialization hints（serialization-config.json）、Java Proxy Hints（proxy-config.json）、JNI Hints（jni-config.json）文件。

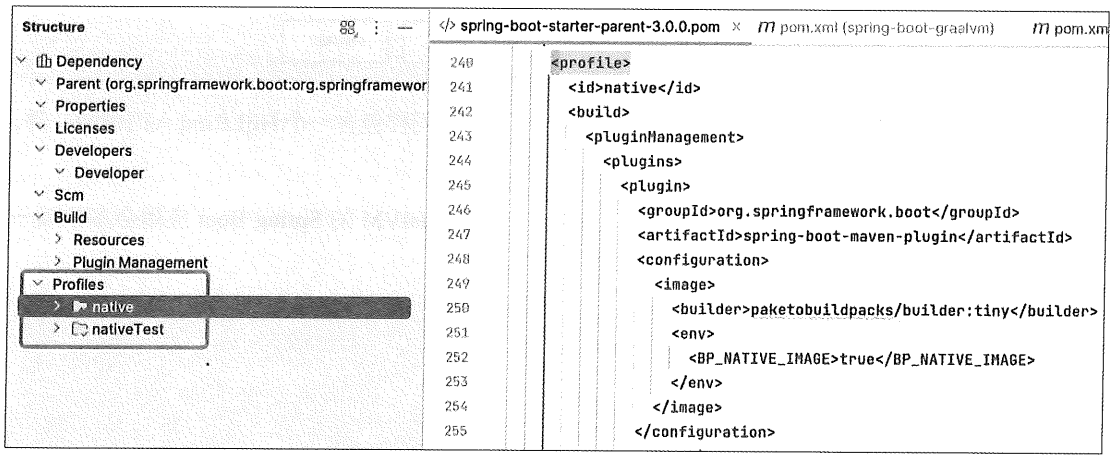
11.5.3 创建 GraalVM 原生镜像的应用

通过前面的章节我们已经很好地了解了 GraalVM 原生镜像及 Spring AOT 引擎的工作机制，本节介绍如何创建一个基于 GraalVM 原生镜像的应用。

构建 Spring Boot 原生镜像应用主要有以下两种方式：

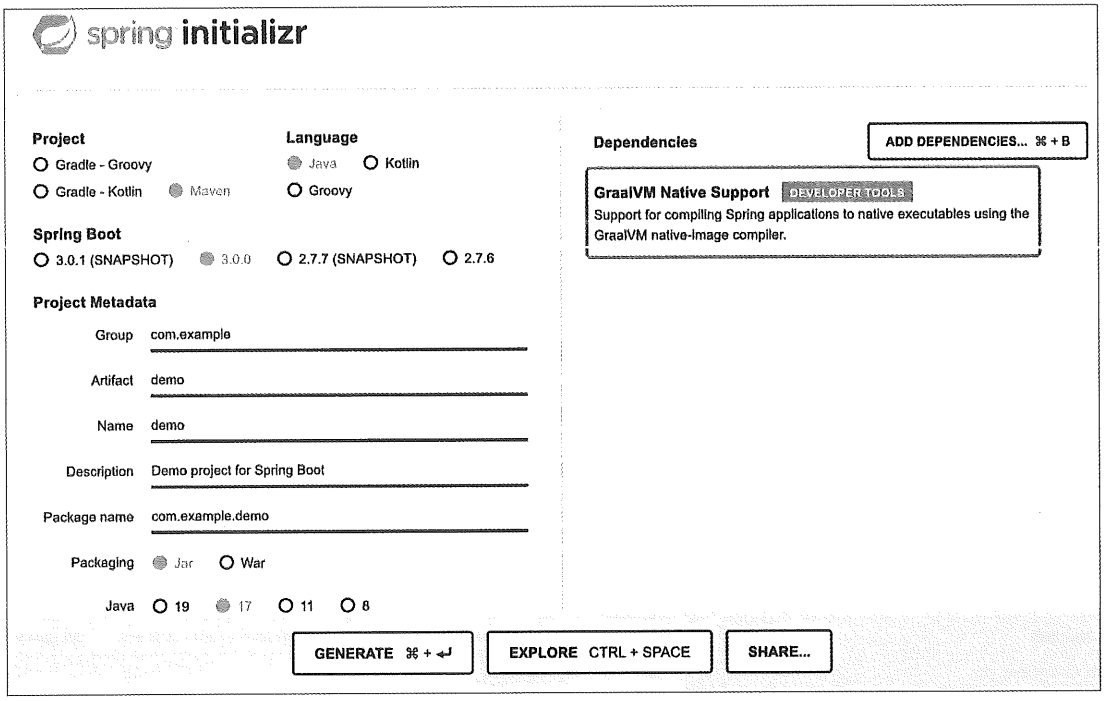
- 使用 Spring Boot 的 Cloud Native Buildpacks 生成一个包含原生可执行文件的轻量级容器。
- 使用 GraalVM 的原生构建工具生成一个原生可执行文件。

先创建一个 GraalVM 原生应用测试项目，建议使用继承 spring-boot-starter-parent 父项目依赖的方式创建 GraalVM 项目，它自带了两个 Profile 原生配置，如下图所示。



该 native 配置项配置了需要运行原生镜像的执行目标，可以在命令行中使用 -P 标识并激活 Profile。如果不能使用 spring-boot-starter-parent，则需要为 Spring Boot 插件的 process-aot 设置项和 Native Build Tools 插件的 add-reachability-metadata 设置项配置执行目标。

最简单方法还是去 Spring 一站式项目生成网站 start.spring.io 生成示例项目，然后添加“GraalVM Native Support”依赖项并生成项目，如下图所示。



创建的项目结构和第1章介绍的生成项目结构一致，其中同样包含一个HELP.md入门帮助文件，帮助开发者了解如何创建和运行原生镜像。

与基于JVM传统的Spring Boot应用不同的是，基于GraalVM的Spring Boot应用会多依赖一个native-maven-plugin插件，如下面的配置所示。

```
<build>
    <plugins>
        <plugin>
            <groupId>org.graalvm.buildtools</groupId>
            <artifactId>native-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

为了便于后续的测试，在启动类中再添加一个/hi测试接口，如下面的示例代码所示。

```
@RestController
@SpringBootApplication
public class Application {
```

```
    @RequestMapping("/hi")
    String hello() {
        return "Hello GraalVM!";
    }

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }

}
```

11.5.4 构建基于 GraalVM 的原生镜像应用

1. 使用 Buildpacks 构建（部署 Docker 容器）

Spring Boot 提供了针对 Maven 和 Gradle 的原生镜像的 buildpack 的支持，只需一个命令即可将镜像放入本地运行的 Docker 守护进程，生成的镜像不会包含 JVM，而是静态编译的原生镜像。

所以，使用 Buildpacks 构建原生镜像时，本机需要安装 Docker 环境，详见 11.4.2 节。

使用 Maven 构建镜像的命令如下：

```
$ mvn -Pnative spring-boot:build-image
```

第一次构建需要下载 GraalVM 相关插件及构建所需要的基础镜像，所以会有点慢，第一次构建足足花费了 7 分钟，如下面的构建日志所示。

```
[INFO] Successfully built image 'docker.io/library/spring-boot-graalvm:1.0'
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  07:29 min
[INFO] Finished at: 2022-11-28T17:21:06+08:00
[INFO] ------------------------------------------------------------------------
```

构建完成之后，通过 docker images 命令可以查看该镜像，镜像大小为 96MB，可以使用 docker run 命令运行该镜像：

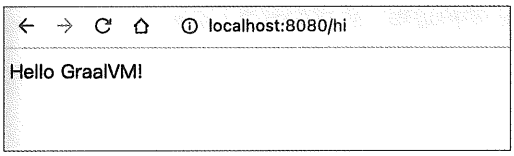
```
$ docker run --rm -p 8080:8080 docker.io/library/spring-boot-graalvm:1.0
```

启动日志如下所示。

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::  (v3.0.0)
....... . . .
....... . . .
....... . . .
........ Started Application in 0.083 seconds (process running for 0.088)
```

0.083 秒便启动了一个简单的 Spring Boot 应用，启动速度真是太快了，比启动运行在传统 JVM 上的 Spring Boot 应用快了很多倍，这也见证了 GraalVM 的性能魅力了。

再访问应用的 /hi 接口，如下图所示。



响应正常，然后停止运行该容器，再以传统的基于 JVM 的方式直接运行 main 方法，启动日志如下所示。

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::  (v3.0.0)
....... . . .
....... . . .
....... . . .
........ Started Application in 2.065 seconds (process running for 2.714)
```

程序正常启动，虽然只花了 2 秒，但与基于 GraalVM 原生镜像的启动速度相比也差了很多数量级，这里只是简单的应用测试，如果是更复杂的应用，差异可能会更加明显。

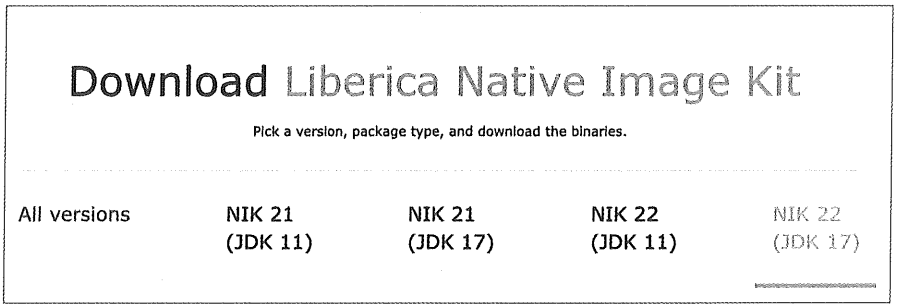
2. 使用 GraalVM 工具构建（直接运行）

如果不想基于 Docker 环境构建并运行及生成原生可执行文件，则可以使用 GraalVM 的原生构

建工具 Native Build Tools，它是 GraalVM 为 Maven 和 Gradle 提供的原生镜像构建插件，使用它可以执行各种 GraalVM 镜像相关的操作。

要使用 GraalVM 的原生构建工具 Native Build Tools，需要在本机安装 GraalVM 发行版，可以手动下载安装，也可以通过 SDKMAN 安装，手动下载地址为链接 17。

GraalVM 支持 Windows、macOS、Linux 等多种操作系统，读者可以根据不同的操作系统及 JDK 版本进行安装，如下图所示。



macOS 和 Linux 操作系统推荐使用 SDKMAN 安装 GraalVM，具体安装过程可见第 1 章的 Spring Boot CLI 一节，比如安装最新的基于 JDK 17 的参考命令如下：

```
$ sdk install java 22.3.r17-nik
$ sdk use java 22.3.r17-nik
```

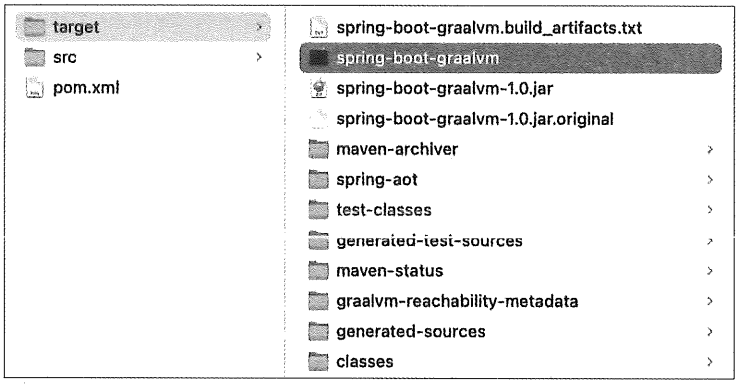
安装完 GraalVM 之后需要确保本机的 JAVA_HOME 环境变量为 GraalVM JDK 所在的目录，可以通过 java -version 查询：

```
$ java -version
openjdk version "17.0.5" 2022-10-18 LTS
OpenJDK Runtime Environment GraalVM 22.3.0 (build 17.0.5+8-LTS)
OpenJDK 64-Bit Server VM GraalVM 22.3.0 (build 17.0.5+8-LTS, mixed mode, sharing)
```

显示 OpenJDK 及 GraalVM 相关字样说明安装成功了，然后使用 Maven 命令构建镜像：

```
$ mvn -Pnative native:compile
```

镜像构建完之后可以在当前应用的 tartget 目录中找到该原生镜像的可执行文件，如下图所示。



主要生成了两个文件：

- spring-boot-graalvm：应用可执行文件。
- spring-boot-graalvm.build_artifacts.txt：对应的说明文本文件。

直接运行应用可执行文件，运行命令如下：

```
$ ./spring-boot-graalvm
```

应用启动日志如下所示。

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::  (v3.0.0)
....... . . .
....... . . .
....... . . .
........ Started MyApplication in 0.089 seconds (process running for 0.102)
```

和 buildpack 方式构建的应用可执行文件一样，Native Build Tools 构建的应用可执行文件一样可以达到极致的启动性能，再访问应用的 /hi 接口也是能正常响应的。

Native Build Tools 和 buildpack 虽然都使用 Maven 插件构建应用可执行文件，但是两者的构建目标和应用最终的运行环境及运行方式是不一样的，buildpack 需要基于 Docker 环境运行，而

Native Build Tools 构建的可执行包可以直接运行，即可以像 11.3.2 节介绍的基于传统的 JVM 的嵌入式容器一样直接运行。

虽然 GraalVM 能够显著提高应用的启动性能，也能节约内存的占用，但它目前还属于快速发展的技术，不是很成熟，很多类库也不支持，也存在已知的种种限制，在 Spring Boot 3.0 中也属于首发版本，要真正代替传统的 JVM 还需要很长的一段路要走。

GraalVM 原生镜像已知的使用限制参考见链接 18。

第 12 章
Spring Boot 监控与报警

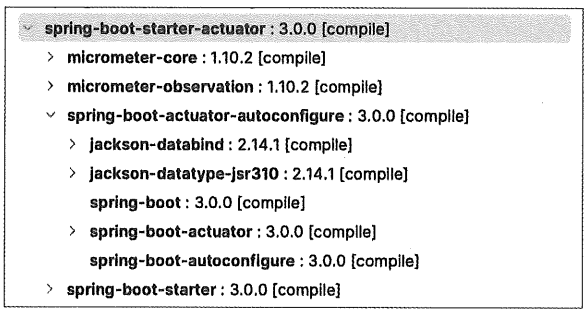
Spring Boot 应用的监控是通过 Actuator 模块进行的，它也是 Spring Boot 的重要特性之一，所以 Actuator 模块一般用于生产环境的应用，使用 HTTP 端点或者 JMX 的方式提供应用的监控和管理操作，包括应用的健康检查、指标收集等，然后通过第三方监控平台抓取指标数据或者推送指标数据给监控平台。本章会按照 Actuator 模块→ Endpoints（端点）→ Metrics（指标）→监控平台的顺序介绍监控平台是如何通过这些组件完成监控的。

12.1 Spring Boot Actuator 概述

启用 Spring Boot Actuator 功能，只需要添加其启动器依赖即可，如下面的依赖配置所示。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

具体导入的依赖如下图所示。



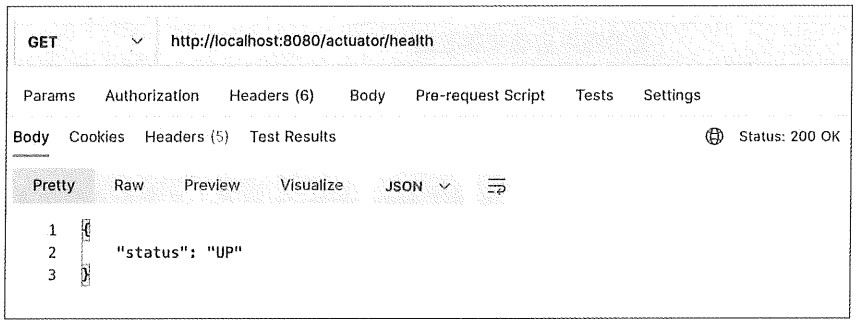
它主要包含了 micrometer 相关依赖包、actuator 自动配置模块。

12.2 Endpoints（端点）

12.2.1 概述

Endpoint 端点用来监控 Spring Boot 应用并与之交互，Spring Boot 内置了许多端点，“开箱即用”，每个端点都可以通过 HTTP 或者 JMX（Java Management Extensions）的方式暴露出去，但绝大多数情况下端点都是以 HTTP 方式暴露的，每个端点都会被映射为 /actuator/${ID} 方式，ID 即端点的 ID，比如健康端点为 /actuator/health。

既然 health 端点是以 HTTP 方式暴露的，那么就可以通过 HTTP 来访问这个端点，如下图所示。



该端点返回了 UP 状态，表示应用为健康运行状态。

12.2.2 内置端点

Spring Boot 的内置端点如下表所示。

端点 ID	功能
auditevents	暴露当前应用的审计事件信息
beans	显示应用中的所有 Spring Bean
caches	暴露可用的缓存
conditions	显示配置类上评估条件及匹配成功与否的原因
configprops	显示所有的 @ConfigurationProperties 列表
env	从 Spring 环境中暴露所有 properties
flyway	显示所有 Flyway 迁移记录

续表

端点 ID	功能
health	显示健康信息
httpexchanges	显示 HTTP 交换信息（默认显示最后 100 个）
info	显示应用的基本信息
integrationgraph	显示 Spring Integration 图表
loggers	显示或者修改日志配置
liquibase	显示所有 liquibase 迁移记录
metrics	显示指标信息
mappings	显示所有 @RequestMapping 映射路径
quartz	显示 Quartz 任务调度信息
scheduledtasks	显示所有的任务调度
sessions	从 Spring Session 中检索和删除用户会话
shutdown	优雅关闭应用
startup	显示应用启动步骤数据
threaddump	线程转储，用于分析某一节点的线程堆栈状态

Spring Boot 3.0.0 新变化：

Spring Boot 3.0.0 移除了 httptrace 端点，新增了 httpexchanges 端点，只是变更了端点名称，功能还是一样的。

如果是 Web 应用，那么还内置了以下端点，如下表所示。

端点 ID	功能
heapdump	返回一个 heap dump 文件： HotSpot JVM 返回的是 HPROF-format 格式文件 OpenJ9 JVM 返回的是 PHD-format 格式文件
logfile	返回日志文件内容（需要设置 logging.file.name 或者 logging.file.path 参数）
prometheus	暴露可以被 Prometheus 服务器抓取的格式指标

如果内置的端点不满足需求，也可以自定义端点。

12.2.3 启用端点

除了 shutdown 端点，其他所有端点默认都是启用的，也可以启用或禁用某个端点，启用或禁用某个端点是通过 management.endpoint.<id>.enabled 参数配置的，比如启用 shutdown 端点，如下面的配置所示。

```
management:
  endpoint:
    shutdown:
      enabled: true
```

如果想一个个手动启用端点，则可以先禁用全部端点，再启用某个端点，如下面的配置所示。

```
management:
  endpoints:
    enabled-by-default: false
  endpoint:
    info:
      enabled: true
```

这里先禁用所有端点，然后只启用 info 端点。

需要注意的是，被禁用的端点会完全从 Spring 上下文中删除，如果只想更改端点暴露的方式，那么可以使用 include 和 exclude 配置参数替代禁用端口的方式，具体见后面的章节。

12.2.4 暴露端点

端点被启用后，并不一定能够被访问，还要看端点是否被暴露，并且暴露的方式是怎样的。因为端点可能包含敏感信息，所以需要谨慎暴露相关端点。在各大 Spring Boot 版本中暴露的端点不太一样，这里以 Spring Boot 2.7.x 为例，内置端点的默认暴露方式如下表所示。

端点 ID	JMX	Web
auditevents	Yes	No
beans	Yes	No
caches	Yes	No
conditions	Yes	No
configprops	Yes	No
env	Yes	No
flyway	Yes	No
health	Yes	Yes
heapdump	N/A	No
httptrace	Yes	No
info	Yes	No
integrationgraph	Yes	No
logfile	N/A	No

续表

端点 ID	JMX	Web
loggers	Yes	No
liquibase	Yes	No
metrics	Yes	No
mappings	Yes	No
prometheus	N/A	No
quartz	Yes	No
scheduledtasks	Yes	No
sessions	Yes	No
shutdown	Yes	No
startup	Yes	No
threaddump	Yes	No

默认以 JMX 方式暴露了所有端点，出于安全考虑，Web 方式则只暴露了 Health 端点。

Spring Boot 3.0.0 新变化：

Spring Boot 3.0.0 更改了默认暴露的端点，默认以 JMX 和 Web 方式只暴露了 Health 一个端点，更加严格、安全了。

可以使用以下参数更改端点暴露的方式，如下表所示。

配置参数	描述	默认值
management.endpoints.jmx.exposure.exclude	停止以 JMX 方式暴露端点	
management.endpoints.jmx.exposure.include	以 JMX 方式暴露端点	health
management.endpoints.web.exposure.exclude	停止以 Web 方式暴露端点	
management.endpoints.web.exposure.include	以 Web 方式暴露端点	health

多个端点以逗号分隔，也支持“*”通配符，表示所有端点，在 .yaml 配置中需要用引号包起来，如下面的配置示例所示。

```
management:
  endpoints:
    jmx:
      exposure:
        include: health,info
    web:
      exposure:
        include: "*"
        exclude: threaddump
```

这里配置了以 JMX 方式只暴露 health 和 info 端点，以 Web 方式暴露除 threaddump 端点外的所有端点，两种暴露方式是独立的，不会互相影响各自暴露的端点，如果同一种暴露方式同时配置了 include 和 exclude，则以 exclude 配置的优先。

一般通过配置参数就能暴露端点，如果想自定义实现端点的暴露策略，则可以实现 EndpointFilter 接口并注册到 Spring 容器中。

12.2.5 端点安全性

前面介绍了，端点提供的信息可能涉及敏感性，所以除了谨慎暴露所需要的端点，还需要对端点进行安全保护，如提供对应的权限访问等。

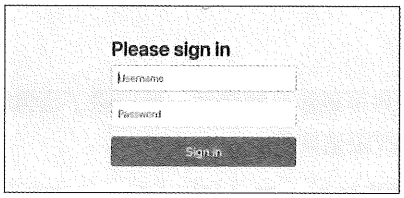
第 3 章介绍了 Spring Boot 提供的 Web 默认保护机制，只要有 Spring Security 类库依赖，Spring Boot 就会自动保护所有接口，还包括除 health 健康端点外的所有端点，具体的实现见 Actuator 的 ManagementWebSecurityAutoConfiguration 自动配置类，它注册在 spring-boot-actuator- autoconfigure 包的新的自动配置文件中，它的源码如下：

```
@AutoConfiguration(before = SecurityAutoConfiguration.class,
        after = { HealthEndpointAutoConfiguration.class, InfoEndpointAutoConfiguration.class,
                WebEndpointAutoConfiguration.class, OAuth2ClientAutoConfiguration.class,
                OAuth2ResourceServerAutoConfiguration.class,
                Saml2RelyingPartyAutoConfiguration.class })
@ConditionalOnWebApplication(type = ConditionalOnWebApplication.Type.SERVLET)
@ConditionalOnDefaultWebSecurity
public class ManagementWebSecurityAutoConfiguration {

    @Bean
    @Order(SecurityProperties.BASIC_AUTH_ORDER)
    SecurityFilterChain managementSecurityFilterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests((requests) -> {
            requests.requestMatchers(EndpointRequest.to(HealthEndpoint.class)).permitAll();
            requests.anyRequest().authenticated();
        });
        if (ClassUtils.isPresent("org.springframework.web.servlet.DispatcherServlet", null)) {
            http.cors();
        }
        http.formLogin(Customizer.withDefaults());
        http.httpBasic(Customizer.withDefaults());
        return http.build();
    }

}
```

然后加入 Spring Security 一站式启动器依赖：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

应用重启后，再访问应用的 /actuator/health 端点，如下图所示。



如果没有登录授权，则 Spring Boot 默认会跳到登录页面，需要输入默认的用户信息才能访问端点，功能和原理与第 3 章介绍的一样，这里不再赘述。如果不想使用 Spring Boot 自动配置的默认机制，则可以注册一个 SecurityFilterChain 实现自定义的安全机制。

自定义安全机制的示例代码如下：

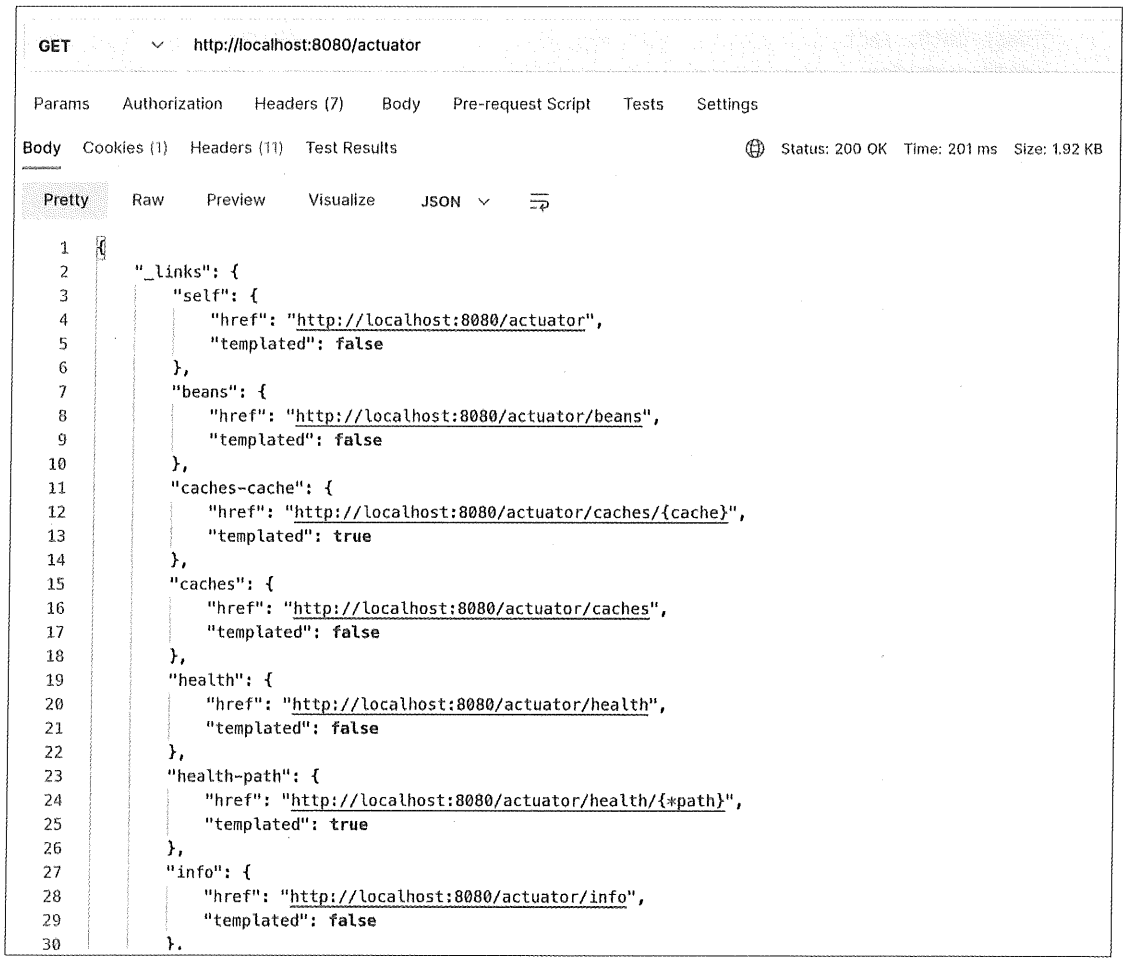
```
@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http.authorizeHttpRequests((authorize) -> {
                    authorize.requestMatchers("/").permitAll()
                            .requestMatchers(EndpointRequest.to("health")).hasRole
("ENDPOINT_ADMIN")
                            .requestMatchers(EndpointRequest.toAnyEndpoint()).permitAll();
                }).formLogin(withDefaults())
                .logout().logoutSuccessUrl("/")
                .and().build();
    }

    @Bean
    protected UserDetailsService userDetailsService() {
        InMemoryUserDetailsManager manager = new InMemoryUserDetailsManager();
        manager.createUser(User.withUsername("test").password("{noop}test")
                .roles("ENDPOINT_ADMIN", "ADMIN", "TEST").build());
        manager.createUser(User.withUsername("root").password("{noop}root")
                .roles("ADMIN").build());
        return manager;
    }

}
```

端点授权配置需要用到 EndpointRequest 类，这里配置了只有 health 端点需要 ENDPOINT_ADMIN 角色才能访问，其他接口的 URL 和端点不需要授权都能正常访问。

然后创建了两个授权用户，{noop} 表示明文密码，只有 test 用户才拥有 ENDPOINT_ADMIN 角色。重启应用，只有输入拥有 ENDPOINT_ADMIN 角色的 test 用户才能访问 health 端点，输入其他用户会返回 403 错误。

12.2.6 自定义端点映射

默认情况下通过 /actuator 根路径可以显示所有暴露出来的端点，如下图所示。



这样不是很安全，可以通过以下配置禁用：

```
management:
  endpoints:
    web:
      discovery:
        enabled: false
```

还可以更改默认的端点映射，因为默认的端点的 URL 前缀和端点 ID 所有人都知道，不是很安全，可以通过配置更改所有端点默认的前缀 URL 及端点 ID，比如以下配置示例：

```
management:
  endpoints:
    web:
      base-path: /act
      path-mapping:
        health: hth
```

这里把默认的端点前缀映射从 /actuator 改为 /act，把 health 健康端点改名为 hth，现在就能只通过 /act/hth 访问健康端点了，之前的映射就失效了。

另外，端点映射的 IP 地址和端口也可以修改，默认和 Web 应用共享一个 Server，可以把端点 Server 独立出来，配置示例如下：

```
management:
  server:
    port: 8088
    address: 127.0.0.1
```

这样表示只能通过 localhost:8088 访问端点，address 和 port 是配套使用的，address 并不是必需的。

重启应用，通过日志可以发现同时启动了两个 Server，如下图所示。

```
main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8080 (http)
main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
main] org.apache.catalina.core.StandardEngine  : Starting Servlet engine: [Apache Tomcat/10.0.22]
main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1195 ms
main] o.s.s.web.DefaultSecurityFilterChain     : Will secure any request with [org.springframework.security.web.se
main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8080 (http) with context path ''
main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8088 (http)
main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
main] org.apache.catalina.core.StandardEngine  : Starting Servlet engine: [Apache Tomcat/10.0.22]
main] o.a.c.c.C.[Tomcat-1].[localhost].[/]     : Initializing Spring embedded WebApplicationContext
main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 70 ms
main] o.s.b.a.e.web.EndpointLinksResolver      : Exposing 13 endpoint(s) beneath base path '/actuator'
main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8088 (http) with context path ''
main] c.j.springboot.actuator.Application      : Started Application in 2.791 seconds (process running for 3.116)
```

这样应用的端口就和端点管理端口分开了，8080用于应用，8088用于端点管理。既然是分开的，那么管理端点的Server要开启HTTPS访问就需要通过自己的management.server.ssl.*系列参数控制。

另外，把端点Server端点改为-1还能禁止以HTTP的方式暴露所有端点，和以exclude方式排除暴露的端点是一样的效果，如以下配置所示。

```
# 两者功能是一样的
management:
  server:
    port: -1

management:
  endpoints:
    web:
      exposure:
        exclude: "*"
```

所以，通过关闭显示所有端点的页面和更改默认的端点映射与端口，也能起到保护端点安全的目的。

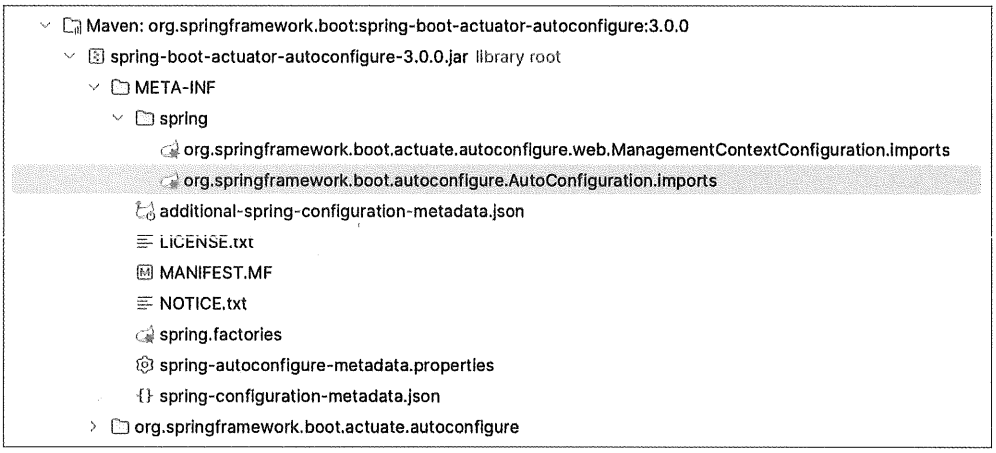
12.2.7 端点实现机制

内置端点一旦启用，Spring Boot就会自动配置，比如health端点的自动配置类为HealthEndpointAutoConfiguration，其源码如下：

```
@AutoConfiguration
@ConditionalOnAvailableEndpoint(endpoint = HealthEndpoint.class)
@EnableConfigurationProperties(HealthEndpointProperties.class)
@Import({ HealthEndpointConfiguration.class, ReactiveHealthEndpointConfiguration.class,
        HealthEndpointWebExtensionConfiguration.class, HealthEndpointReactiveWeb-
        ExtensionConfiguration.class })
public class HealthEndpointAutoConfiguration {

}
```

自动配置类是空的，其实是通过@Import注解导入了不同Web类型的端点配置类。

根据*EndpointAutoConfiguration这个自动配置命名规则，可以很容易地找到其他端点的自动配置类及端点类，端点自动配置类都被注册在新的org.springframework.boot.autoconfigure.AutoConfiguration.imports自动配置文件中，如下图所示。



端点具体的实现类命名规则为 *Endpoint，比如健康端点的实现类为 HealthEndpoint，部分源码如下：

```
@Endpoint(id = "health")
public class HealthEndpoint extends HealthEndpointSupport<HealthContributor,
HealthComponent> {
    public static final EndpointId ID = EndpointId.of("health");

    ...

    @ReadOperation
    public HealthComponent health() {
        HealthComponent health = health(ApiVersion.V3, EMPTY_PATH);
        return (health != null) ? health : DEFAULT_HEALTH;
    }

    @ReadOperation
    public HealthComponent healthForPath(@Selector(match = Match.ALL_REMAINING)
String... path) {
        return health(ApiVersion.V3, path);
    }

    ...

}
```

端点实现类通过一个 @Endpoint 注解修饰，并且需要注册为 Spring 中的 Bean，在 Web 应用中支持 JMX 和 Web 两种暴露方式，也支持 @JmxEndpoint 和 @WebEndpoint 注解以不同的方式暴露，还可以通过 @EndpointJmxExtension 和 @EndpointWebExtension 注解扩展现有的端点。

通过在方法上定义不同的操作注解以实现不同 HTTP 方法与应用交互，支持的操作注解如下表所示。

操作注解	HTTP 方法
@ReadOperation	GET
@WriteOperation	POST
@DeleteOperation	DELETE

如果 Health 端点中定义了两个 @ReadOperation 注解，则说明这个端点只支持 HTTP GET 方法的请求，一个不带参数，另一个通过 @Selector 注解绑定 Path 路径上的参数。

另外，POST 请求 Content-Type 只能接受“application/vnd.spring-boot.actuator.v2+json、application/json”类型，所有方法的返回 Content-Type 则根据不同的返回类型决定，如下表所示。

方法返回	Content-Type
Void，Void	空
org.springframework.core.io.Resource	application/octet-stream
其他类型	application/vnd.spring-boot.actuator.v2+json，application/json

12.2.8 自定义端点

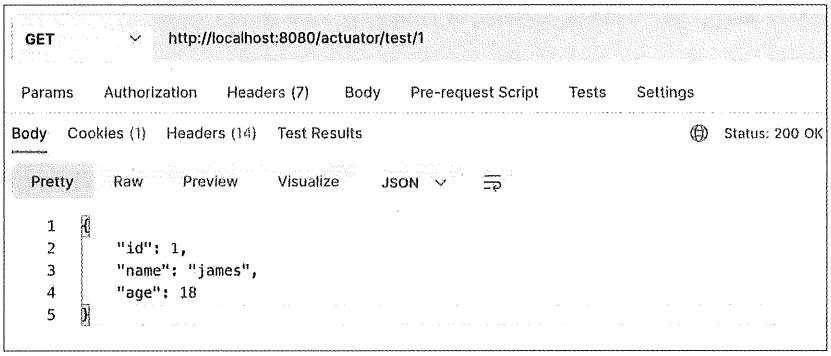
了解了端点实现机制，下面实现一个自定义的端点，示例如下：

```
@Component
@WebEndpoint(id = "test")
public class TestEndpoint {

    @ReadOperation
    public User getUser(@Selector Integer id) {
        return new User(id, "james", 18);
    }

    @WriteOperation
    public User updateUser(int id, @Nullable String name, @Nullable Integer age) {
        User user = getUser(id);
        user.setName(StringUtils.defaultIfBlank(name, user.getName()));
        user.setAge(ObjectUtils.defaultIfNull(age, user.getAge()));
        return user;
    }

}
```

这里创建了一个 TestEndpoint 端点类并通过 @Component 注册到 Spring 中，@WebEndpoint 注解表示只暴露为 HTTP 方式访问，端点类提供了两个方法，用于获取用户信息和更新用户信息。端点 ID 为 test，即可以通过 GET、POST 方法调用 /actuator/test 端点。

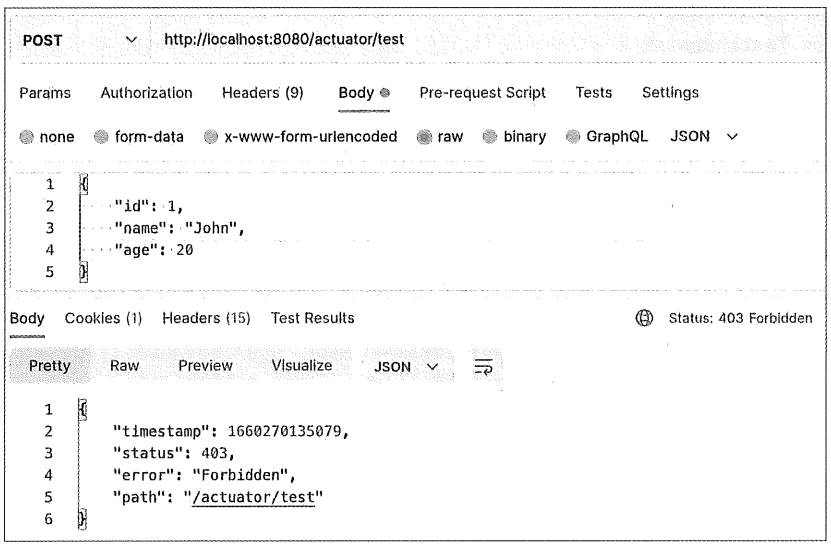
由于 @WebEndpoint 等端点注解的 enableByDefault 参数默认是 true，所以新建的端点默认是启用状态的，不需要再额外配置，虽然 @WebEndpoint 注解允许暴露为 HTTP 方式，但并未真正开启暴露。因为在之前的章节中已经把 Web 方式暴露为“*”了，所以这里也不需要再配置了。

发起 GET 请求，如下图所示。



/actuator/test/1 后面的 1 是通过 @Selector 注解绑定的 URL 路径上的参数，默认只绑定单个参数，如果要绑定多个，则可以指定 @Selector(match = Match.ALL_REMAINING) 匹配所有 URL 路径上的参数。

再发起 POST 请求，如下图所示。



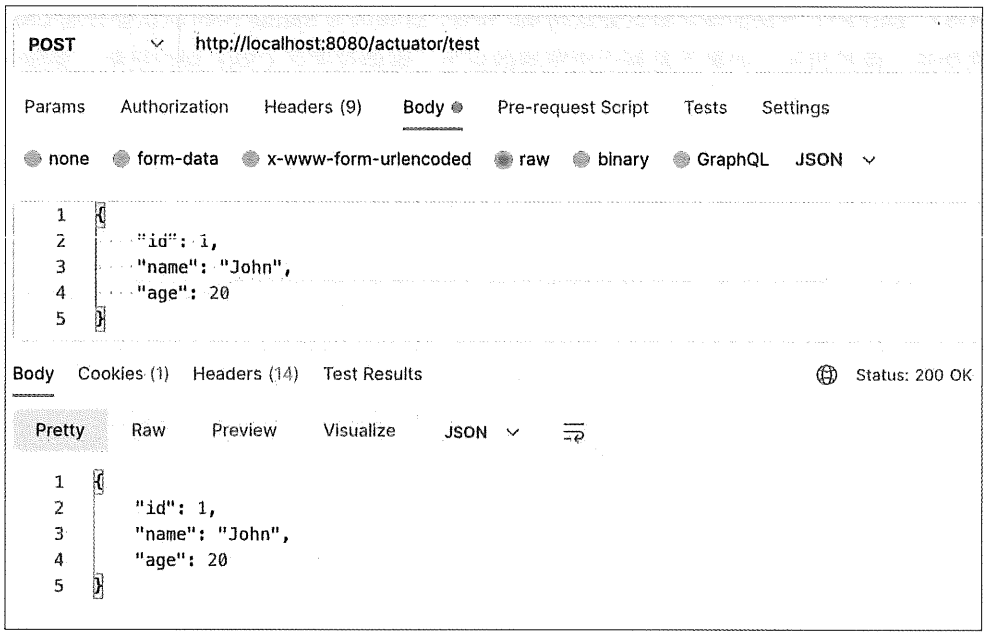
POST、DELETE 方法的请求默认会返回 403 错误，这是因为 Spring Boot 默认会开启 CSRF（跨站请求伪造）保护机制，如果不是基于浏览器的应用，则建议关闭 CSRF 保护机制，关闭示例代码如下：

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http.authorizeHttpRequests((authorize) -> {
                authorize.requestMatchers("/").permitAll()
                        .requestMatchers(EndpointRequest.to("health")).hasRole
("ENDPOINT_ADMIN")
                        .requestMatchers(EndpointRequest.toAnyEndpoint()).permitAll();
            })
            .csrf(csrf -> csrf.disable())
            .formLogin(withDefaults())
            .logout().logoutSuccessUrl("/")
            .and().build();
}
```

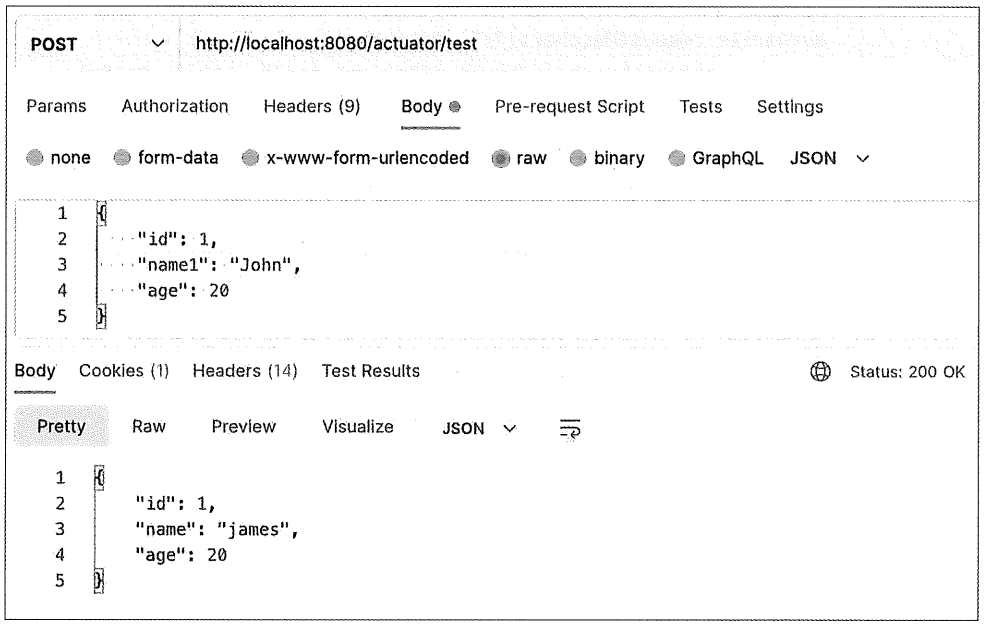
或者只对端点映射关闭 CSRF 保护机制，如以下配置示例所示。

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http.authorizeHttpRequests((authorize) -> {
                authorize.requestMatchers("/").permitAll()
                        .requestMatchers(EndpointRequest.to("health")).hasRole
("ENDPOINT_ADMIN")
                        .requestMatchers(EndpointRequest.toAnyEndpoint()).permitAll();
            })
//          .csrf(csrf -> csrf.disable())
            .csrf(csrf -> csrf.ignoringRequestMatchers(EndpointRequest.toAnyEndpoint()))
            .formLogin(withDefaults())
            .logout().logoutSuccessUrl("/")
            .and().build();
}
```

关闭 CSRF 保护机制后，再重启应用以 POST 方式调用自定义端点，如下图所示。



如果端点类操作方法定义了参数，则参数不能为空，否则返回 400 错误，通过在方法参数前添加 @Nullable 注解就可以允许参数为空，把 name 参数名改为 name1 后再调用，操作不会返回异常，而是返回默认值，如下图所示。



另外，端点也支持跨域调用，如以下跨域配置所示。

```
management:
  endpoints:
    web:
      cors:
        allowed-origins: "https://javastack.cn"
        allowed-methods: "GET,POST"
```

这里配置了可以允许来自 https://javastack.cn 网站所有 GET 和 POST 方法的请求，这两个配置也支持“*”通配符。

12.3 loggers（日志端点）

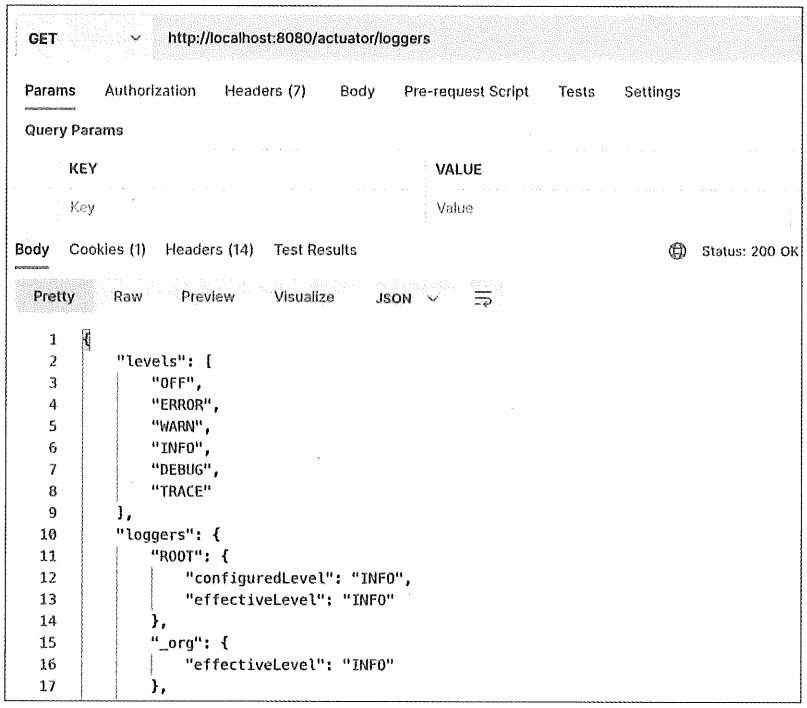
Spring Boot 提供了一个 loggers 日志端点，对应的端点类为 LoggersEndpoint，核心源码如下：

```
@Endpoint(id = "loggers")
@RegisterReflectionForBinding({ GroupLoggerLevelsDescriptor.class, SingleLoggerLevels-
Descriptor.class })
public class LoggersEndpoint {

    ...

    @ReadOperation
    public LoggersDescriptor loggers() {
        Collection<LoggerConfiguration> configurations = this.loggingSystem.getLogger-
Configurations();
        if (configurations == null) {
            return LoggersDescriptor.NONE;
        }
        return new LoggersDescriptor(getLevels(), getLoggers(configurations), getGroups());
    }

    @ReadOperation
    public LoggerLevelsDescriptor loggerLevels(@Selector String name) {
        Assert.notNull(name, "Name must not be null");
        LoggerGroup group = this.loggerGroups.get(name);
        if (group != null) {
            return new GroupLoggerLevelsDescriptor(group.getConfiguredLevel(),
group.getMembers());
        }
        LoggerConfiguration configuration = this.loggingSystem.getLoggerConfiguration(name);
```

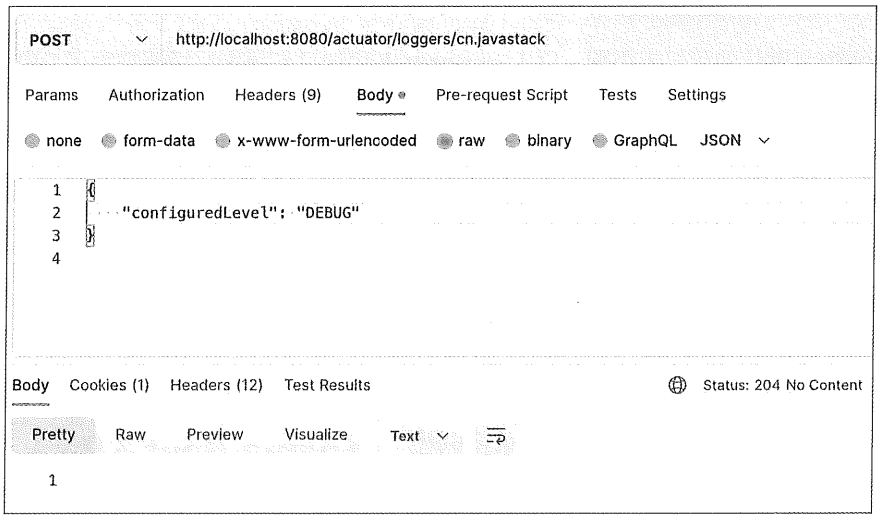
```
        return (configuration != null) ? new SingleLoggerLevelsDescriptor(configuration): null;
    }

    @WriteOperation
    public void configureLogLevel(@Selector String name, @Nullable LogLevel configuredLevel) {
        Assert.notNull(name, "Name must not be empty");
        LoggerGroup group = this.loggerGroups.get(name);
        if (group != null && group.hasMembers()) {
            group.configureLogLevel(configuredLevel, this.loggingSystem::setLogLevel);
            return;
        }
        this.loggingSystem.setLogLevel(name, configuredLevel);
    }

    ...

}
```

日志端点提供了两个读操作及一个写操作，可以查看正在运行的应用的日志级别设置，也可以动态调整某个包的日志级别，该端点默认不会暴露，需要自行暴露，详见之前章节的介绍。

获取日志端点信息（GET），如下图所示。



调整某个包的日志级别（POST），如下图所示。



这里调整 cn.javastack 包的日志级别为 DEBUG，再重新获取日志端点信息，如下图所示。

如上图所示，该包的日志级别已经设置为 DEBUG 了。

loggers日志端点支持设置的日志级别有TRACE、DEBUG、INFO、WARN、ERROR、FATAL、OFF、null（恢复默认设置）。

null表示恢复默认的包日志级别，以上cn.javastack包的日志级别已经调整为了DEBUG，发送null就会恢复默认的INFO日志级别，发送的内容如下：

```
{
    "configuredLevel": null
}
```

loggers日志端点目前只能用来查看和更新包的日志级别，以及查看所有的日志分组，但不支持查看和更新复杂的日志配置。

12.4 Observability（可观测性，Spring Boot 3.0+）

Observability（可观测性）是指从系统外部观测正在运行的系统的内部状态的能力，Spring Boot 3.0通过Micrometer和Micrometer追踪来提高应用的可观测性，支持集成Micrometer 1.10+，引入了全新的可观测API并自动配置Micrometer追踪，包括对Brave、OpenTelemetry、Zipkin和Wavefront组件的支持。

使用Micrometer可观测API完成观测后，可将观测数据主动报告给Zipkin等组件，并支持自定义追踪参数的配置，这个新特性可以更好地帮助我们了解并监控应用的健康状况。

Observability主要由以下三大组件组成：

- Logging（日志记录）。
- Metrics（度量/指标）。
- Traces（链路跟踪，Spring Boot 3.0+）。

日志就是指平常的日志输出，对于指标和跟踪，Spring Boot使用的是Micrometer Observation，如果要创建自己的Observations，则可以注入一个ObservationRegistry，并使用它添加对应的数据，如下面的示例所示。

```
@Slf4j
@RequiredArgsConstructor
@Component
public class IndexObservation {
```

```
    private final ObservationRegistry observationRegistry;

    public void observe() {
        Observation.createNotStarted("indexObservation", this.observationRegistry)
                .lowCardinalityKeyValue("area", "cn")
                .highCardinalityKeyValue("userId", "10099")
                .observe(() -> {
                    // 执行观测时的业务逻辑
                    log.info("开始执行业务逻辑 ...");
                });
    }

}
```

几个核心方法如下：

- createNotStarted：创建一个 Observation，但不会马上启动观测。
- lowCardinalityKeyValue：指定低基数的标签，它们将被添加到 Metrics 和 Traces 中。
- highCardinalityKeyValue：指定高基数的标签，它们将被添加到 Traces 中。
- observe：开始执行当前 Observation 观测，并传入一个 Runnable 线程用于执行业务逻辑。

observe方法接收一个 Runnable 线程参数，其源码如下：

```
default void observe(Runnable runnable) {
    start();
    try (Scope scope = openScope()) {
        runnable.run();
    }
    catch (Exception exception) {
        error(exception);
        throw exception;
    }
    finally {
        stop();
    }
}
```

它主要由以下几个过程组成：

- 启动 Observation。
- 打开一个 Scope。

- 调用线程的 Runnable.run() 方法。
- 关闭 Scope。
- 如果有错误，则向观测对象发出错误信号。
- 停止 Observation。

现在改造一下 /首页接口，让首页执行 IndexObservation 观测，如下面的示例所示。

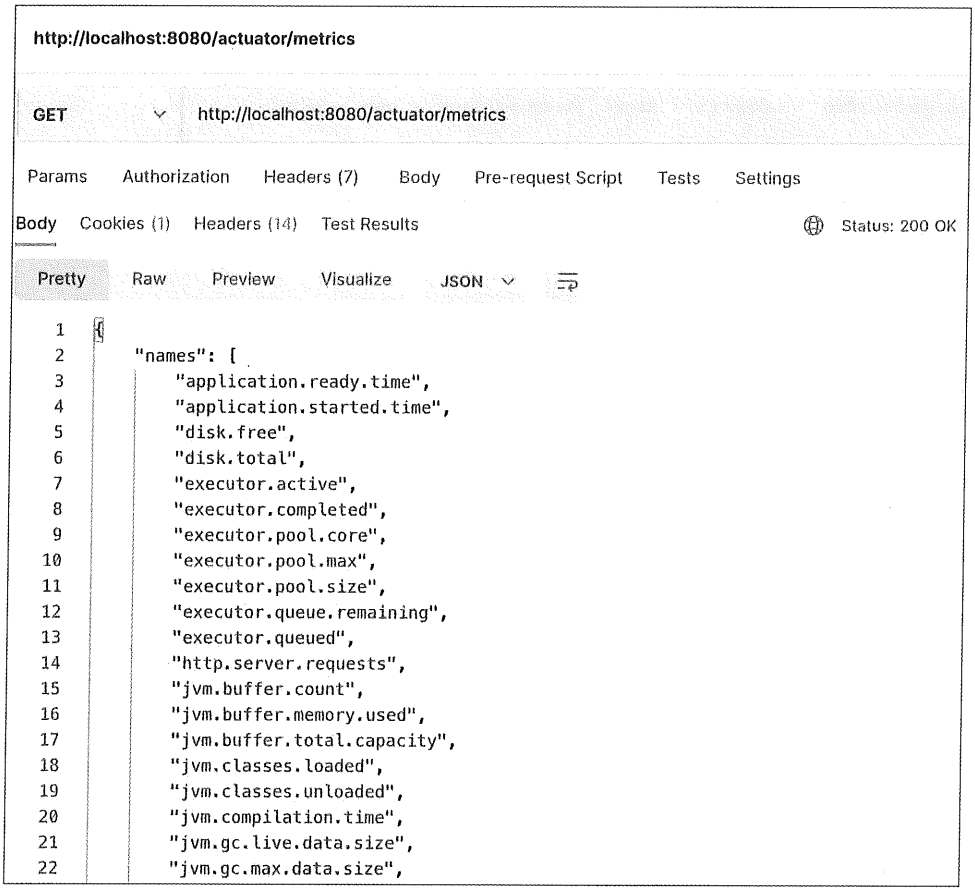
```
@RequiredArgsConstructor
@Slf4j
@Controller
public class LoginController {

    private final IndexObservation indexObservation;

    @GetMapping("/")
    @ResponseBody
    public String index() {
        log.info("this is index page.");
        indexObservation.observe();
        return "index page.";
    }

}
```

注入 IndexObservation 并调用其 indexObservation 方法开始执行 Observation 观测，然后就会记录首页观测到的相关标签数据，可在链路跟踪中查找并展示，具体效果见后续的 Traces 链路跟踪章节。

12.5 Metrics（指标）

12.5.1 内置指标

Spring Boot 内置了各种技术的指标的实现，比如 JVM、System、Tomcat、Logger、Spring MVC 等，通过 /actuator/metrics 指标端点可以显示所有应用收集到的指标，如下图所示。



此端点默认是不暴露的，需要自行暴露。另外还可以通过 /actuator/metrics/xxx 查看详细指标信息，比如可通过 /actuator/metrics/jvm.memory.max 路径查看 JVM 最大内存，如下图所示。

有了这些指标端点为前提，监控系统才能收集并展示相关信息。

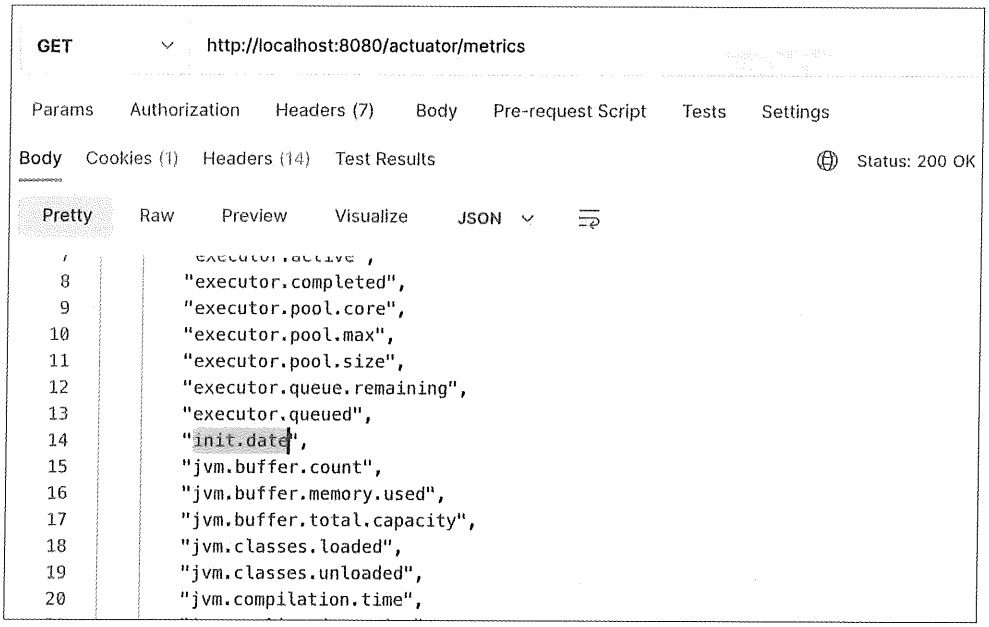
12.5.2 自定义指标

如果 Spring Boot 提供的指标不符合要求，则可以通过注册多个 MeterBinder 实现多个自定义指标，比如下面的自定义指标示例：

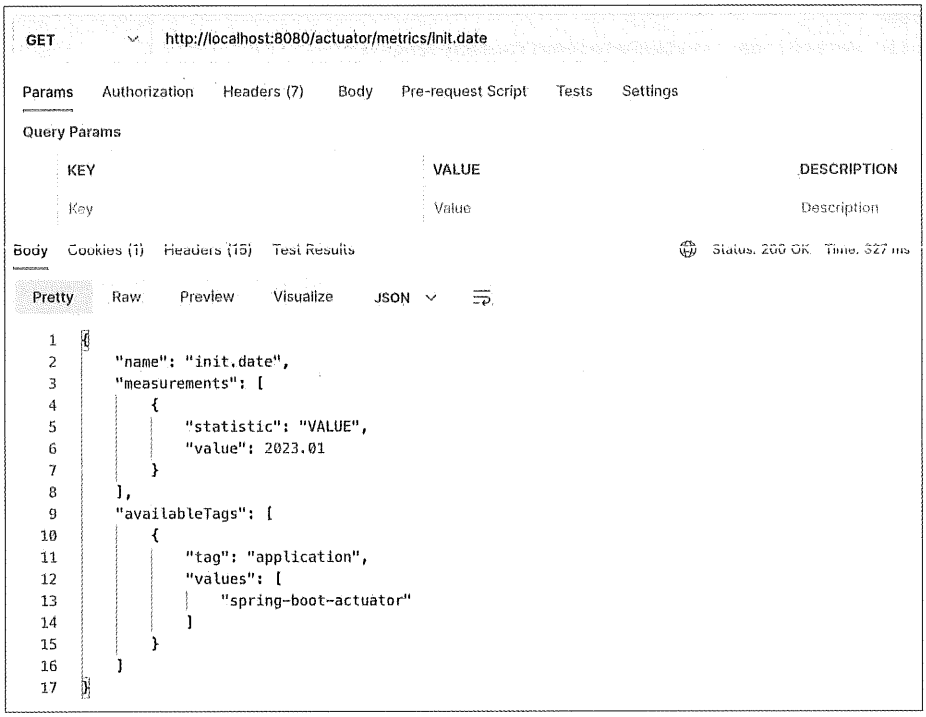
```
@Configuration
public class MetricsConfig {

    @Bean
    public MeterBinder initDate(Environment env) {
        return (registry) -> Gauge.builder("init.date", this::date).register(registry);
    }
```

```
    @Bean
    public MeterBinder systemDate(Environment env) {
        return (registry) -> Gauge.builder("system.date", this::date).register(registry);
    }

    private Number date() {
        return 2022.01;
    }

}
```

这里创建了两个测试指标，通过 /actuator/metrics 或者 /actuator/prometheus 端点可以验证指标是否被收集，如下图所示。



然后获取该指标的数据，如下图所示。



12.6 Traces（链路跟踪，Spring Boot 3.0+）

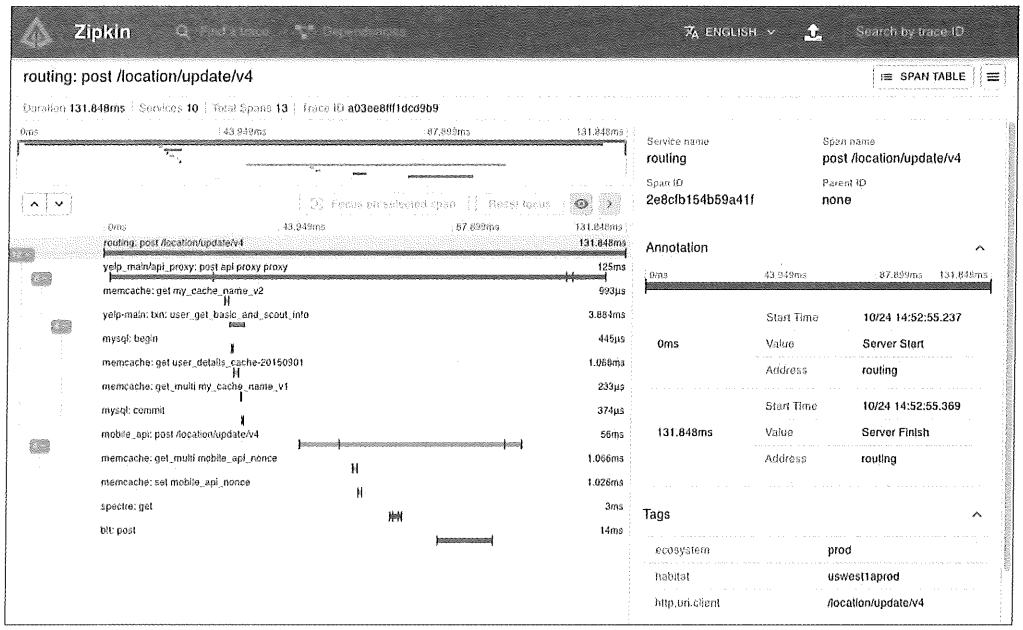
12.6.1 概述

Spring Boot Actuator 为 Micrometer Tracing 提供了依赖管理和自动配置，Micrometer 是当前比较流行的跟踪类库门面，Spring Boot 为以下 Tracers 链路跟踪解决方案提供了自动配置：

- 使用 OpenTelemetry 结合 Zipkin 或者 Wavefront。
- 使用 OpenZipkin Brave 结合 Zipkin 或者 Wavefront。

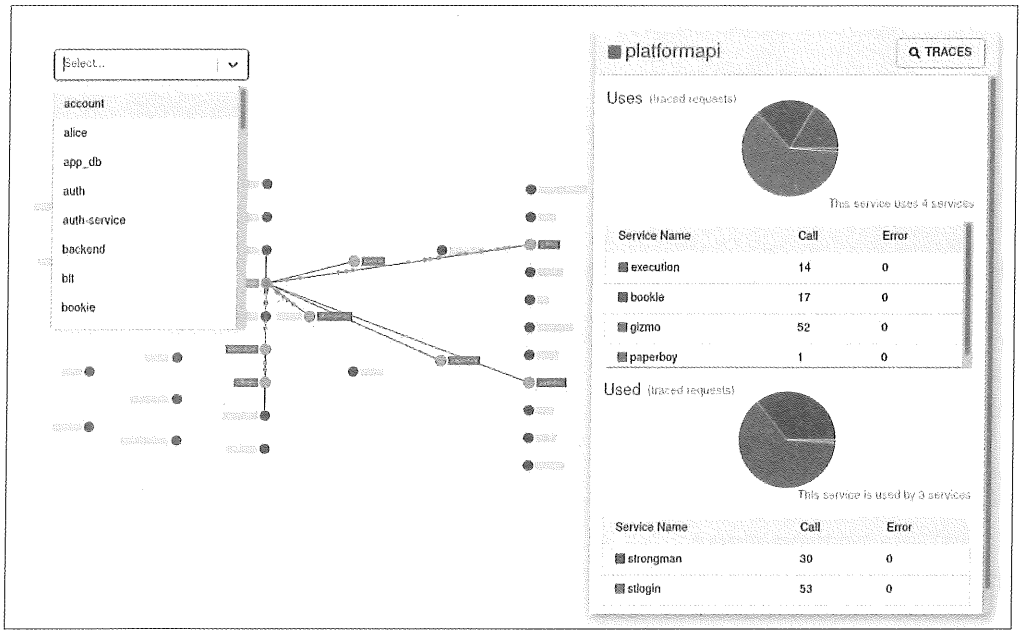
OpenTelemetry 是一款开源的可观测框架及数据收集中间件，可以使用它来生成、收集和导出观测数据（Metrics、Logs 和 Traces），它只是收集数据，分析和展示数据还需要结合其他中间件。

Zipkin 是由 Twitter 开源的一个分布式追踪系统，它可以收集、解决服务中的延迟问题所需的时间数据，包括收集和查找此数据并进行展示，如果日志中有 Trace ID，则直接可以展示，如下图所示。



也可以根据其他属性查询数据，数据展示非常详细。

Zipkin UI 还提供了一个依赖关系图，显示有多少跟踪请求通过了每个应用，这有助于识别聚合行为，包括错误路径或对已弃用服务的调用，如下图所示。



应用要使用 Zipkin 就需要向 Zipkin 发送跟踪数据，一般是通过 HTTP 或 Kafka 的方式，但也可以是 Apache ActiveMQ、gRPC 和 RabbitMQ 等，数据可以存储在内存中，也可以在 MySQL、Apache Cassandra 或 Elasticsearch 等组件中持久化存储。

12.6.2 链路跟踪环境搭建

这里使用 OpenTelemetry 结合 Zipkin 的方案来实现 Tracing 调用链跟踪，本地需要安装 Zipkin 环境。

Java 8+/Linux环境下最快安装 Zipkin 的方法的参考命令如下：

```
curl -sSL https://zipkin.io/quickstart.sh | bash -s
java -jar zipkin.jar &
```

本地可以访问链接 19 下载最新版再运行 Zipkin。

使用默认的配置，安装并运行 Zipkin 之后，访问控制台 http://localhost:9411，如下图所示。

应用需要导入相关依赖，如下面的依赖配置所示。

```
<!-- 将 Micrometer Observation API 桥接到 OpenTelemetry -->
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>

<!-- 向 Zipkin 报告调用链跟踪信息 -->
<dependency>
```

```
    <groupId>io.opentelemetry</groupId>
    <artifactId>opentelemetry-exporter-zipkin</artifactId>
</dependency>
```

添加采样参数，如下面的配置所示。

```
management:
  tracing:
    sampling:
      probability: 1.0
```

默认情况下，Spring Boot 仅对 10% 的请求进行采样（0.1），以防止 Traces 跟踪导致后端系统不堪重负，这里为了便于测试，将采样参数切换为 1.0，即对所有请求（100%）进行采样。

如果需要在日志中显示 Trace ID 和 Span ID，则可以进行以下设置：

```
logging:
  pattern:
    level: ${spring.application.name:},%X{traceId:-},%X{spanId:-}
```

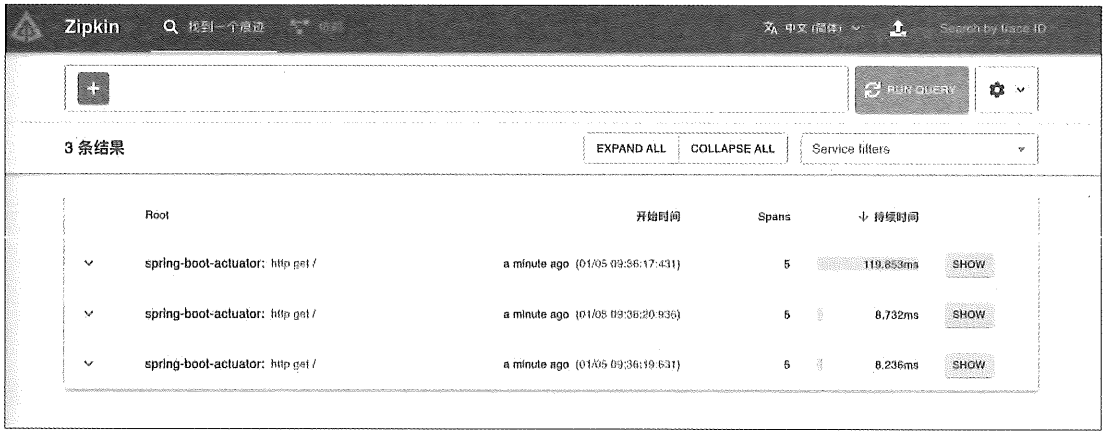
参数说明如下：

- Trace ID：链路跟踪 ID，一次请求完整链路的唯一标识，由一组 Span 单元组成的一个树状结构。
- Span ID：基本工作单元 ID，每条链路都由不同的 Span ID 组成。

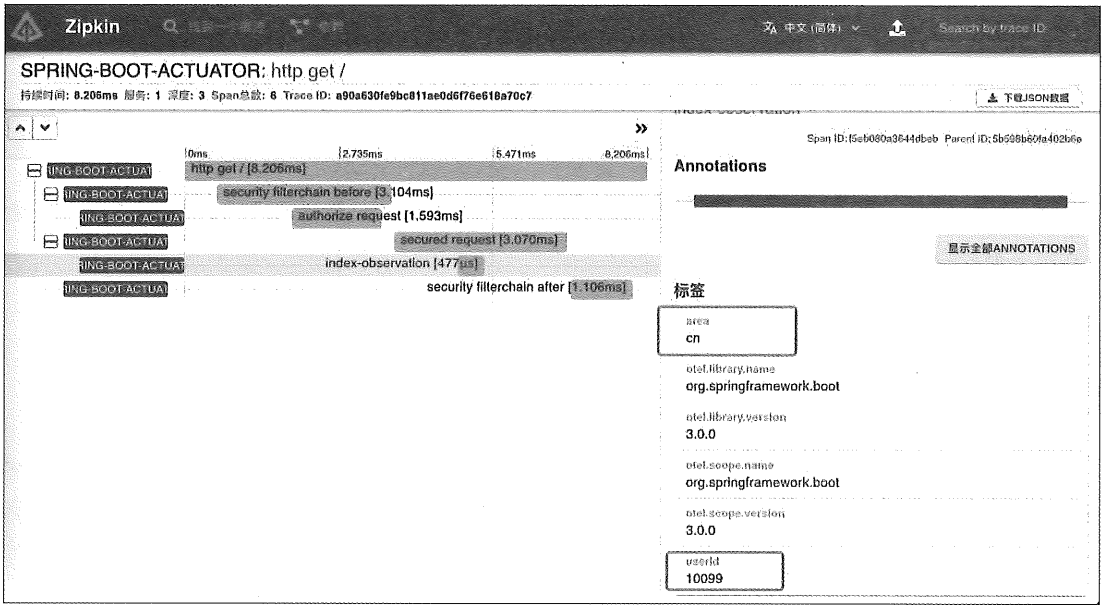
12.6.3 链路跟踪 / 展示

链路跟踪环境搭建好及 Zipkin 启动之后，再启动应用，并发起 HTTP 接口调用，Spring Boot 会为 HTTP 请求创建一个 Observation 观测，它桥接到 OpenTelemetry，然后通过 OpenTelemetry 向 Zipkin 报告每个链路的跟踪数据。

在 Observability可观测性一节中，已经为 / 首页实现了 IndexObservation 观测并记录了相关标签数据，现在连续调用三次 http://localhost 接口，然后回到 Zipkin 查询当前应用的调用链，如下图所示。



可以看到这三次的请求链路，随便单击一条进入详细链路，如下图所示。



可以看到首页观测记录及携带的 area 和 userId 标签数据，后台也记录了许多调用链跟踪日志，如下图所示。

```
11f2b34ee93367c82b9f08e68ed4be,64869e2e085dd2f3 66642 --- [nio-8080-exec-2] c.j.springboot.actuator.LoginController  : this is index page.
11f2b34ee93367c82b9f08e68ed4be,1bbfe6ddf3a27658 66642 --- [nio-8080-exec-2] c.j.s.a.observation.IndexObservation     : 开始执行业务逻辑...
981ce75bafe26fdc0890f3b7db6da0,c4c45a6aa86ba925 66642 --- [nio-8080-exec-3] c.j.springboot.actuator.LoginController  : this is index page.
981ce75bafe26fdc0890f3b7db6da0,f718955db3b49bb6 66642 --- [nio-8080-exec-3] c.j.s.a.observation.IndexObservation     : 开始执行业务逻辑...
37e924e5e4942b2e2690ea5bfec674,a47d674426f09624 66642 --- [nio-8080-exec-4] c.j.springboot.actuator.LoginController  : this is index page.
37e924e5e4942b2e2690ea5bfec674,7a479684f3df9f8d 66642 --- [nio-8080-exec-4] c.j.s.a.observation.IndexObservation     : 开始执行业务逻辑...
e37d9ada7a58a0194273fcfc03f956,5d960c5a22519737 66642 --- [nio-8080-exec-5] c.j.springboot.actuator.LoginController  : this is index page.
e37d9ada7a58a0194273fcfc03f956,a38455d2b0c44318 66642 --- [nio-8080-exec-5] c.j.s.a.observation.IndexObservation     : 开始执行业务逻辑...
fbffea7a6d02930091a9b043ca81cd,dea8128e55a6645a 66642 --- [nio-8080-exec-6] c.j.springboot.actuator.LoginController  : this is index page.
fbffea7a6d02930091a9b043ca81cd,0a8b29f3145e68f4 66642 --- [nio-8080-exec-6] c.j.s.a.observation.IndexObservation     : 开始执行业务逻辑...
806e0e73d302017d5813ec879aeb69,6702204cac426ec6 66642 --- [nio-8080-exec-7] c.j.springboot.actuator.LoginController  : this is index page.
806e0e73d302017d5813ec879aeb69,3fd85238c7aa4f93 66642 --- [nio-8080-exec-7] c.j.s.a.observation.IndexObservation     : 开始执行业务逻辑...
```

可以看到每条链路输出了两条日志，也就是在 / 首页方法中输出的两条日志，其 Trace ID 相同，Span ID 不同。

Spring Boot 可观测性是针对 Spring Boot 项目的，在 Spring Cloud 微服务体系中一般推荐使用 Spring Cloud Sleuth+Zipkin 方案，但在 Spring Cloud 2022.0.0 发布后，Spring Cloud 移除了 Spring Cloud Sleuth 链路跟踪模块，如官方发布的文档所示。

- Spring Cloud Sleuth
  - This project has been removed from the release train. The core of this project has moved to Micrometer Tracing project and the instrumentations will be moved to Micrometer and all respective projects (no longer all instrumentations will be done in a single repository.

Spring Cloud Sleuth 链路跟踪模块也被 Micrometer 相关项目替代了，笔者升级到了 Spring Cloud 2022.0.0 后，发现已经不能下载 Spring Cloud Sleuth 依赖了，之前老版本的 Spring Cloud Sleuth 应用也要全部更换。

12.7 Spring Boot Admin

12.7.1 概述

Spring Boot Admin=Spring Boot+Admin，用于管理和监控 Spring Boot 应用，它并不是 Spring Boot 官方提供的，而是某个社区开源的。

开源地址为链接 20。

开源官网显示 Spring Boot Admin 的主要功能如下：

- 显示健康状况。
- 显示详细指标（如 JVM、内存、micrometer、数据源、缓存等）。

- 监控并下载日志文件。
- 显示 JVM 中的系统及环境变量。
- 显示 Spring Boot 配置属性。
- 简单的日志级别管理。
- 支持与 JMX Bean 交互。
- 显示线程堆栈。
- 下载堆信息。
- 显示 HTTP 跟踪信息。
- 显示 HTTP 端点。
- 显示计划任务。
- 显示 / 删除活动会话。
- 显示 Flyway/Liquibase迁移信息。
- 状态变更通知。

Spring Boot Admin 包括 Spring Boot Admin Server 和 Spring Boot Admin Client，Spring Boot Admin Client 即 Spring Boot 应用，以 HTTP 的方式向 Spring Boot Admin Server 注册，Spring Cloud 应用则通过 Eureka、Consul 等注册中心进行注册，Spring Boot Admin UI 只是基于 Spring Boot Actuator 端点之上的 Vue.js 应用。

12.7.2 环境搭建

Spring Boot 集成 Spring Boot Admin 需要先搭建 Spring Boot Admin Server，再搭建 Spring Boot Admin Client，Spring Boot Admin Server 用于收集 Spring Boot Admin Client 的健康数据并进行监控报警。

1. 搭建 Spring Boot Admin Server

添加 Spring Boot Admin Server 所需要的依赖，如以下依赖配置所示。

```
<dependency>
    <groupId>de.codecentric</groupId>
    <artifactId>spring-boot-admin-starter-server</artifactId>
    <version>${spring-boot-admin-starter-server.version}</version>
```

```
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

在启动类上添 @EnableAdminServer 注解以启用 Spring Boot Admin Server，如以下示例所示。

```
@EnableAdminServer
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }


}
```

2. 搭建 Spring Boot Admin Client

添加 Spring Boot Admin Client 所需要的依赖，如以下依赖配置所示。

```
<dependency>
    <groupId>de.codecentric</groupId>
    <artifactId>spring-boot-admin-starter-client</artifactId>
    <version>${spring-boot-admin-starter-client.version}</version>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

然后添加以下配置：

```
spring:
  boot:
    admin:
      client:
        url: http://localhost:8080 #1
```

```
management:
  endpoints:
    web:
      exposure:
        include: '*' #2
  info:
    env:
      enabled: true #3
```

核心参数说明如下：

- #1：指定需要注册的 Spring Boot Admin Server 的地址。
- #2：Spring Boot 3.0 默认只暴露 health 健康端点，这里全部暴露了，在生产环境中需谨慎选择。
- #3：Spring Boot 2.6+ 中的 env 信息默认是禁用的，这里需要手动开启。

然后禁用所有安全机制，让 actuator 所有端点可以被访问，如以下配置示例所示。

```
@Configuration
public class SecurityPermitAllConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.authorizeRequests().anyRequest().permitAll()
            .and().csrf().disable();
    }

}
```

这里为了测试，先禁用了所有安全措施，确保所有端点可以被访问，不然无法被监控到，实际项目中可自行设置安全性，具体可参考前面的端点安全性章节。

其他项目基础代码略，有需要的读者可以参考笔者提供的 Spring Boot 仓库的完整项目代码。

项目搭建和配置完成后，先后启动 Spring Boot Server 和 Spring Boot Client，必须先启动 Spring Boot Server，因为 Spring Boot Client 启动的时候需要向 Spring Boot Server 注册，否则会注册失败。

12.7.3 监控页面

1. 首页菜单

启动 Spring Boot Admin Server 和 Spring Boot Admin Client 后，打开 Spring Admin Server 控制台页面，地址如下：

http://localhost:8088/

打开首页，如下图所示。

服务端和客户端都运行成功了，Spring Boot Admin 3.x 比 2.x 版本的界面看起来更舒服了，单击对应的项目，可以看到更多的细节。

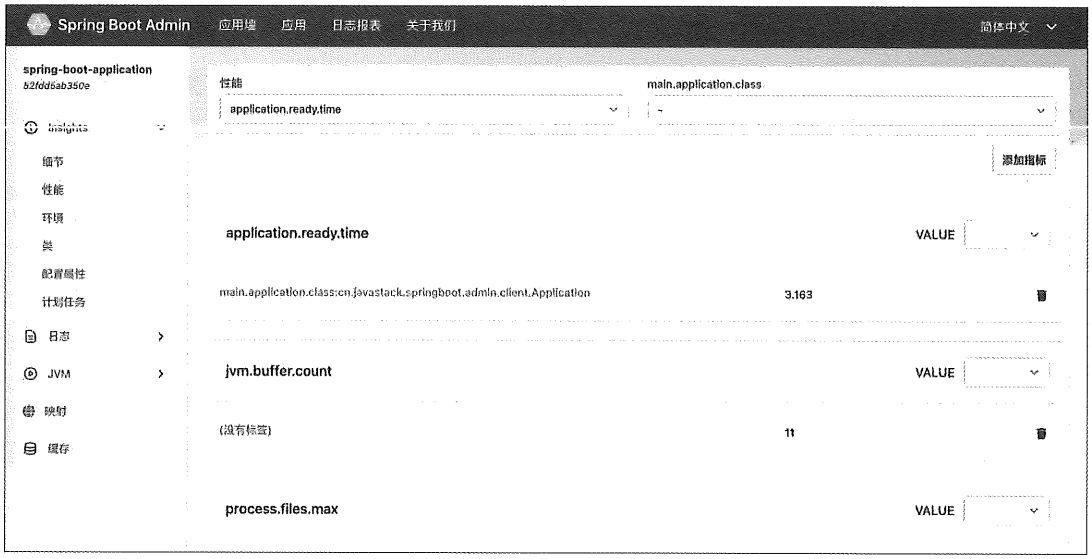
2. 细节菜单

细节菜单中展示了一些健康、JVM 相关信息，如下图所示。

这些信息有助于开发者掌握应用的 JVM 健康状况。

3. 性能菜单

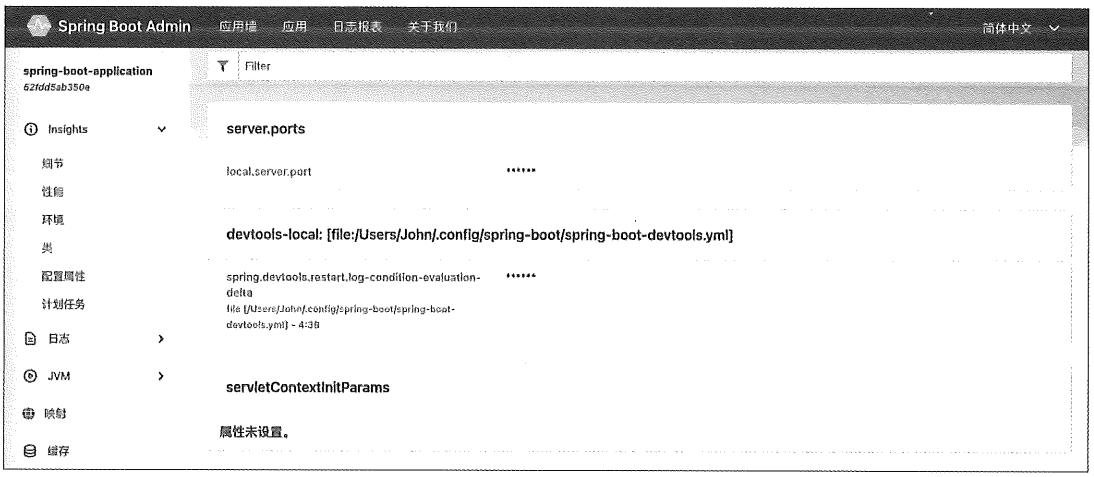
性能菜单中可以添加并显示各种性能指标，如下图所示。



这些指标有助于我们监控更多的应用运行时的性能参数。

4. 环境菜单

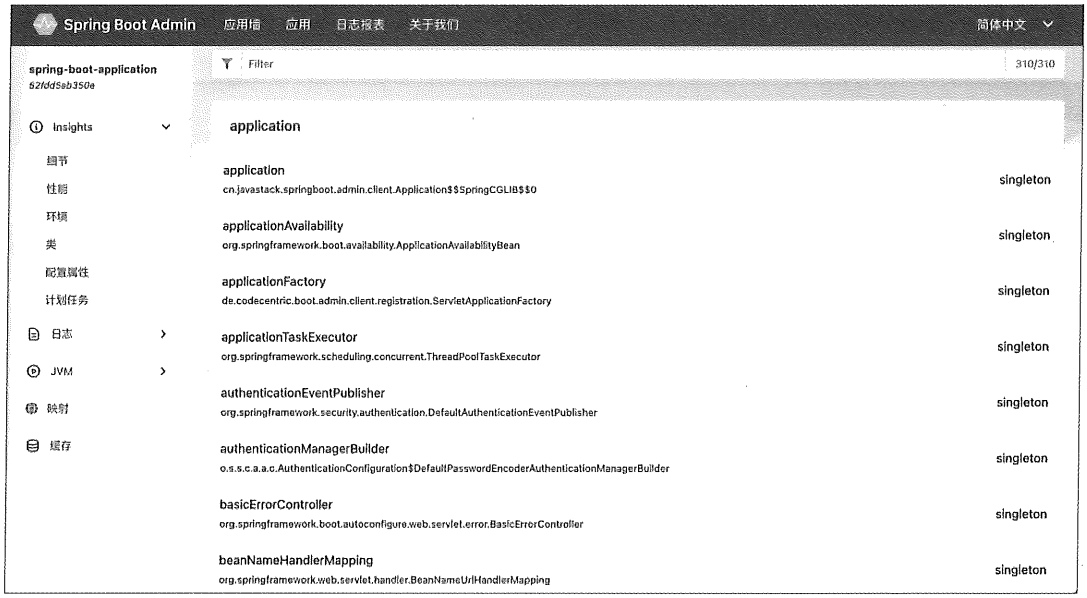
环境菜单中显示所有 Servlet 初始参数、系统、环境变量参数等，如下图所示。



可视化搜索应用的相关参数并展示，应用的相关参数就不必用 Linux 命令查询了。

5. 类菜单

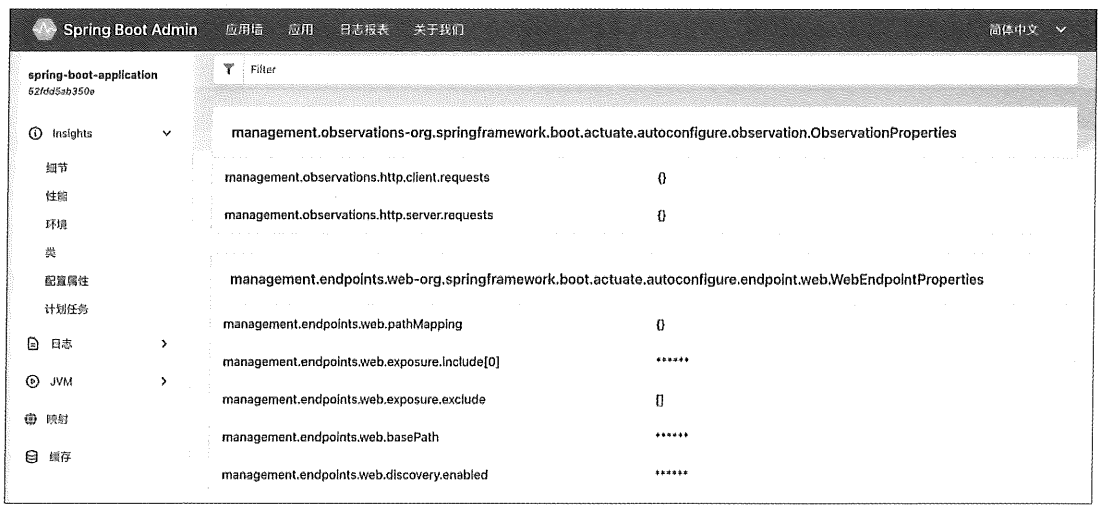
类菜单中显示所有 Spring 容器中的 Bean，以及是否单例，如下图所示。



这项功能对排查疑难问题很有用。

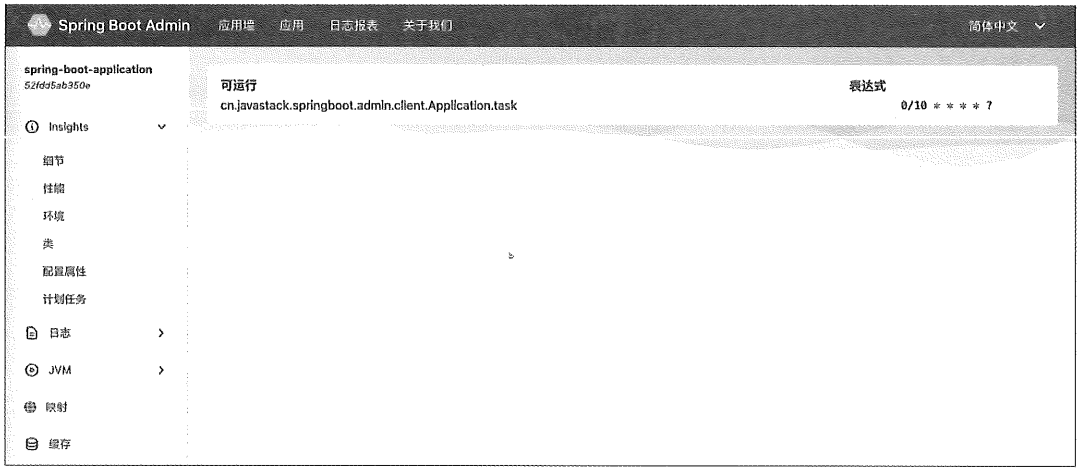
6. 配置属性菜单

配置属性菜单中显示所有 Spring Boot 的配置参数，如下图所示。



7. 计划任务菜单

计划任务菜单中显示所有的计划任务，如下图所示。



这里只是展示了任务信息及状态，并不能进行控制。

8. 日志配置菜单

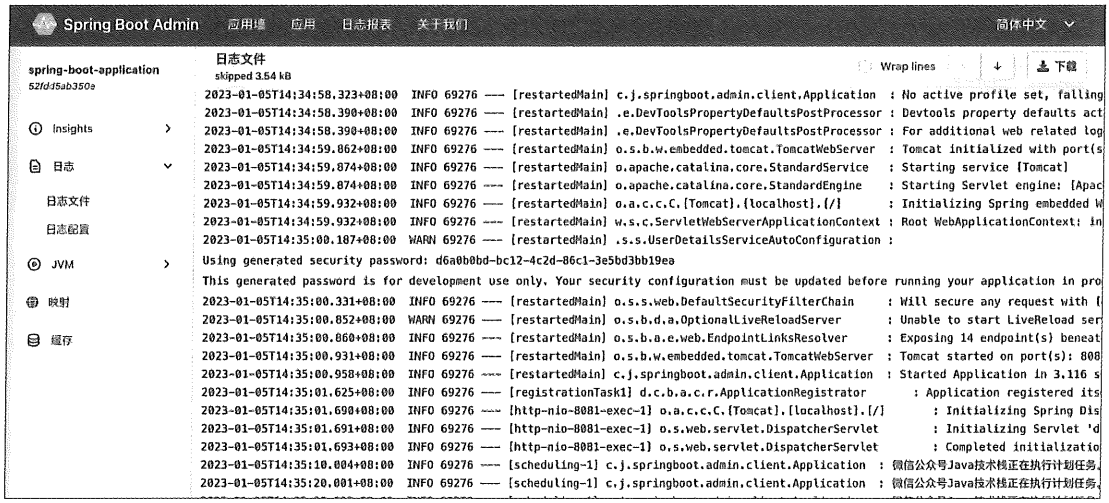
日志配置菜单中显示所有类的日志级别，并能修改某个类的日志级别，如下图所示。

默认情况下，日志文件无法通过日志端点访问，需要在客户端应用配置中配置 logging.file. path

或 logging.file.name 参数来启用 logfile 端点，如以下配置所示。

```
logging:
  file:
    name: ./logs/sba-client.log
```

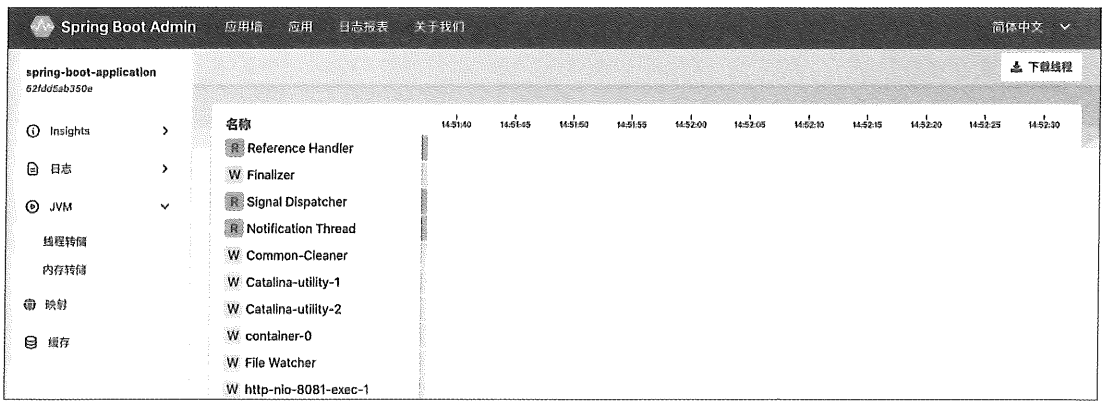
再重启客户端应用，刷新页面，如下图所示。

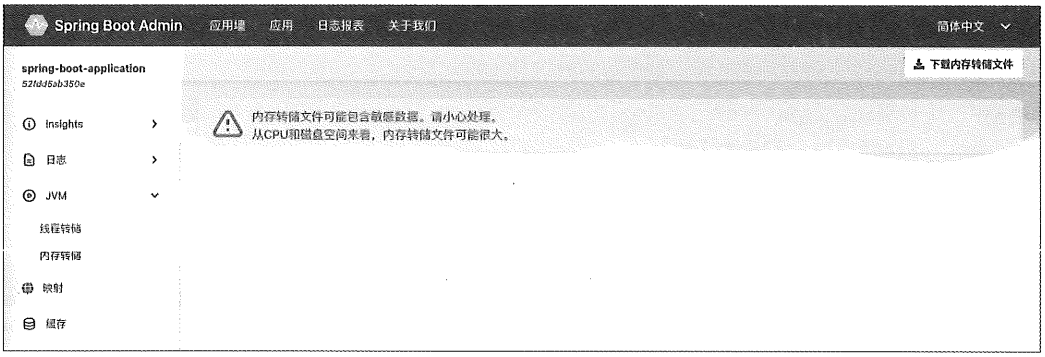


日志页面实时展示日志，不用再使用一堆命令查看了。

9. JVM 菜单

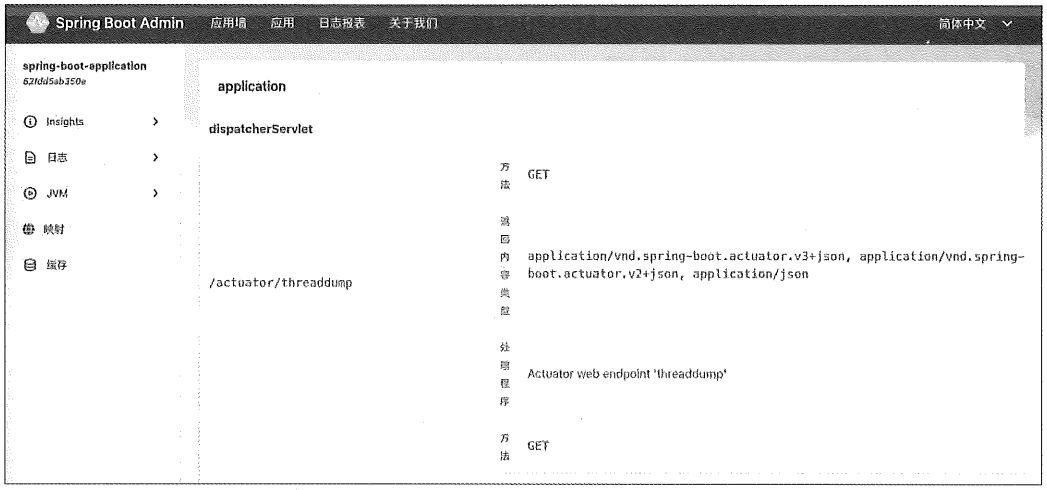
JVM 菜单中显示当前所有线程堆栈、堆栈，并能进行下载分析，如下面两个图所示。





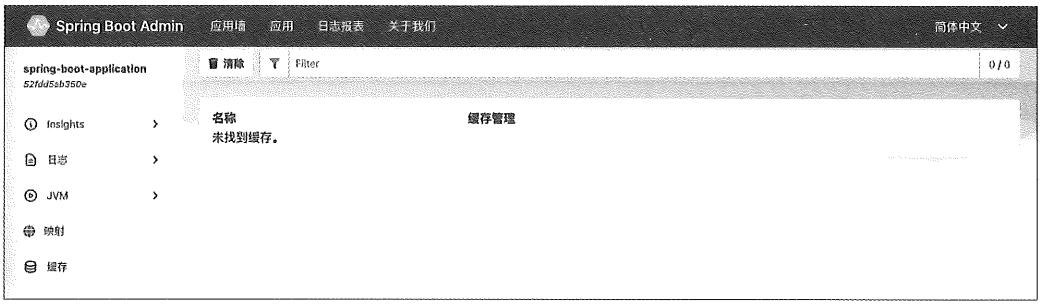
10. 映射菜单

映射菜单中显示所有的 URL 映射，如下图所示。



11. 缓存菜单

缓存菜单中显示所有缓存，如下图所示。



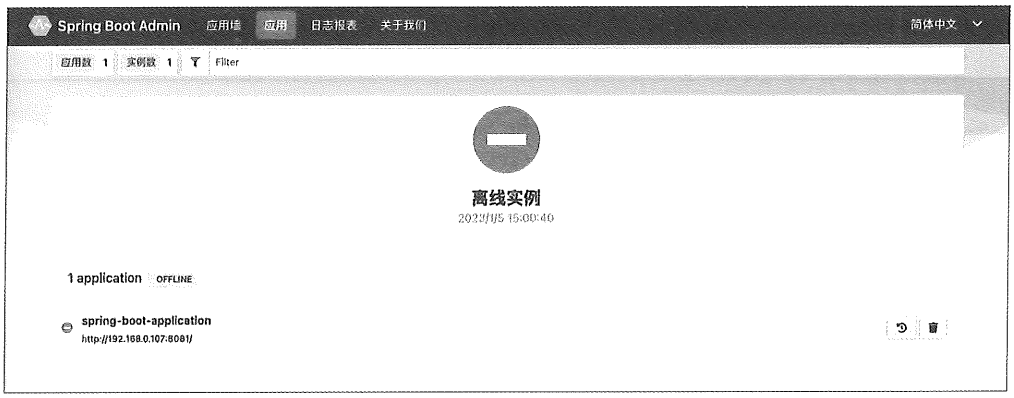
这里不仅是展示，还能清除和管理缓存。

Spring Boot Admin 用于管理和监控 Spring Boot 应用，具体就是通过客户端暴露的各种 /actuator 端点和指标实现的，并在此基础上进行了一些包装和 UI 展示。

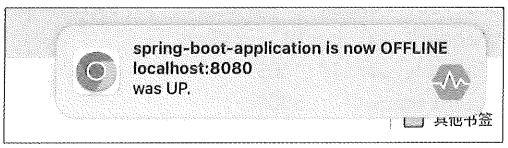
12.7.4 监控报警

1. 浏览器通知

现在把客户端停掉，若干秒后页面会自动刷新，如下图所示。



离线实例显示 1，应用出现在离线实例中了，如果浏览器（Chrome）没有禁用通知，那么同时会发出应用上线 / 下线的通知，如下图所示。



在 AdminServerAutoConfiguration 自动配置类中注册了一个状态变更触发器，如下面的源码所示。

```
@Bean(initMethod = "start", destroyMethod = "stop")
@ConditionalOnMissingBean
public StatusUpdateTrigger statusUpdateTrigger(StatusUpdater statusUpdater,
Publisher<InstanceEvent> events) {
    StatusUpdateTrigger trigger = new StatusUpdateTrigger(statusUpdater, events);
    trigger.setInterval(this.adminServerProperties.getMonitor().getStatusInterval());
    trigger.setLifetime(this.adminServerProperties.getMonitor().getStatusLifetime());
    return trigger;
}
```

默认每10秒会检测一下实例的状态，如果需要修改检测间隔时间，那么可以参考AdminServerProperties和MonitorProperties参数配置类进行配置。

2. 邮件通知

浏览器通知是比较被动的方式，由于不可能所有人都时时盯着计算机，因此这种方式不能及时通知所有需要关注告警信息的人。Spring Boot Admin支持在应用实例状态变更时报警通知，支持邮件、钉钉等多种通知方式。本节就以邮件作为示例，演示当监控到应用下线时如何及时发邮件进行报警通知。

首先在Spring Boot Admin Server项目中加入spring-boot-starter-mail邮件启动器，如下面的依赖配置所示。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

然后配置JavaMailSender及收件人信息，如下面的依赖配置示例所示。

```
spring:
  mail:
    host: smtp.exmail.qq.com
    username: xxx@xxx.com
    password: xxx
    properties:
      "[mail.smtp.socketFactory.class]": javax.net.ssl.SSLSocketFactory
      "[mail.smtp.socketFactory.fallback]": false
      "[mail.smtp.socketFactory.port]": 465
      "[mail.smtp.connectiontimeout]": 5000
      "[mail.smtp.timeout]": 3000
      "[mail.smtp.writetimeout]": 5000
  boot:
    admin:
      notify:
        mail:
              enabled: true
          from: Spring Boot Admin <xxx@xxx.com>
          to: xxx@xxx.com # 多个以，分隔
          cc: xxx@xxx.com # 多个以，分隔
```

这里只是简单地配置了发件人、收件人、抄送人等信息，更多的配置信息可以参考AdminServ

erNotifierAutoConfiguration.mailNotifier 方法注册的 MailNotifier 实例，源码如下：

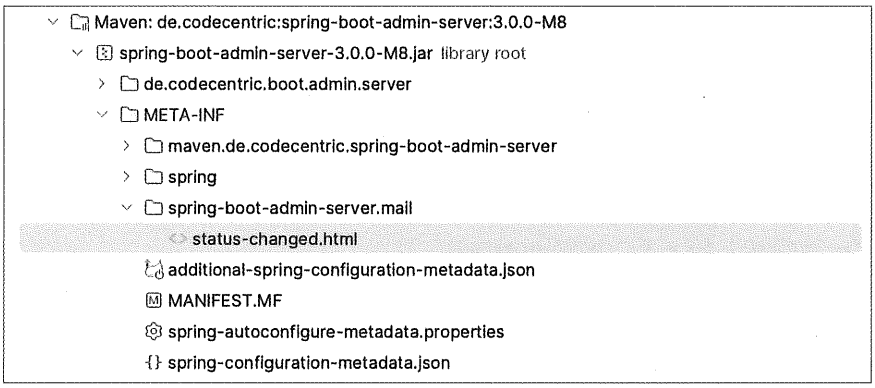
```
@Bean
@ConditionalOnMissingBean
@ConfigurationProperties("spring.boot.admin.notify.mail")
public MailNotifier mailNotifier(JavaMailSender mailSender, InstanceRepository repository,
        TemplateEngine mailNotifierTemplateEngine) {
    return new MailNotifier(mailSender, repository, mailNotifierTemplateEngine);
}
```

MailNotifier 类的源码如下：

```
public class MailNotifier extends AbstractStatusChangeNotifier {

    ...

    /**
     * recipients of the mail
     */
    private String[] to = { "root@localhost" };

    /**
     * cc-recipients of the mail
     */
    private String[] cc = {};

    /**
     * sender of the change
     */
    private String from = "Spring Boot Admin <noreply@localhost>";

    /**
     * Additional properties to be set for the template
     */
    private Map<String, Object> additionalProperties = new HashMap<>();

    /**
     * Base-URL used for hyperlinks in mail
     */
    @Nullable
    private String baseUrl;

    /**
```

```
     * Thymleaf template for mail
     */
    private String template = "classpath:/META-INF/spring-boot-admin-server/mail/
status-changed.html";

    ...

}
```

源码中显示的所有参数都是可配置的，可以看到邮件通知默认使用的是 Spring Boot Admin 内置的 Thymeleaf 模板，如下图所示。

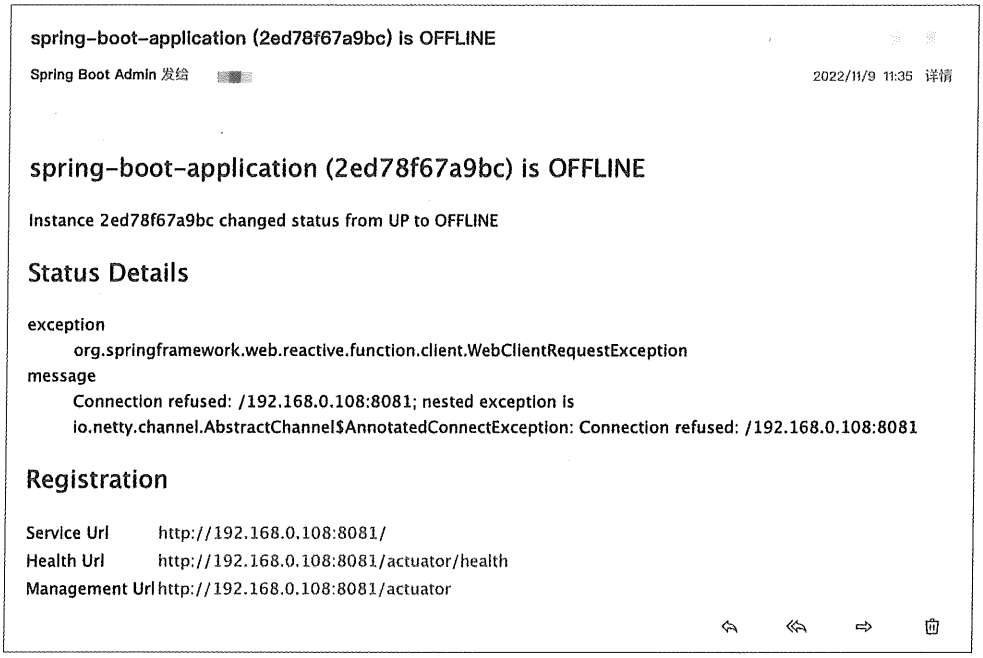


邮件通知会以 HTML 网页的形式发送，如果有需要，也可以更改此模板。配置好邮件通知参数之后，再重新启动 Spring Boot Admin Server 和 Spring Boot Admin Client，然后关闭 Spring Boot Admin Client 模拟应用掉线的场景。

从服务端控制台可以看到服务端无法连接到客户端应用的异常日志，如下图所示。

```
org.springframework.web.reactive.function.client.WebClientRequestException Create breakpoint : Connection refused: /192.168.0.108:8081; nest
    Suppressed: reactor.core.publisher.FluxOnAssembly$OnAssemblyException:
Error has been observed at the following site(s):
    *__checkpoint ⇢ Request to GET health [DefaultWebClient]
Original Stack Trace: <20 internal lines>
        at reactor.netty.http.client.HttpClientConnect$MonoHttpConnect$ClientTransportSubscriber.onError(HttpClientConnect.java:307) ~[r
        at reactor.netty.resources.DefaultPooledConnectionProvider$DisposableAcquire.onError(DefaultPooledConnectionProvider.java:158) ~
        at reactor.netty.internal.shaded.reactor.pool.AbstractPool$Borrower.fail(AbstractPool.java:477) ~[reactor-netty-core-1.0.19.jar:
        at reactor.netty.internal.shaded.reactor.pool.SimpleDequePool.lambda$drainLoop$9(SimpleDequePool.java:431) ~[reactor-netty-core-
        at reactor.netty.resources.DefaultPooledConnectionProvider$PooledConnectionAllocator$PooledConnectionInitializer.onError(Default
Caused by: io.netty.channel.AbstractChannel$AnnotatedConnectException: Connection refused: /192.168.0.108:8081
Caused by: java.net.ConnectException Create breakpoint : Connection refused
    at java.base/sun.nio.ch.Net.pollConnect(Native Method) ~[na:na]
    at java.base/sun.nio.ch.Net.pollConnectNow(Net.java:672) ~[na:na]
    at java.base/sun.nio.ch.SocketChannelImpl.finishConnect(SocketChannelImpl.java:946) ~[na:na] <10 internal lines>
```

收件人和抄送人也及时收到邮件了，如下图所示。



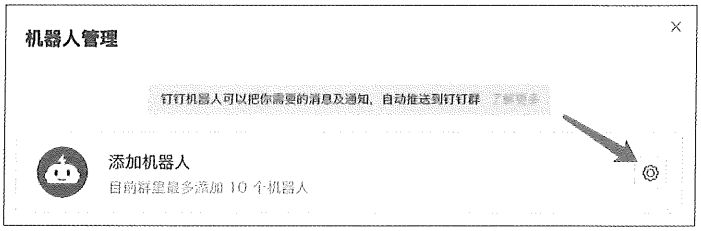
从邮件内容中可以看到应用实例下线的相关信息，为了防止敏感信息被泄露，内置的默认邮件模板不会显示实例的任何元数据信息，如果需要显示，则可以使用自定义的 Thymeleaf 模板，这里不再赘述。

邮件启动器的具体介绍及应用可以查看第 3 章邮件 Starter 章节。

3. 钉钉通知

首先需要在钉钉中创建和授权钉钉机器人，创建方法如下所示。

创建钉钉警报群，单击钉钉群设置→智能群助手→添加机器人。
具体过程如下面 4 个图所示。



机器人管理
心知天气
自动提送天气预报和预警信息
防疫精灵
新冠疫情实况和预防咨询服务
复工宝
企业复工复产提报及相关服务
阿里云Codeup
阿里云提供的代码托管服务
GitHub
基于Git的代码托管服务
极狐GitLab
基于ROR的开源代码托管软件
JIRA
出色的项目与事务跟踪工具
Travis
出色的项目与事务跟踪工具
Trello
实时的卡片墙，管理任何事情
自定义
通过Webhook接入自定义服务

机器人管理
机器人名字:
钉钉钉
* 添加到群组:
Java技术栈
* 安全设置
自定义关键词
说明文档
警报
我已阅读并同意《自定义机器人服务及免责条款》
取消
完成

钉钉机器人创建完成后会得到一个签名用的 Secret 和 Webhook 地址，然后配置在 Spring Boot Admin Server 中，如下面的配置示例所示。

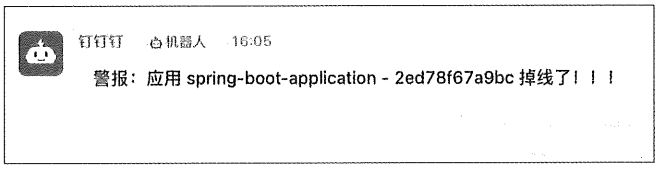
```
spring:
  mail:
    host: smtp.exmail.qq.com
    username: xxx@xxx.com
    password: xxx
    properties:
      "[mail.smtp.socketFactory.class]": javax.net.ssl.SSLSocketFactory
      "[mail.smtp.socketFactory.fallback]": false
      "[mail.smtp.socketFactory.port]": 465
      "[mail.smtp.connectiontimeout]": 5000
      "[mail.smtp.timeout]": 3000
      "[mail.smtp.writetimeout]": 5000
  boot:
    admin:
      notify:
        mail:
          enabled: true
          from: Spring Boot Admin <xxx@xxx.com>
          to: xxx@xxx.com # 多个以，分隔
          cc: xxx@xxx.com # 多个以，分隔
        dingtalk:
          enabled: true
          webhook-url: https://oapi.dingtalk.com/robot/send?access_token=xxx
          secret: xxx
          message: "警报 应用 #{instance.registration.name} - #{instance.id} 掉线了！！！
"
```

几个重要的配置参数的说明如下表所示。

参数名	描述	默认值
spring.boot.admin.notify.dingtalk.enabled	开启钉钉通知	true
spring.boot.admin.notify.dingtalk.webhook-url	钉钉机器人 Webhook 地址	
spring.boot.admin.notify.dingtalk.secret	消息签名的密钥	
spring.boot.admin.notify.dingtalk.message	要发送的通知消息，支持 SpEL 表达式	#{instance.registration.name} #{instance.id} is #{event.statusInfo.status}

具体实现逻辑见 AdminServerNotifierAutoConfiguration 和 DingTalkNotifierConfiguration 配置类，它会注册一个 DingTalkNotifier 通知实例，这里不再赘述。配置好钉钉通知参数之后，再重新启动 Spring Boot Admin Server 和 Spring Boot Admin Client，然后关闭 Spring Boot Admin Client 模拟应用掉线的场景。

不久钉钉就会收到应用掉线的警报消息，如下图所示。



如果同时开启了多个通知方式，那么会同时收到警报信息，比如笔者同时启用了邮件和钉钉通知，应用掉线时两者会同时收到警报通知。

Spring Boot Admin 监控的功能很简单，只能监控应用的状态，而不能监控中间件的健康状况，所以不能代替专业的监控系统（比如下一节要介绍的 Prometheus+Grafana 监控系统组合），但至少 Spring Boot Admin 比较轻量极，只需要启动一个服务端应用，而不需要太多复杂的组件，所以，对于那些简单的 Spring Boot 应用的监控还是够用的。

12.8 Prometheus+Grafana

12.8.1 概述

Spring Boot Actuator 为 Micrometer 提供了依赖管理和自动配置，Micrometer 是一个支持众多监控系统的监控系统门面，具有和 Slf4j 一样的性质，Micrometer 可以支持以下监控系统的指标数据收集：AppOptics、Atlas、Datadog、Dynatrace、Elastic、Ganglia、Graphite、Humio、Influx、

JMX、KairosDB、New Relic、OpenTelemetry、Prometheus、SignalFx、Simple（in-memory）、Stackdriver、StatsD、Wavefront。

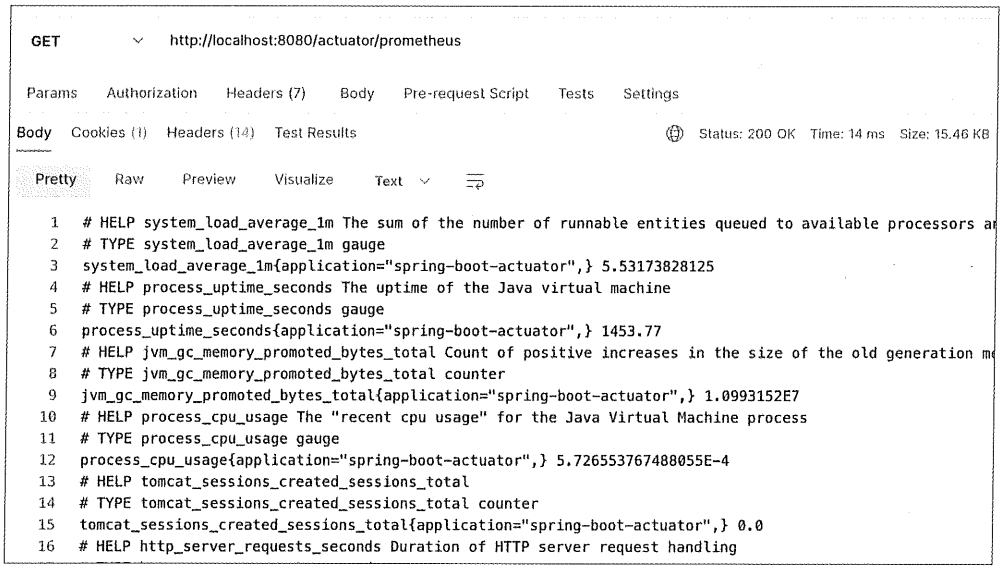
Spring Boot 自动配置了一个指标注册表 MeterRegistry，只需要在类路径中添加一个 micrometer-registry-{system} 依赖即可，自动配置类会根据类路径下不同的监控系统依赖配置指标注册表 MeterRegistry。

本节将以当前主流的 Prometheus 监控系统为基础进行介绍，Prometheus 是一个开源的应用监控系统，也是一个时序数据库，它可以从应用收集指标数据并显示结果，并且在达到指定条件时触发警报，现在已经成为 Cloud Native Computing Foundation（CNCF）云原生计算基金会的项目。

12.8.2 Prometheus 指标暴露

1. pull 方式

对于 Prometheus，Spring Boot 提供了一个 /actuator/prometheus 端点，如下图所示。



Prometheus 会通过此端点来抓取单个应用实例的指标，该端点默认是不暴露的，需要手动暴露，之前的章节中已经暴露为“*”了，所以这里不再单独暴露了。

添加基于 Prometheus 的 Micrometer 注册依赖，如下面的依赖配置所示。

```
<dependency>
    <groupId>io.micrometer</groupId>
```

```
    <artifactId>micrometer-registry-prometheus</artifactId>
    <version>${micrometer-registry-prometheus.version}</version>
</dependency>
```

引入该依赖后 Spring Boot 就会自动配置，对应的自动配置类为 PrometheusMetricsExportAutoConfiguration，对应的参数绑定类为 PrometheusProperties，可以通过一系列 management.prometheus.metrics.export.* 参数配置 Prometheus 指标。

2. push 方式

对于 Spring Boot 提供的端点，Prometheus 是以 pull 的方式获取指标数据的，Spring Boot 也支持以 push 的方式将指标数据推送给 Prometheus，这对于时间不够长而无法被抓取的临时任务或者批处理任务特别有用。

推送指标数据需要使用 Prometheus 的 Pushgateway 模块，先引入应用的依赖：

```
<dependency>
    <groupId>io.prometheus</groupId>
    <artifactId>simpleclient_pushgateway</artifactId>
    <version>${simpleclient_pushgateway.version}</version>
</dependency>
```

默认不会启用 pushgateway 功能，需要手动配置以启用 pushgateway 功能：

```
management:
  prometheus:
    metrics:
      export:
        pushgateway:
          enabled: true
```

Spring Boot 3.0 新变化：

以 management.metrics.export.<product> 管理的参数需要使用新的 management.<product>.metrics.export 参数形式。

在 PrometheusMetricsExportAutoConfiguration 自动配置类中会自动注册一个 PrometheusPushGatewayManager 类型的 Bean，通过这个 Bean 可以将应用的指标推送给 Prometheus Pushgateway。

既可以通过一系列 management.prometheus.metrics.export.pushgateway.* 参数配置 PrometheusPushGatewayManager，也可以自定义一个覆盖默认自动配置的 PrometheusPush- GatewayManager。

12.8.3 Prometheus 环境搭建

本节以 CentOS Linux 为例搭建 Prometheus 环境，首先下载并安装 Prometheus，如下面的参考命令：

```
$ wget https://github.com/prometheus/prometheus/releases/download/v2.37.0/prometheus-2.37.0.linux-amd64.tar.gz

$ tar -zxvf prometheus-2.37.0.linux-amd64.tar.gz
```

然后进入解压后的目录，打开 prometheus.yml 配置文件，修改并添加以下配置：

```
...

scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped from this config.
  - job_name: "prometheus"
    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.
    static_configs:
      - targets: ["localhost:9090"]
  - job_name: "javastack"
    # metrics_path defaults to '/metrics'
    metrics_path: "/actuator/prometheus"
    # scheme defaults to 'http'.
    static_configs:
      - targets: ["localhost:8080"]
```

这里配置了两个抓取任务，第一个是 Prometheus 默认提供的，可以忽略；第二个是 Spring Boot Actuator 应用，也就是我们要监控的 Spring Boot 应用，它需要使用 metrics_path 配置 Spring Boot 提供的 /actuator/prometheus 端点，这也是通过 pull 的方式获取应用指标数据的来源。

启动 Prometheus，参考命令如下：

```
$ ./prometheus --config.file=prometheus.yml &
```

Prometheus 提供了优雅关闭的方式，需要先指定参数启动，参考命令如下：

```
./prometheus --config.file=prometheus.yml --web.enable-lifecycle &
```

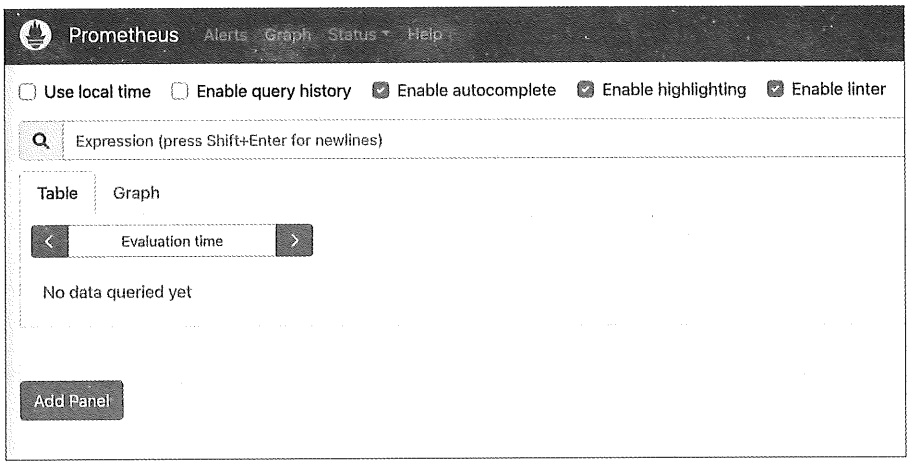
然后通过发送一个 POST 请求优雅关闭 Prometheus，参考命令如下：

```
curl -X POST http://localhost:9090/-/quit
```

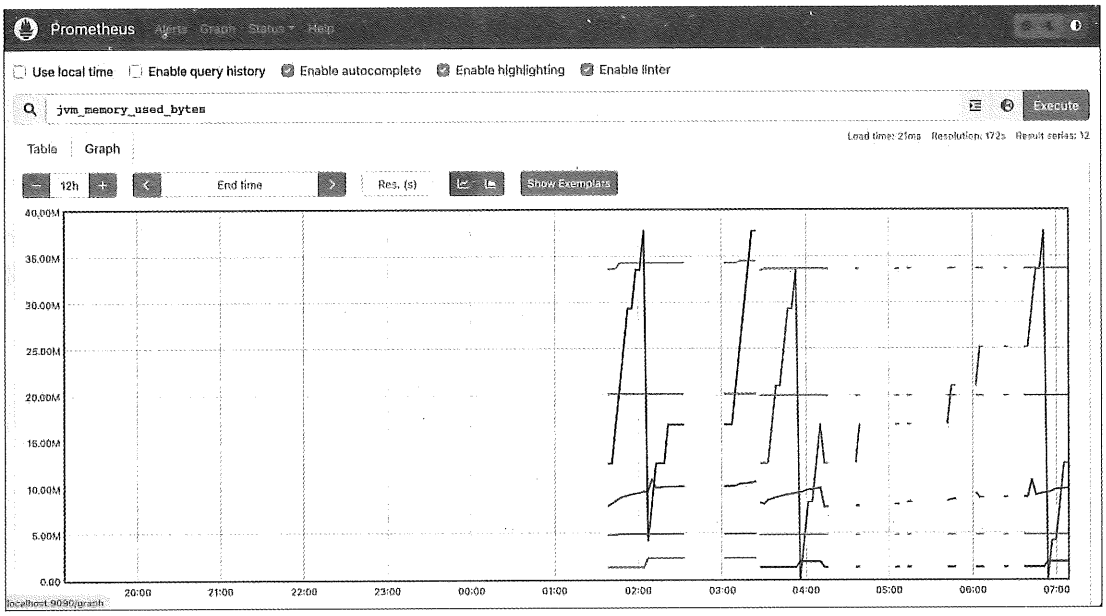
启动 Prometheus 后默认绑定为 9090 端口，访问 Prometheus 控制台，地址如下：

http://localhost:9090/

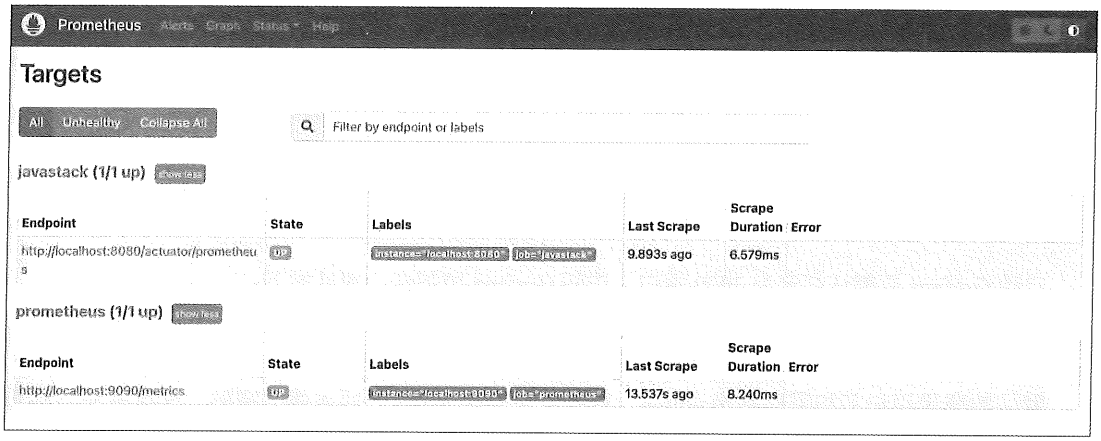
Prometheus 首页如下图所示。



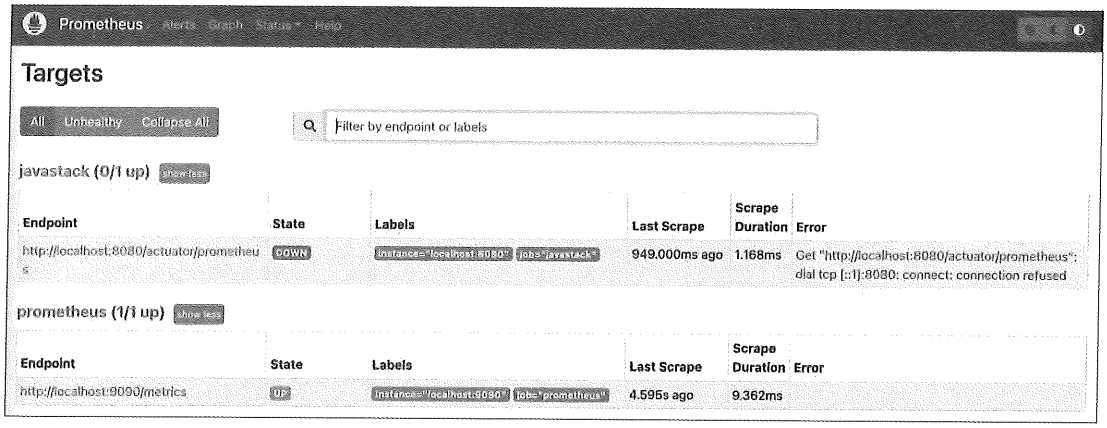
在首页，可以根据条件检索并展示一段时间内的指标数据，如下图所示。



通过 Status → Targets 菜单进入监控的实例，如下图所示。



可以看到我们配置好的监控实例都是正常状态的，假如现在关闭 Spring Boot Actuator 应用，过几秒再刷新一下，可以看到 Spring Boot Actuator 应用已经是下线状态了，如下图所示。



12.8.4 Grafana 数据可视化

1. Grafana 环境搭建

从上一节可以看到，Prometheus 的数据展示能力非常一般，也不是很直观，所以通常会结合 Grafana 可视化平台一起使用。Grafana 是一个开源的数据可视化平台，可以对指标数据进行可视化、查询、提醒等操作，可以帮助我们更好地了解指标数据，它支持来自不同数据源的数据，比如 Prometheus、Loki、Elasticsearch、InfluxDB、Postgres 等，并且提供了各种丰富的面板以展示不同类型的指标数据。

本节以 CentOS 为例搭建 Grafana 环境，首先下载并安装 Grafana，参考命令如下所示。

```
$ wget https://dl.grafana.com/enterprise/release/grafana-enterprise-9.0.7.linux-amd64.tar.gz$ tar -zxvf grafana-enterprise-9.0.7.linux-amd64.tar.gz
```

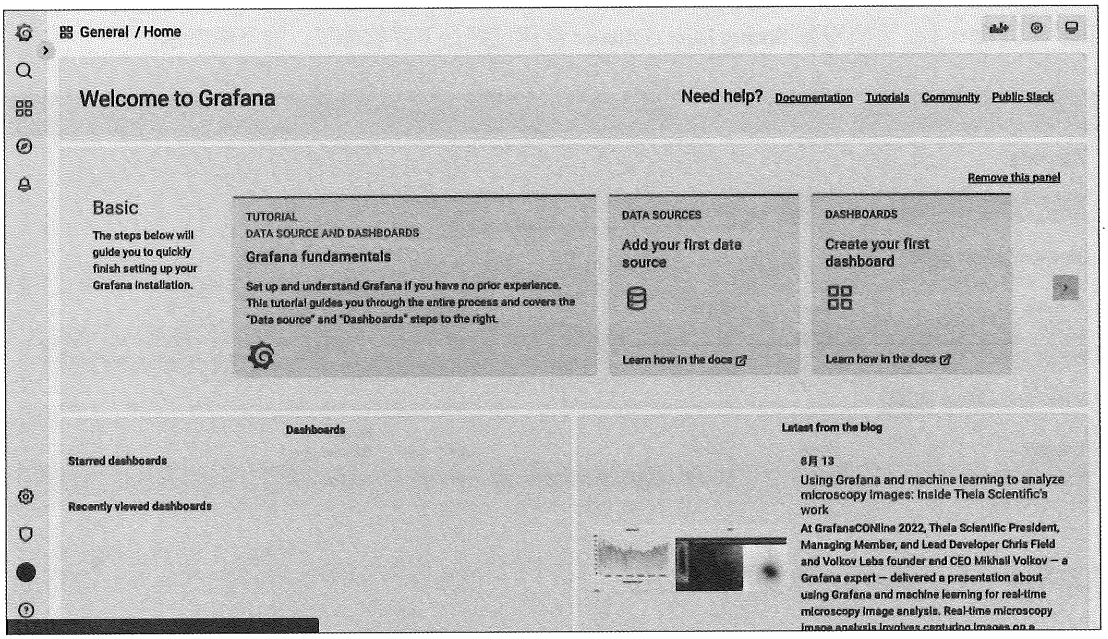
进入解压后的 bin 目录启动 Grafana，参考命令如下：

```
$ ./grafana-server &
```

启动 Grafana 后默认绑定为 3000 端口，访问 Grafana 控制台，地址如下：

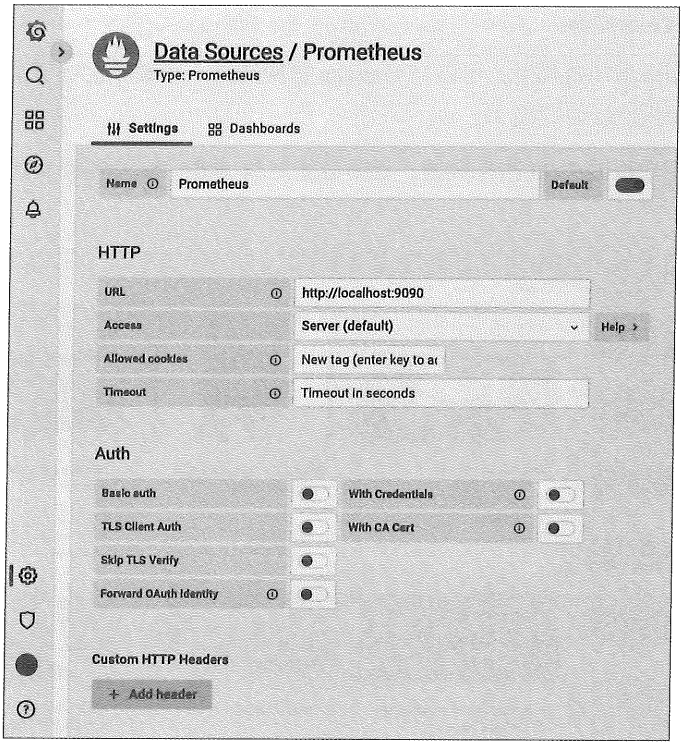
```
http://localhost:3000/
```

默认的登录用户名和密码都是 admin，首次登录 Grafana 后会要求修改密码，修改密码后再重新登录，登录成功后进入 Grafana 主界面，如下图所示。

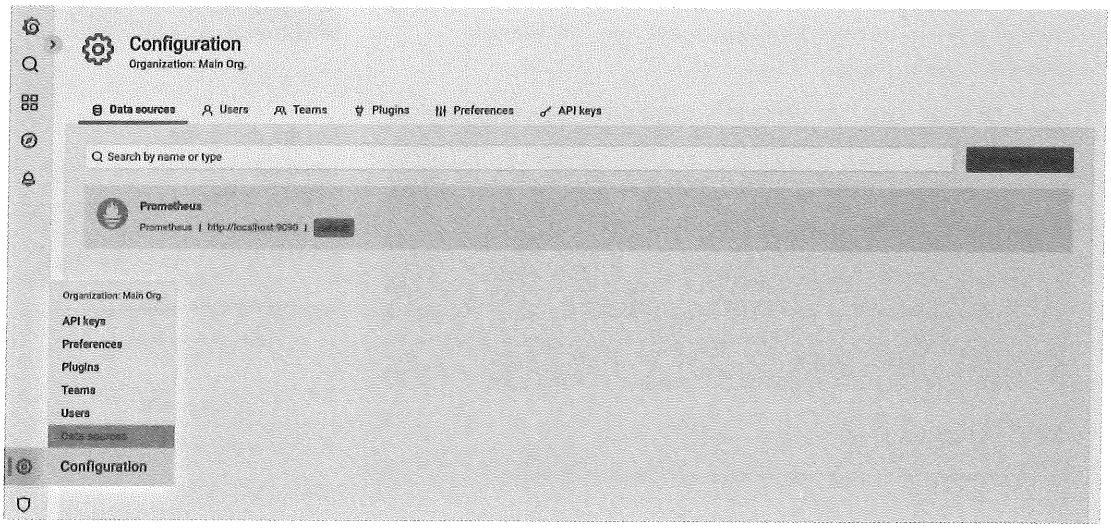


2. 添加 Prometheus 数据源

搭建好 Grafana 之后，需要 Prometheus 的指标数据来源，单击首页的 DATA SOURCES 面板项，然后选择新建一个 Prometheus 数据源，如下图所示。



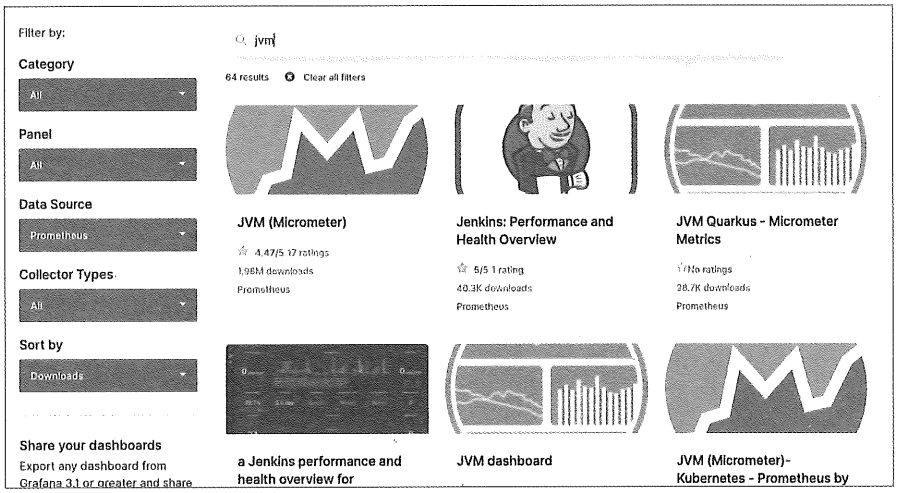
这里主要是输入 Prometheus 的 URL：http://localhost:9090。部署到生产环境中需要有 Auth 认证信息，Prometheus 数据源保存成功后，单击设置按钮→ Data sources 可以查询 / 新建数据源，如下图所示。



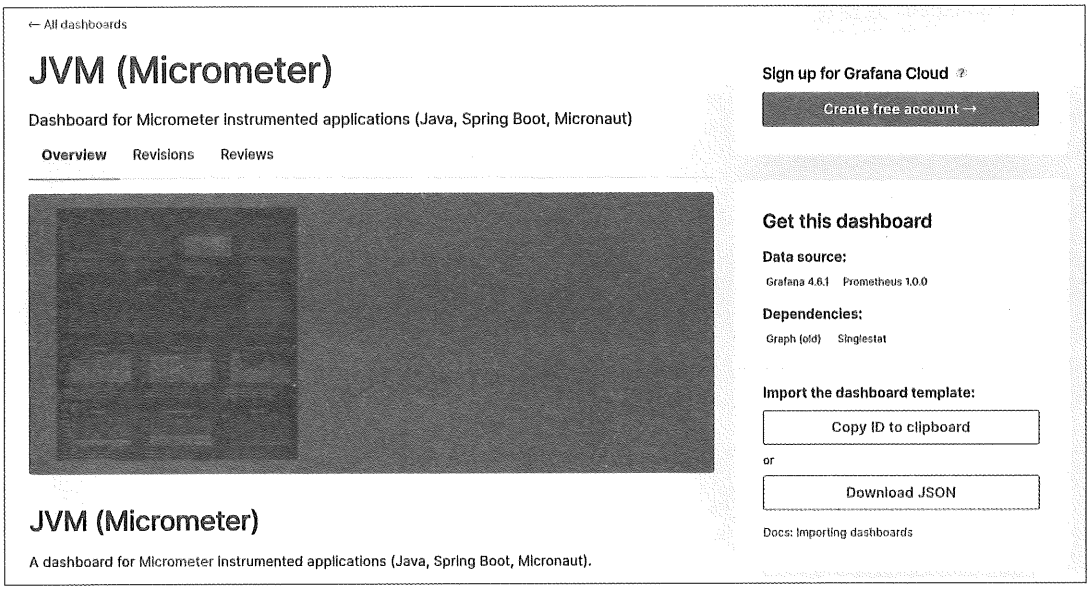
3. 添加 JVM 数据面板

Prometheus 数据源添加成功后，可以在 Grafana 官网选择一个数据面板以展示数据源中的指标数据，官方数据的面板地址为链接 21。

比如我们想展示应用的 JVM 指标数据，根据数据源和关键字条件搜索一个 JVM 数据面板，如下图所示。

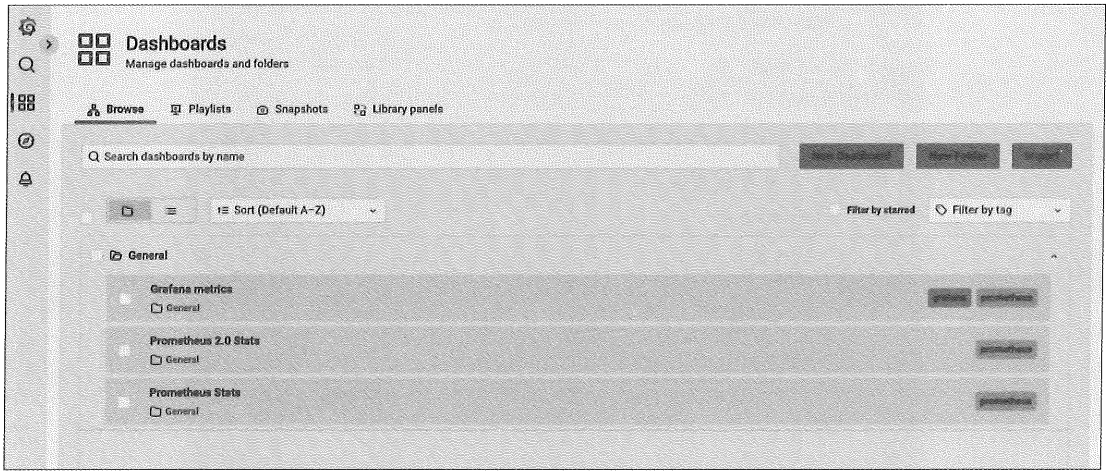


然后单击 JVM(Micrometer) 项进入该数据面板，如下图所示。

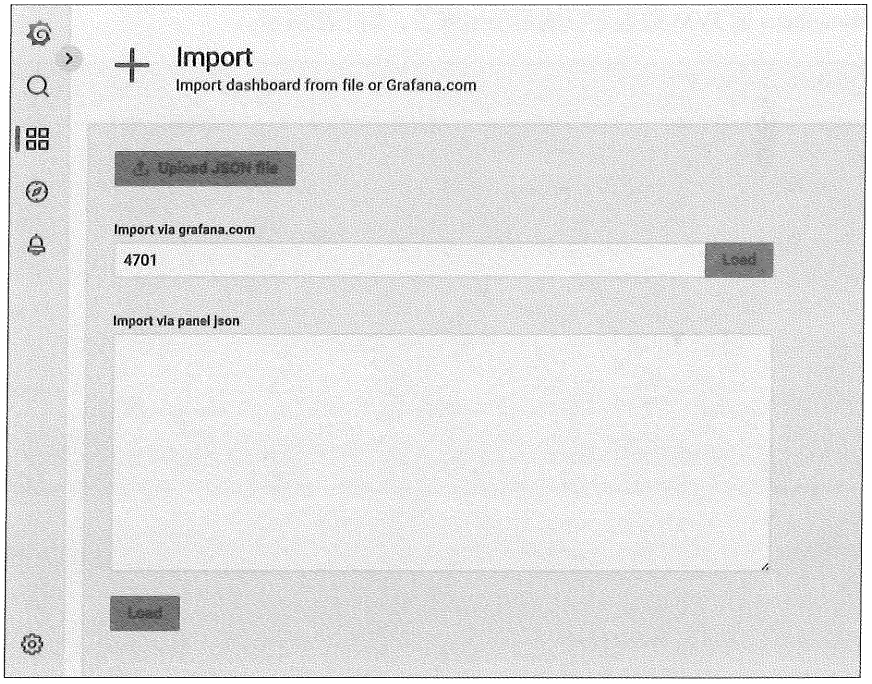


数据面板有两种方式可以集成到 Grafana 可视化平台，一是复制面板 ID 再导入 Grafana，二是下载 JSON 文件再导入 Grafana，这里使用第一种方式复制面板 ID 再导入 Grafana。

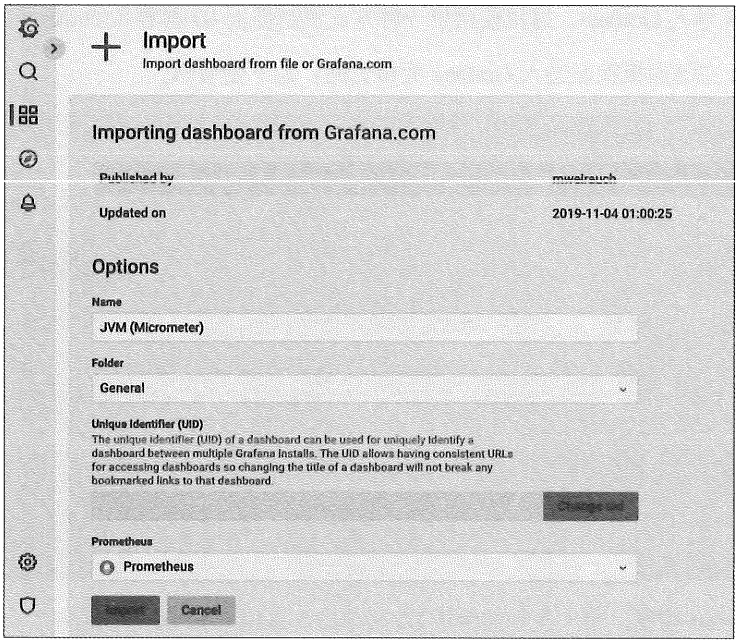
单击 Grafana 左侧面板按钮进入 Grafana 面板页面，如下图所示。



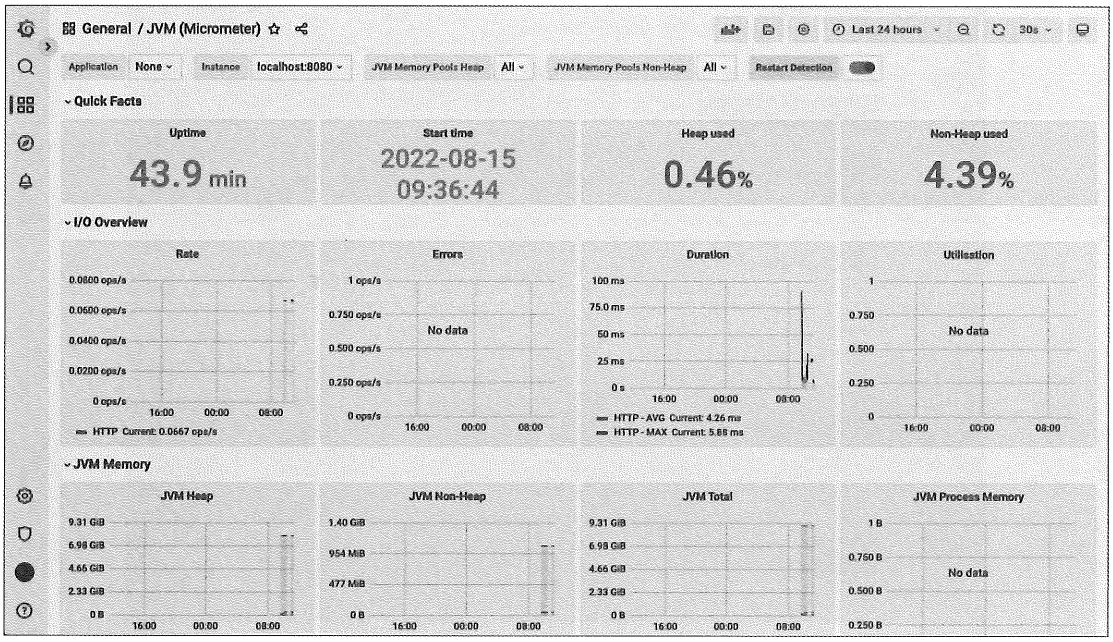
然后单击 Import 按钮，输入复制的面板 ID 后单击 Load 按钮，如下图所示。



在导入界面下选择 Prometheus 数据源，然后单击 Import 按钮，如下图所示。



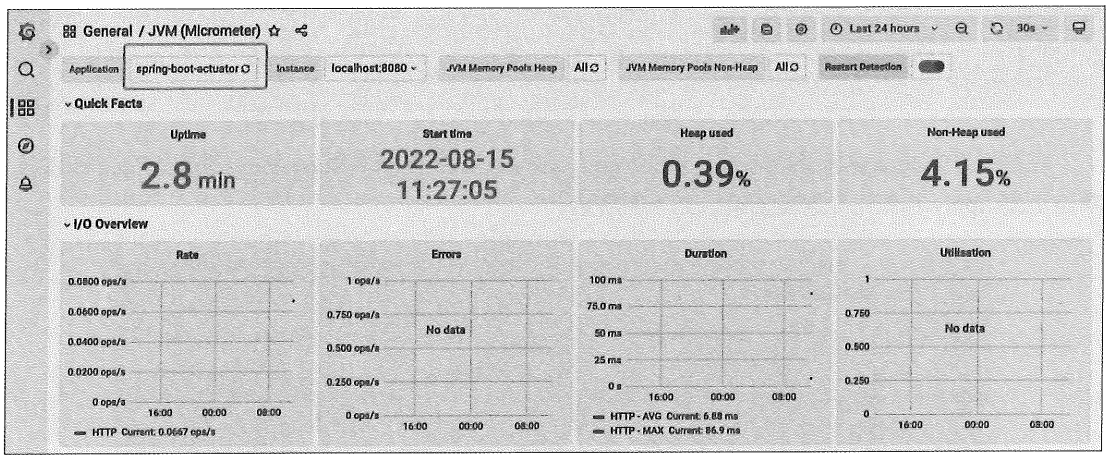
基于 Prometheus 的 JVM 数据面板就呈现出来了，如下图所示。



左上方的 Application显示为 None，需要在应用配置中配置一下，如下面的配置示例所示。

```
spring:
  application:
    name: spring-boot-actuator

management:
  metrics:
    tags:
      application: ${spring.application.name}
```

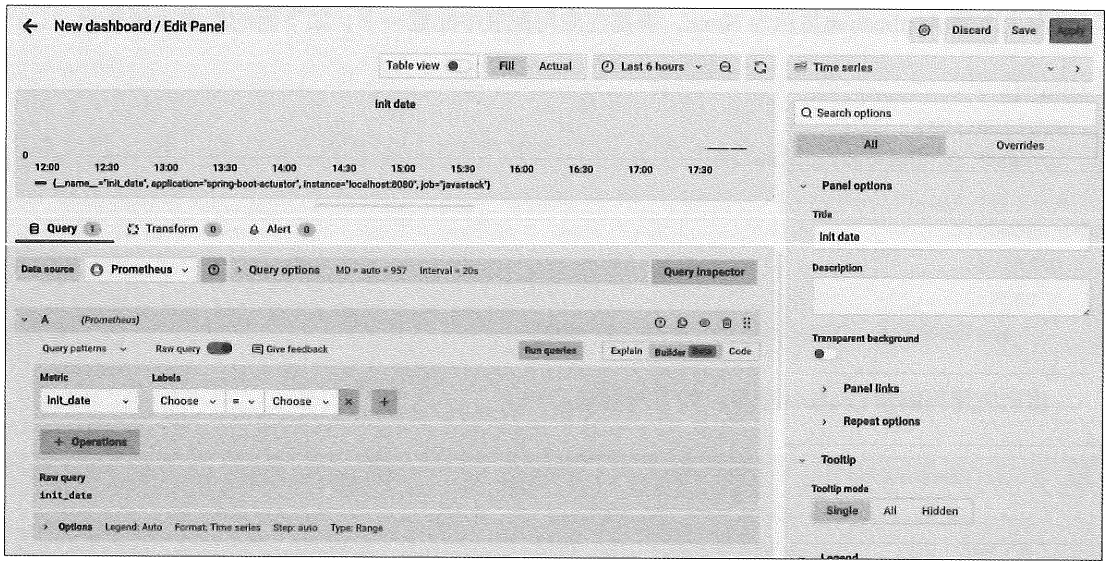
重启应用再刷新 Grafana 面板，如下图所示。



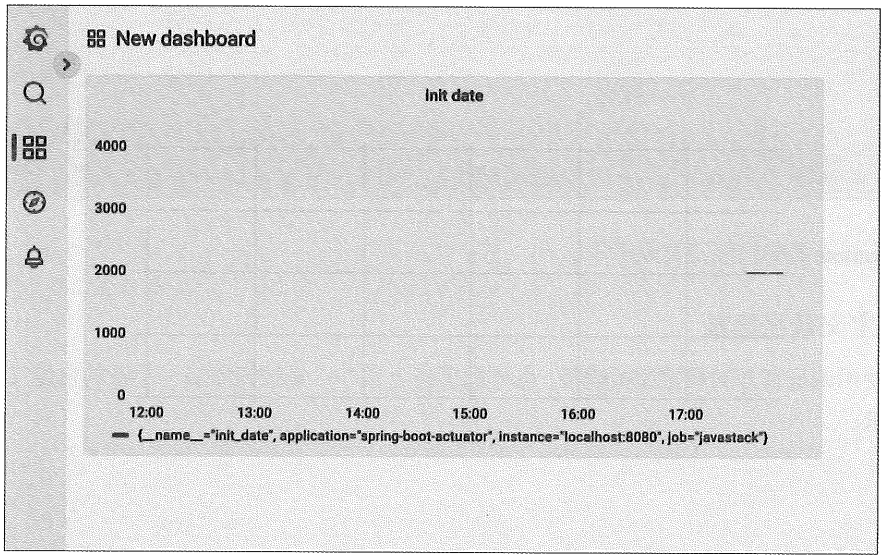
Application 名称已经正常显示了。

4. 自定义数据面板

以上使用的是官方的数据面板模板，也可以单击上方“+”号按钮添加一个新的自定义数据面板，如下图所示。



在左下方Metric处选择之前自定义好的init_date指标，然后单击Run queries按钮查询指标数据，单击上面Apply按钮可以预览面板，如下图所示。



再单击上方的保存按钮可以保存自定义面板。

12.8.5 监控报警

使用 Prometheus+Grafana 监控报警一般有以下几种方案：

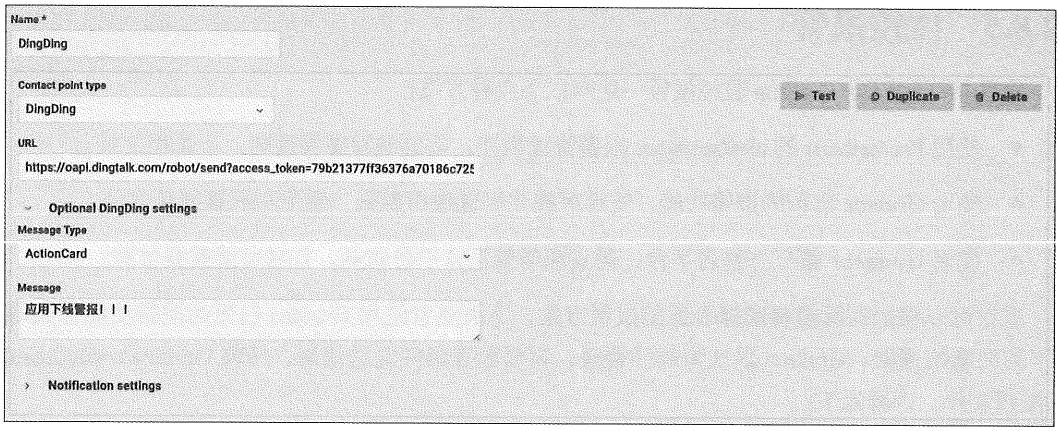
- 使用 Prometheus 的 alertmanager 报警管理组件，需要编写报警规则，不是很友好。
- 使用 Grafana 自带的报警功能，支持可视化配置报警规则，使用和配置简单方便。
- 使用 Grafana+ 第三方警告平台，需要注册第三方平台账号，并且可能需要付费。

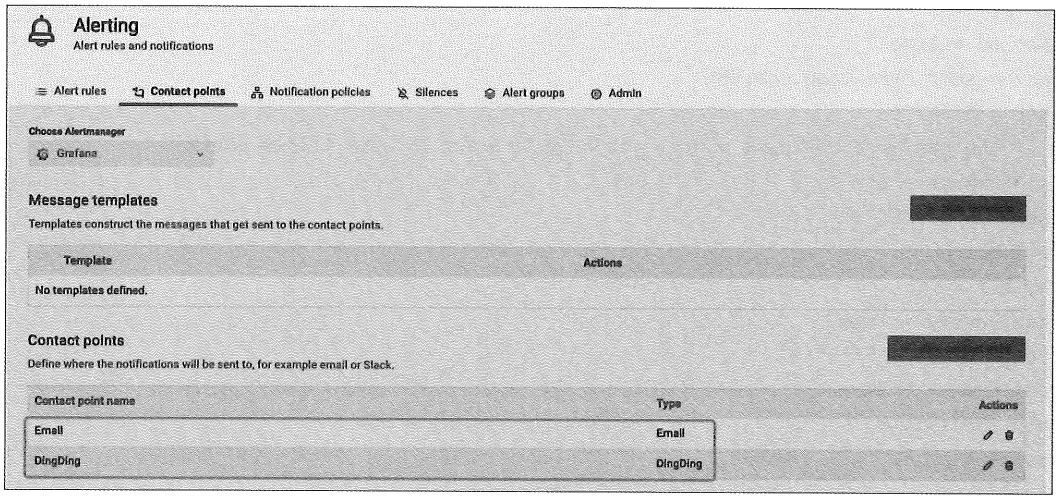
读者可以根据实际需要选择合适的报警方案，下面笔者使用 Grafana 自带的报警功能来实现应用下线时警报通知。Grafana 默认为邮件警报，需要配置邮件发送信息，修改 Grafana/confi/defaults.ini 配置文件，内容如下：

```
#################################### SMTP / Emailing ##########################
[smtp]
enabled = true
host = smtp.exmail.qq.com:465
user = ********
# If the password contains # or ; you have to wrap it with triple quotes.
Ex """#password;"""
password = ********
cert_file =
key_file =
skip_verify = true
from_address = ********
from_name = Grafana
ehlo_identity =
startTLS_policy =

[emails]
welcome_email_on_sign_up = false
templates_pattern = emails/*.html, emails/*.txt
content_types = text/html
```

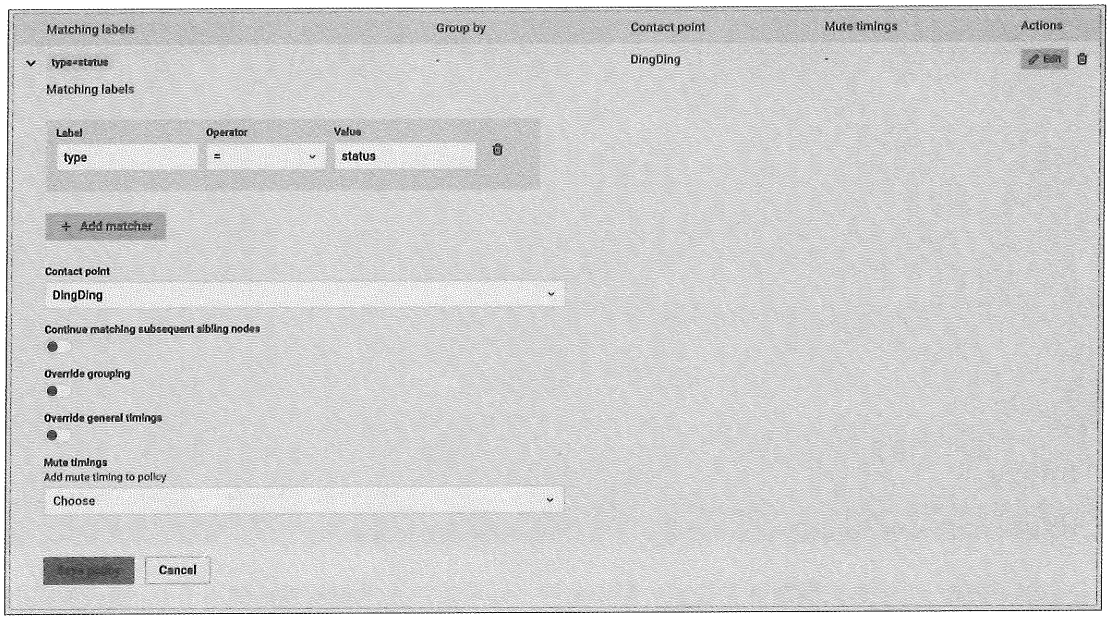
修改 SMTP 部分的配置，然后重启 Grafana 即可。

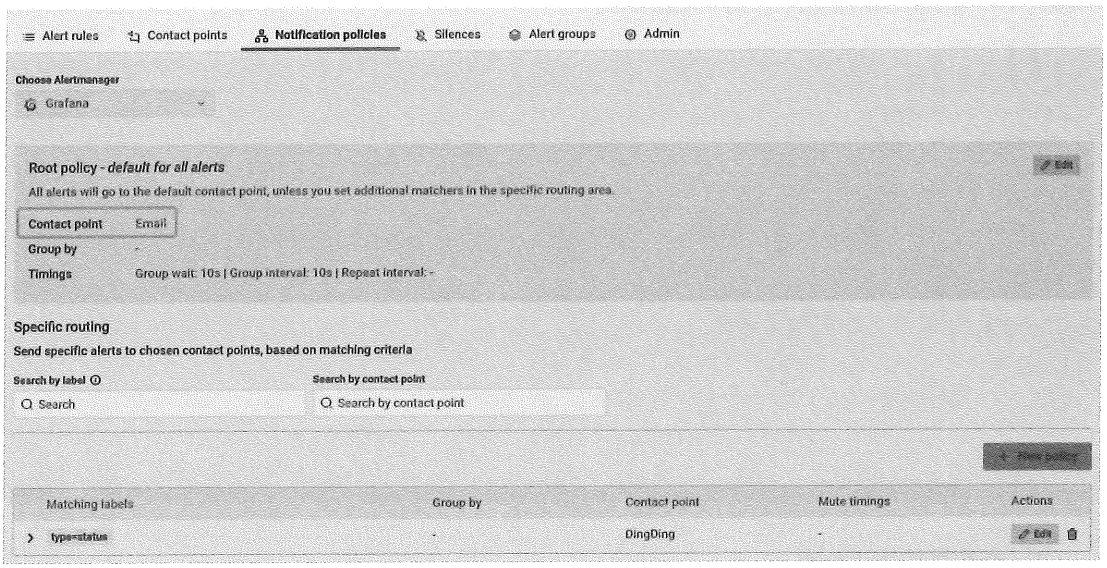
进入 Alerting → Contact points 面板，默认只有邮件警报，这里添加一个钉钉通知，如下面两个图所示。





然后进入 Alerting → Notification policies 面板，这里设置所有警报规则为邮件通知方式，并且添加了一个自定义的警报匹配规则，设置标签值 type=status 的警报规则都使用钉钉通知，如下面两个图所示。

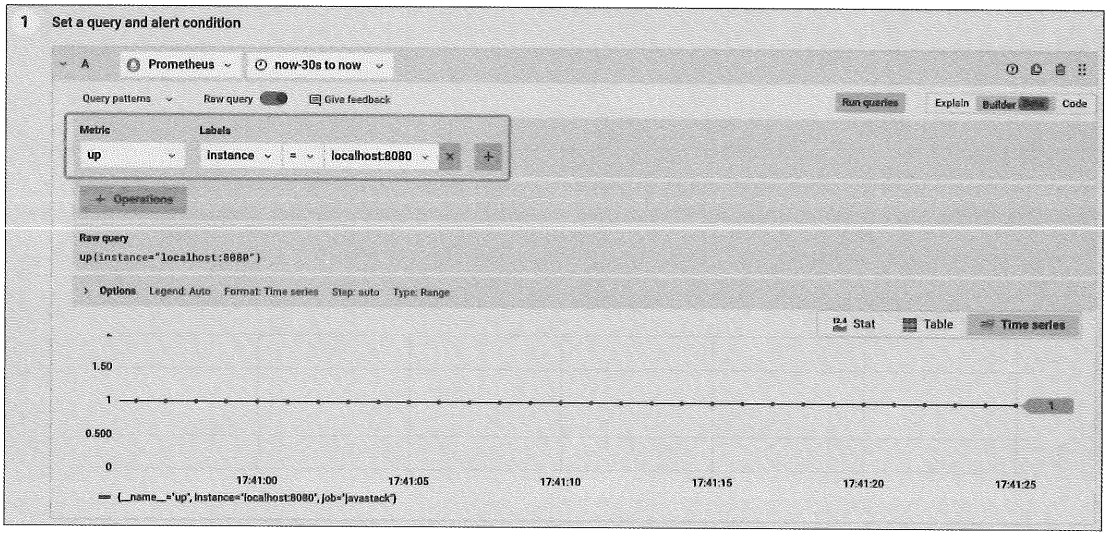




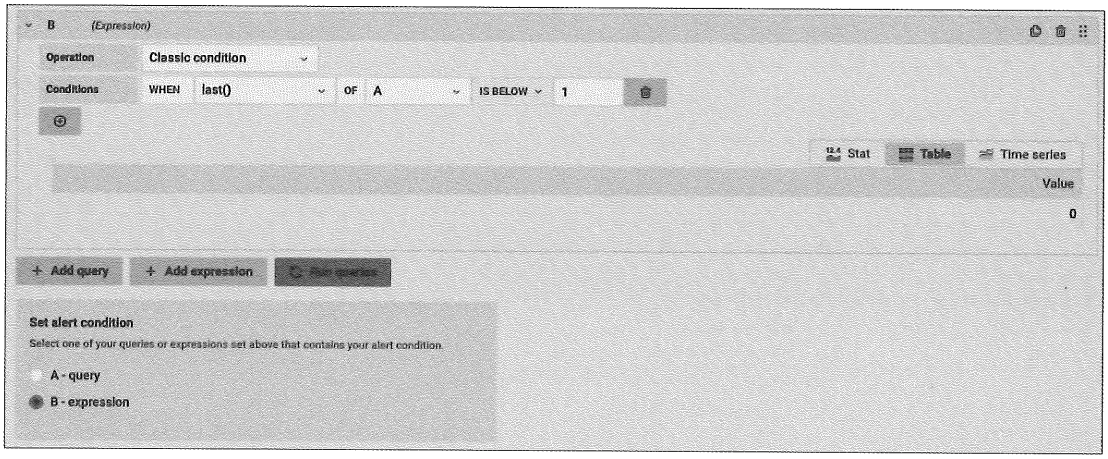
最后进入 Alerting → Alert rules 面板，添加一个应用下线警报规则，如下面的流程所示。

1）设置警报触发条件

添加一个基本指标规则 A，用于获取实例的在线状态，如下图所示。



然后添加一个表达式规则 B 并设置为警报条件，如果条件 A 的最新返回值小于 1，则代表实例下线了（1：在线；0：下线），如下图所示。



2）设置警报评估行为

设置每 10 秒检测一下警报条件，如果符合警报条件，则延迟 15 秒再进行警报通知，如下图所示。